# Machine Learning for Kira IBP Reduction

Seed selection and elimination ordering

October 8, 2026

**Abstract**

Feynman integral calculations are an important component of multiloop amplitude calculations in quantum field theory. In Quantum Chromodynamics (QCD), multiloop amplitudes and their associated Feynman integrals arise in higher-order perturbative calculations and can become increasingly complex at higher loop orders. Recent progress has automated these calculations to some extent using the framework Kira, which organizes target integrals into integral families and sectors, generate algebraic relations among the integrals by using integration by parts (IBP) identities, and reduce the resulting system to expressions in terms of a finite set of master integrals. We aim to explore the application of System 1 machine learning models to accelerate the IBP reduction process by investigating if lightweight learned models can improve seed selection. These models can predict the most useful integrals from which to generate IBP relations while minimizing redundant calculations. We will also explore System 1 models for learning efficient reduction and elimination sequences to minimize the number of algebraic operations required to express target integrals in terms of master integrals. Because the machine learning models will serve as decision-makers while the underlying IBP reduction and verification remain exact, these models can improve computational efficiency without compromising the correctness of the resulting reductions.

# 1 Introduction

Feynman integrals arise in higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes. In QCD, these corrections involve quark and gluon loops and contribute to predictions for cross sections and decay rates. Calculating a loop contribution requires integration over the independent internal momenta that remain after imposing momentum conservation [1].

The scalar integrals that appear in an amplitude have the general form

$$\mathcal{J} = \left(\prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}\right) \frac{\mathcal{N}(\boldsymbol{\ell}, \boldsymbol{p})}{\prod_{i=1}^{n_{\mathrm{phys}}} D_i^{a_i}}, \qquad D_i = q_i^2 - m_i^2 + \mathrm{i}0. \tag{1}$$

Here $L$ is the number of loops and $\boldsymbol{\ell} = (\ell_1, \ldots, \ell_L)$ is the list of independent loop momenta over which we integrate. The number of independent external momenta is $E$, and we keep these momenta, $\boldsymbol{p} = (p_1, \ldots, p_E)$, fixed during integration. The numerator $\mathcal{N}$ is a polynomial obtained from the spin and tensor algebra; we leave its mass dependence implicit in the notation. The number of physical denominators is $n_{\mathrm{phys}}$, and each $D_i$ corresponds to an internal particle line with momentum $q_i$ and mass $m_i$. The positive integer $a_i$ is the power of $D_i$ in the denominator, and $q_i$ is a fixed linear combination of loop and external momenta, with $q_i^2 = q_i \cdot q_i$ denoting the Lorentz scalar product.

The integration dimension is $d = 4 - 2\epsilon$, where $\epsilon$ is the regulator used to define divergent integrals, so the result depends on $\epsilon$ as well as the external momenta and masses. The factor $1/(\mathrm{i}\pi^{d/2})$ is our normalization convention for each integration measure, with $\mathrm{i}^2 = -1$. In each denominator, i0 is an infinitesimal positive imaginary term that specifies how to integrate past a propagator pole, where $q_i^2 = m_i^2$.

Higher loop orders produce collections of integrals with related numerators and denominators and algebraic identities can be used to express them in terms of an independent set of master integrals. Kira automates this reduction [2], after which the calculation requires the values of the master integrals. To explain how the reduction works and where machine learning (ML) could speed it up, we first define the notation used in this paper, then derive the identities and describe how Kira solves the resulting equations.

# 2 The mathematical form of Feynman integrals

To label the integrals related to Eq. (1), we specify their momentum dependence and write the numerator and denominator powers as a vector of integers. This vector identifies each integral as an unknown in the reduction equations.

## 2.1 Loop momenta, propagators and regularization

After choosing how the loop and external momenta flow through the internal lines, we write each physical denominator as

$$D_i = q_i^2 - m_i^2 + \mathrm{i}0, \qquad q_i = \sum_{k=1}^{L} \alpha_{ik} \ell_k + \sum_{\beta=1}^{E} \eta_{i\beta} p_\beta. \tag{2}$$

The constants $\alpha_{ik}$ and $\eta_{i\beta}$ specify which loop and external momenta make up $q_i$, with indices $k, j$ for loop momenta and $\beta, \gamma$ for external momenta. Squaring $q_i$ gives the scalar products $\ell_k \cdot \ell_j$, $\ell_k \cdot p_\beta$ and $p_\beta \cdot p_\gamma$, and those containing loop momenta vary during integration. The

vector $\boldsymbol{x}$ contains the independent external scalar products and squared masses, which are the kinematic parameters we keep fixed during integration.

Loop integrals can diverge at large momenta or when massless momenta become small or parallel. Dimensional regularization defines these integrals at values of $d$ for which they converge and extends the result by analytic continuation to $d = 4 - 2\epsilon$, using additional regulators where necessary. In an expansion around $\epsilon = 0$, the divergences appear as negative powers of $\epsilon$, which renormalization and infrared cancellation in the observable address later in the calculation [1]. Having specified the regulated measure and momentum dependence, we can now use the powers of the numerator and denominator factors to label individual integrals.

### 2.2 A common notation for an integral family

We add auxiliary expressions to the physical denominators so that we can write every required loop-momentum scalar product in terms of the list $D_1, \ldots, D_N$ and the fixed parameters $\boldsymbol{x}$. Here $N$ is the number of expressions in the list, and the auxiliary expressions allow us to represent numerator factors. Writing the numerator in terms of this list expresses $\mathcal{J}$ as a linear combination of integrals of the form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \prod_{i=1}^{N} D_i^{-a_i}, \qquad \int_\ell \equiv \prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}, \qquad \boldsymbol{a} = (a_1, \ldots, a_N) \in \mathbb{Z}^N. \tag{3}$$

Here $\int_\ell$ is shorthand for the loop integration measure and $\boldsymbol{a}$ is the vector of $N$ integer powers, with $\mathbb{Z}$ denoting the set of integers. These powers can be positive, zero or negative, so we can write numerator and denominator factors in the equivalent form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \frac{\prod_{i:a_i<0} D_i^{-a_i}}{\prod_{i:a_i>0} D_i^{a_i}}. \tag{4}$$

A positive index places $D_i^{a_i}$ in the denominator, a negative index places $D_i^{-a_i}$ in the numerator, and a zero index removes the factor, with an empty product equal to one. Auxiliary expressions used only as numerators therefore have nonpositive indices. An irreducible scalar product is a numerator scalar product that cannot be written as a linear combination of the physical denominators and terms depending only on $\boldsymbol{x}$ [3].

An integral family contains the integrals obtained by varying $\boldsymbol{a}$ for a fixed list $D_1, \ldots, D_N$ and fixed kinematics. Within this family, the target list specifies the integrals that the amplitude requires, and grouping family members by their positive indices organizes them into sectors.

### 2.3 Sectors and the organization of related integrals

A sector is a set of integrals with the same positive-index positions, so they contain the same denominators. We assign a label to each sector using

$$\theta_i(\boldsymbol{a}) = \begin{cases} 1, & a_i > 0, \\ 0, & a_i \leq 0, \end{cases} \qquad S(\boldsymbol{a}) = \sum_{i=1}^{N} 2^{i-1} \theta_i(\boldsymbol{a}). \tag{5}$$

The indicator $\theta_i$ is one when denominator $i$ is present and zero otherwise, and $S$ is an integer that identifies this set of denominators. Integrals with different powers of the same denominators belong to the same sector because $S$ depends only on which indices are positive. Removing one or more denominators gives a subsector containing a proper subset of the original sector's denominators.

Scaleless sectors contain no nonzero mass or kinematic scale and therefore vanish in dimensional regularization, while changes of loop integration variables can yield symmetry relations between integrals. Kira uses these zero sectors and symmetry mappings to reduce the number of unknowns [2]. We can then use the sector and index labels to write the identities that relate the remaining family members.

# 3 From integration to algebraic reduction

To express the targets in terms of master integrals, we generate relations between family members through integration by parts and solve the resulting equations. Evaluating the master integrals then gives the target integral values needed to calculate the amplitude.

## 3.1 How integration by parts produces relations

For differentiable functions $f(z)$ and $g(z)$ of a real variable $z$, the product rule gives the integration identity between endpoints $z_0, z_1$:

$$\int_{z_0}^{z_1} f'(z)g(z)\,\mathrm{d}z = [f(z)g(z)]_{z_0}^{z_1} - \int_{z_0}^{z_1} f(z)g'(z)\,\mathrm{d}z. \tag{6}$$

Primes denote derivatives with respect to $z$, and the brackets denote the endpoint difference $f(z_1)g(z_1) - f(z_0)g(z_0)$. To apply this product rule to loop-momentum derivatives, we derive the integration-by-parts (IBP) identity where the regulated integral has a vanishing boundary term and then extend it by analytic continuation [4, 3]. For a loop momentum $\ell_k$ and a vector $v \in \{\ell_1, \ldots, \ell_L, p_1, \ldots, p_E\}$, the identity is

$$0 = \int_\ell \frac{\partial}{\partial \ell_k^\mu} \left( v^\mu \prod_{i=1}^{N} D_i^{-a_i} \right). \tag{7}$$

The index $\mu$ is a spacetime index, and a repeated Lorentz index means that we sum over components using the spacetime metric. The derivative acts on both $v^\mu$ and the factors $D_i^{-a_i}$, so the product rule gives

$$0 = \int_\ell \left[ \frac{\partial v^\mu}{\partial \ell_k^\mu} - \sum_{i=1}^{N} a_i \frac{v^\mu (\partial D_i / \partial \ell_k^\mu)}{D_i} \right] \prod_{j=1}^{N} D_j^{-a_j}. \tag{8}$$

The first term equals $d$ for $v = \ell_k$ and vanishes when $v$ is an external momentum or another independent loop momentum. Differentiating an inverse propagator produces scalar products, which we express through the family basis to convert each term into an integral with shifted indices:

$$\sum_{j=1}^{n_\lambda} c_{\lambda j}(\boldsymbol{a}; \boldsymbol{x}, \epsilon) I_{\boldsymbol{a}+\boldsymbol{\Delta}_{\lambda j}}(\boldsymbol{x}, \epsilon) = 0. \tag{9}$$

Here $\lambda$ identifies the chosen pair $(k, v)$, $n_\lambda$ is the number of terms, and $\boldsymbol{\Delta}_{\lambda j}$ is the integer change in the index vector for term $j$. Differentiation gives the coefficient $c_{\lambda j}$, which depends on the integral indices and the parameters $\boldsymbol{x}, \epsilon$ [4].

The $L$ choices of differentiated momentum and $L+E$ choices of $v$ give $L(L+E)$ basic identities at each index vector, some of which may depend linearly on the others. Further relations follow from Lorentz invariance because a common rotation or Lorentz boost of all external momenta preserves a scalar integral, giving the infinitesimal identity

$$\sum_{\beta=1}^{E} \left( p_\beta^\nu \frac{\partial}{\partial p_{\beta\mu}} - p_\beta^\mu \frac{\partial}{\partial p_{\beta\nu}} \right) I_{\boldsymbol{a}} = 0. \tag{10}$$

Here $\mu, \nu$ are free spacetime indices, meaning that the identity holds for each pair of components, and the derivatives act on the external-momentum dependence of the integral. Contracting these indices with external momenta gives scalar relations [5] that join the IBP and symmetry identities to form the reduction system.

### 3.2 From relations to master integrals

Let $n_I$ be the number of distinct integrals in the generated equations and $n_{\text{eq}}$ the number of equations. We arrange these integrals in a column vector so that the system takes the matrix form

$$A(\boldsymbol{x}, \epsilon)\, \boldsymbol{I} = 0, \qquad \boldsymbol{I} = (I_{\boldsymbol{a}^{(1)}}, \ldots, I_{\boldsymbol{a}^{(n_I)}})^{\mathsf{T}}. \tag{11}$$

The superscript $\mathsf{T}$ denotes transpose, so $\boldsymbol{I}$ is a column vector of integrals whose dependence on $\boldsymbol{x}, \epsilon$ we leave implicit here. The matrix $A$ has $n_{\text{eq}}$ rows and $n_I$ columns, with each entry $A_{ij}$ giving the coefficient of integral $j$ in equation $i$. Solving this system expresses dependent integrals in terms of an independent set. This set is a master basis if the reduction relations express every required integral as a linear combination of its members. For the standard class of Feynman integrals at fixed graph, this basis contains a finite number of masters [6]. We write them as $M_1, \ldots, M_{n_M}$, where $n_M$ is the number of master integrals, so the reduction formula is

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \sum_{m=1}^{n_M} C_{\boldsymbol{a}m}(\boldsymbol{x}, \epsilon) M_m(\boldsymbol{x}, \epsilon). \tag{12}$$

Here $C_{\boldsymbol{a}m}$ is the coefficient of master $M_m$ in the reduction of target $I_{\boldsymbol{a}}$. The solver obtains it by adding, multiplying and dividing the identity coefficients, so it is a rational function, meaning a ratio of polynomials in $\boldsymbol{x}, \epsilon$. Different independent sets can define different master bases, and an insufficient equation set can leave additional integrals unresolved. Establishing the master basis therefore requires checking that the generated equations are enough to reduce the required integrals.

Equation (12) gives the coefficients multiplying the masters, whose evaluation requires a further calculation, for example by solving differential equations with known values or limits as boundary conditions [5]. Kira provides the reduction rules needed for this calculation by generating and processing the identities through the procedure described next.

## 4 How Kira performs the reduction

Kira implements the Laporta approach by generating a finite set of identities and eliminating integrals according to an ordering [7]. To obtain the coefficients in Eq. (12), Kira selects seeds, removes redundant equations and solves the retained system. We examine how Kira performs these steps to specify the seed and elimination choices that the proposed models would make.

### 4.1 Kira's input and seed generation

The user supplies the families in Eq. (3), relations among the parameters $\boldsymbol{x}$, the target list, and bounds on equation generation so that Kira can initialize the families and sectors. During this initialization, Kira detects scaleless sectors and identifies symmetry relations and mappings between compatible families [2].

A seed specifies the index vector $\boldsymbol{a}$ at which Kira evaluates Eq. (9) to generate several derivative identities, and the seed set may include integrals beyond the target list. The shifts $\boldsymbol{\Delta}_{\lambda j}$ in these

identities can introduce further integrals outside the seed set, so the resulting system depends on both the chosen seeds and the terms generated from them. To set limits on the indices of the seeds, Kira uses the quantities

$$t(\boldsymbol{a}) = \sum_{i=1}^{N} \theta_i(\boldsymbol{a}), \qquad r(\boldsymbol{a}) = \sum_{a_i>0} a_i, \tag{13}$$

$$s(\boldsymbol{a}) = -\sum_{a_i<0} a_i, \qquad n_{\mathrm{dot}}(\boldsymbol{a}) = r(\boldsymbol{a}) - t(\boldsymbol{a}). \tag{14}$$

Here $t$ is the number of distinct denominators, $r$ is the sum of their powers, and $s$ is the sum of the numerator powers. The quantity $n_{\mathrm{dot}}$ is the sum of the extra denominator powers beyond one for each denominator. We use this symbol to keep $d$ reserved for the integration dimension, and we restrict equation generation by specifying the allowed sectors and limits

$$r \le r_{\max}, \qquad s \le s_{\max}, \qquad n_{\mathrm{dot}} \le n_{\mathrm{dot,max}}, \tag{15}$$

where $r_{\max}$, $s_{\max}$ and $n_{\mathrm{dot,max}}$ are the upper limits on the corresponding quantities. The user sets $r_{\max}$ and $s_{\max}$ and may also set $n_{\mathrm{dot,max}}$. Because these choices determine which seeds contribute equations, a reduction run must establish whether the bounds supply enough relations for the targets [8].

Kira 3 changes seeding in symmetry-related sectors and provides the optional `truncate_sp` setting to restrict numerator seeding in subsectors, with parameter choices that require checks against the expected master basis [9]. The generated system can still contain redundant equations, which elimination identifies as it processes the relations into reduction rules.

### 4.2 Integral ordering, sector order and forward elimination

Elimination solves an identity for one integral and substitutes the result into other identities, using an integral ordering to select which unknown to eliminate from each equation. Kira defines these orderings through sector labels and the powers of the numerator and denominator factors, with choices that prioritize numerator powers or repeated denominators and thereby affect the master basis and runtime [8].

To describe this procedure, write $I_j = I_{\boldsymbol{a}^{(j)}}$ for the integrals in Eq. (11) and choose the ordering $I_1 \succ I_2 \succ \cdots \succ I_{n_I}$, where $\succ$ gives priority for elimination. In a row whose first nonzero term contains $I_k$, let $b_j$ denote the coefficient of $I_j$, so choosing the nonzero coefficient $b_k$ as the pivot gives

$$b_k I_k + \sum_{j>k} b_j I_j = 0 \quad \Longrightarrow \quad I_k = -\sum_{j>k} \frac{b_j}{b_k} I_j. \tag{16}$$

The coefficients $b_j$ depend on $\boldsymbol{x}, \epsilon$, and substituting this expression into the other rows removes their dependence on $I_k$. Repeating this step gives forward elimination, which builds a triangular system in which each pivot integral depends on integrals later in the ordering. Back substitution then replaces the remaining pivot integrals using their own rules, completing the Laporta solution [7].

Sector scheduling sets the order in which the solver processes groups of integrals or equations, taking into account which other integrals their equations contain. Ordinary IBP identities for a sector involve that sector and its subsectors because nonpositive indices remain nonpositive when applying the derivative. Equations for one sector can therefore depend on integrals from its subsectors. The Kira 3 description starts equation generation with sectors earlier in its ordering and inserts previously obtained pivot relations into new equations [9], with the order of processing depending on the stage of the reduction.

When several rows share the same first integral, the choice of pivot row changes the substitutions that follow. Choosing row $\rho$ to eliminate column $k$, for example, updates row $i$ by

$$A_{ij} \longleftarrow A_{ij} - \frac{A_{ik}}{A_{\rho k}} A_{\rho j}, \qquad A_{\rho k} \neq 0. \tag{17}$$

Here $i$ is the index of a row other than the pivot row, so $i \neq \rho$, and $j$ is the column index. Subtracting the stated multiple of row $\rho$ sets the coefficient in column $k$ to zero and can create nonzero entries in other columns, a change called fill-in when a previously zero entry becomes nonzero. These substitutions also make the coefficient expressions longer by combining their numerators and denominators, so elimination can require more time and memory as it proceeds. Redundant equations add to this work [10], so Kira uses modular elimination to select equations before processing their full symbolic coefficients.

### 4.3 Removing redundant equations and selecting target dependencies

To reduce the number of equations it must solve, Kira identifies rows that follow from other equations. Elimination reduces every coefficient in such a row to zero, allowing the solver to omit that row. Kira tests these dependencies with modular arithmetic [2] by substituting sample values $\boldsymbol{x}_0, \epsilon_0$ and mapping the coefficients to a finite field $\mathbb{F}_P$ for a prime integer $P$:

$$A(\boldsymbol{x}, \epsilon) \longmapsto A(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{18}$$

Integers from 0 to $P - 1$ represent the elements of $\mathbb{F}_P$, and addition and multiplication keep results in this range by taking the remainder upon division by $P$. Division uses the modular inverse of a nonzero element, which multiplies that element to one modulo $P$, so the sampling procedure excludes points where a required denominator vanishes. This arithmetic keeps field elements bounded in size while allowing the solver to identify independent rows and retain their original symbolic identities.

The matrix rank is the largest number of independent rows, where no row is a linear combination of the others in the chosen set. Some sample values can make otherwise independent rows dependent, giving a rank smaller than that of the symbolic matrix. Checks must account for this possibility when using the probabilistic equation-selection method developed by Kant [10].

Kira also tracks which independent equations contribute to the target reductions, using the dependencies of the forward-eliminated system for this selection in Kira 3 [9]. Seed selection therefore supplies the integrals from which Kira generates equations, and equation selection then retains the relations needed for the targets so that the solver can perform the full reduction on this smaller system.

### 4.4 Back substitution and the final symbolic result

To obtain $C_{\boldsymbol{a}m}$ from the retained equations, Kira applies back substitution to the pivot rules produced by forward elimination. Since these rules can contain other pivot integrals, back substitution inserts their expressions recursively until the right-hand sides contain only masters, giving Eq. (12). Kira's original symbolic solver carries out this calculation using rational-function arithmetic through the computer algebra system Fermat [2].

Kira also solves systems at finite-field sample points and reconstructs the rational coefficients with FireFly, the reconstruction library introduced into the solver in Kira 2.0 [11]. In this procedure, each sampled coefficient has the form

$$C_{\boldsymbol{a}m}(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{19}$$

The modular operations use exact arithmetic, and reconstruction recovers $C_{\boldsymbol{am}}(\boldsymbol{x}, \epsilon)$ from sufficient samples before checking the result at additional points. Reconstruction therefore forms part of the calculation of the symbolic reduction coefficients, after which the master integrals still require evaluation [11].

Because seed selection determines the equations that enter this procedure and elimination decisions determine the substitutions and intermediate expressions, the proposed ML models would guide these two operations to reduce the time and memory needed for the reduction.

# 5 ML for seed selection and elimination ordering

To study learned choices in IBP reduction, we use two System 1 models: one for seed selection and one for elimination ordering. Here a System 1 model is a small machine learning model that learns from recorded reductions and assigns scores to choices available at the current step. Training requires targets with known master expansions, the seeds that supply their equations and the steps used to reduce them. The implemented models and their completed training appear in Sections 7 and 9, followed by their held-out tests.

The present experiments test seed-reference agreement, whether the stored identities from the predicted seeds suffice inside Kira, and reduction of a target coefficient row. We then compare complete symbolic reductions with Kira's existing strategies. A native adapter converts predicted seeds into sector bounds for equation generation, while the row adapter applies exact symbolic target-row updates before Kira finishes the reduction. Section 11 measures their costs. Section 12 then places the row model inside Kira's evolving matrix, and Section 13 tests direct seed filtering and a compiled row policy with lower prediction costs.

## 5.1 Selecting seeds before equation generation

The first model receives the family definition, the target terms and their index vectors $\boldsymbol{a}$, the chosen master basis and a candidate seed list within stated sectors and index bounds. It assigns each candidate a separate probability of belonging to the target's recorded sufficient seed set, which allows several seeds to receive label one.

During training, we pair each target with the seeds used by its checked reference reduction. Different reductions can use different sufficient sets, and the stored labels identify the selected reference set. They do not establish that every omitted seed is unnecessary. For use inside Kira, we supply the stored equations associated with the predicted seeds and check whether they suffice in the chosen master basis. An unresolved target requires further identities. Section 8 measures recovery of the recorded sets and then tests their algebraic sufficiency through symbolic Kira reductions. The second test also checks the full candidate pool as a fallback.

## 5.2 Choosing elimination steps within the solver

The second model receives the current coefficient matrix and the supplied allowed row operations. It assigns a probability to each operation, selects one operation and reports the selected probability as its confidence. An exact row update then applies Eq. (17). The next state supplies the information for the next choice.

Each training example specifies the matrix before the choice, the allowed pivots, and the recorded source row and integral to eliminate. The retained later steps permit checks of the complete path and its arithmetic count, while predictive features use the current state and supplied actions. In

the present data, the source identities stay fixed and one target row changes during reduction. The native integration follows Kira's integral ordering and supplies allowed choices from the current evolving equations. This changes the states seen by the model and requires separate exact and runtime checks.

The computational benefit depends on the total work: preparing features and evaluating the model must cost less than the algebraic work that its choices save. The tests therefore record complete-path arithmetic and timing as well as next-step agreement. The comparison in Section 11 measures complete symbolic reductions with the selected operations.

### 5.3 Integral data required for training and testing

To train and test both models, we need family denominators, kinematic parameters, targets and a master basis together with the exact coefficients in Eq. (12). For a target containing several integrals, these coefficients specify the master expansion of the complete expression. Each example also identifies its source equations, actual IBP seeds and derivative operators. Algebraic identities, including mass partial fractions, remain separately identified because they carry no IBP seed.

We construct targets from known master combinations and retain the operations used to form each target and reduce it back to the same basis. The elimination records give the coefficients before and after each step, all allowed source rows, the selected pivot and the operation counts. These records connect seed selection to the equations available for elimination.

The known master coefficients provide a reference for checking complete reduction in the chosen basis, as described in Section 3 [9]. We keep these answers, the selected labels and future construction histories separate from the information used to predict a choice. Section 6 explains how we obtain the bases, construct the targets and check their seed and elimination records.

## 6 Target Integral Generation for Machine Learning

To train and test the two models described in Section 5, we constructed targets from master bases while retaining their source identities, seeds and algebraic steps. The initial collection contains 49 master entries and 1,662 targets across 16 families. We then extended the collection to 1,000 families, giving 9,678 master entries and 240,472 targets with checked reference reductions [12, 13, 14]. We first added 100 restricted coupled-vacuum mass profiles at each loop order from two through twelve, then 1,250 further profiles at each order from one through twelve. The complete collection contains 17,100 family instances, 57,979 master entries and 2,301,272 targets. The following subsections define the earlier construction and then derive the added vacuum bases and reverse targets. Every target retains its exact symbolic definition and master expansion.

### 6.1 Initial one-loop families and their kinematics

We use the integral definition and normalization in Eq. (3), with $d = 4 - 2\epsilon$ and inverse propagators $D_i = q_i^2 - m_i^2 + \mathrm{i}0$. For each family, we specify the internal momenta $q_i$, the masses $m_i$ and the scalar products of the external momenta, so every index vector defines a particular integral. In the equations below we suppress the common i0 prescription. We write $\mu = m^2$ for a common nonzero squared mass and $\mu_1 = m_1^2$, $\mu_2 = m_2^2$ for two independent nonzero squared masses. The symbols $x_s$ and $x_t$ denote momentum invariants and remain distinct from the index sums $s$ and $t$ in Eq. (14).

The one-loop collection begins with a massive tadpole, whose single inverse propagator is $D_1 = \ell_1^2 - \mu$ and which has no external momentum. We then include four bubble families with external momentum $p$ and inverse propagators

$$D_1 = \ell_1^2 - m_1^2, \qquad D_2 = (\ell_1 - p)^2 - m_2^2, \qquad p^2 = x_s. \tag{20}$$

The squared-mass pairs are $(0,0)$, $(\mu,0)$, $(\mu,\mu)$ and $(\mu_1,\mu_2)$, corresponding to a massless bubble, a bubble with one massive line, an equal-mass bubble and an unequal-mass bubble. We keep $x_s$ as an independent nonzero scale, with the massive families defined at generic values of their independent parameters.

To include three-point integrals, we define triangle families by

$$\begin{aligned} &D_1 = \ell_1^2 - m_1^2, \quad D_2 = (\ell_1 - p_1)^2 - m_2^2, \qquad D_3 = (\ell_1 - p_1 - p_2)^2 - m_3^2, \\ &p_1^2 = u_1, \qquad\qquad p_2^2 = u_2, \qquad\qquad (p_1 + p_2)^2 = u_3. \end{aligned} \tag{21}$$

Here $u_1, u_2, u_3$ are the squared external momenta, also called external virtualities, and $p_1 \cdot p_2 = (u_3 - u_1 - u_2)/2$ fixes the remaining external scalar product. We use three massless families with $(u_1,u_2,u_3) = (0,0,x_s)$, $(0,x_2,x_3)$ and $(x_1,x_2,x_3)$. The parameters $x_1, x_2, x_3$ denote independent nonzero virtualities. We also use two families with $m_1^2 = m_2^2 = m_3^2 = \mu$ and virtualities $(0,0,x_s)$ and $(x_1,x_2,x_3)$. Thus the five triangle families vary both the internal masses and the number of nonzero external virtualities.

For four-point integrals, we use three massless box families with $p_4 = -p_1 - p_2 - p_3$ and

$$\begin{aligned} &D_1 = \ell_1^2, \qquad\qquad D_2 = (\ell_1 - p_1)^2, \\ &D_3 = (\ell_1 - p_1 - p_2)^2, \quad D_4 = (\ell_1 - p_1 - p_2 - p_3)^2, \\ &x_s = (p_1 + p_2)^2, \qquad x_t = (p_2 + p_3)^2. \end{aligned} \tag{22}$$

Writing $u_i = p_i^2$ for $i = 1, \ldots, 4$, we fix the independent external scalar products through

$$p_1 \cdot p_2 = \frac{x_s - u_1 - u_2}{2}, \qquad p_2 \cdot p_3 = \frac{x_t - u_2 - u_3}{2}, \qquad p_1 \cdot p_3 = \frac{u_4 + u_2 - x_s - x_t}{2}. \tag{23}$$

The virtuality choices are $(0,0,0,0)$, $(0,0,0,x_4)$ and $(x_1,x_2,x_3,x_4)$, where $x_4$ is the fourth nonzero virtuality parameter. These choices give boxes with zero, one and four off-shell legs; in this classification an on-shell external leg has $p_i^2 = 0$. We treat the remaining invariants as independent subject to the stated relations. These definitions give 13 one-loop families, which we extend with the following three two-loop families.

### 6.2 Initial two-loop families and auxiliary expressions

The two-loop families use loop momenta $\ell_1, \ell_2$. For a massless two-point integral with external momentum $p$ and $p^2 = x_s$, we take

$$(D_1, D_2, D_3, D_4, D_5) = (\ell_1^2, \ell_2^2, (\ell_1 - \ell_2)^2, (\ell_1 - p)^2, (\ell_2 - p)^2). \tag{24}$$

These five expressions determine the five loop-dependent scalar products $\ell_1^2, \ell_2^2, \ell_1 \cdot \ell_2, \ell_1 \cdot p$ and $\ell_2 \cdot p$. Varying their indices therefore includes numerator insertions as well as repeated propagators.

For the equal-mass sunrise with the same external invariant, we use

$$\begin{aligned} &D_1 = \ell_1^2 - \mu, \qquad D_2 = \ell_2^2 - \mu, \qquad D_3 = (p - \ell_1 - \ell_2)^2 - \mu, \\ &D_4 = (p - \ell_1)^2, \quad D_5 = (p - \ell_2)^2, \quad a_4, a_5 \le 0. \end{aligned} \tag{25}$$

The first three expressions are the physical inverse propagators. The auxiliary expressions $D_4, D_5$ complete the five scalar products needed to express the derivatives in the IBP identities. Their nonpositive indices restrict them to numerator factors or their absence, so the top sector contains only $D_1, D_2, D_3$ as denominators. We keep $x_s$ and $\mu$ independent.

The final family is the equal-mass two-loop vacuum sunset, which has no external momentum and uses

$$D_1 = \ell_1^2 - \mu, \qquad D_2 = \ell_2^2 - \mu, \qquad D_3 = (\ell_1 - \ell_2)^2 - \mu. \tag{26}$$

These three expressions determine all loop scalar products for this vacuum family. For every family, we include the sector containing all physical propagators and all its subsectors, so the resulting basis also accounts for integrals with removed propagators. The number of physical propagators in this top sector is denoted by $t_{\text{top}}$. It is one, two, three and four for the tadpole, bubble, triangle and box families, respectively, five for the massless two-loop propagator, and three for the sunrise and vacuum families. This number sets the initial range of identities used to identify the masters.

### 6.3 Identification and checks of the master bases

For each initial family, we used Kira 3.1 to generate IBP relations together with the applicable Lorentz-invariance and symmetry relations. Kira removes scaleless sectors and uses finite-field elimination to identify the integrals that remain independent in the equations needed for a selected integral range [2, 9]. We used Kira's integral ordering 1 throughout, which orders sectors by their labels and prefers numerator insertions over extra denominator powers when selecting representatives within a sector [8]. Keeping this ordering fixed gives a specified basis against which to compare later calculations.

We first generated identities from seeds satisfying

$$r \le t_{\text{top}} + 2, \qquad s \le 2, \qquad n_{\text{dot}} \le 2, \tag{27}$$

and selected integrals to probe the basis within

$$r \le t_{\text{top}} + 1, \qquad s \le 1, \qquad n_{\text{dot}} \le 1. \tag{28}$$

Here $r$ is the sum of positive indices, $s$ is minus the sum of negative indices and $n_{\text{dot}} = r - t$ is the number of extra denominator powers, as defined in Eq. (14). Both sets of limits apply to the top sector and its subsectors. The probe integrals specify the range for which Kira must identify the required masters, while the wider seed range supplies relations that can eliminate the additional integrals introduced by index shifts. These probes serve the basis calculation; the ML examples come from the subsequent reverse construction.

Kira evaluated the coefficients of these relations over a finite field using the substitution in Eq. (18). For the first calculation we used the prime $P_1 = 2147483647$ and dimension value $d_1 = 37$. To specify the kinematic substitutions, define the ordered parameter list

$$\boldsymbol{z} = (x_s, x_t, x_1, x_2, x_3, x_4, \mu, \mu_1, \mu_2). \tag{29}$$

The first sample was

$$\boldsymbol{z}^{(1)} = (-13, -19, -11, -17, -29, -43, 7, 7, 23), \tag{30}$$

using only the entries present in each family and retaining all zeros and equal-mass conditions in its definition. We interpret these assignments modulo $P_1$ when evaluating the identity coefficients. The dimension sample corresponds to $\epsilon_1 = (4 - d_1)/2$ in the same field. These

substitutions test algebraic relations; the master labels continue to refer to integrals at symbolic $\boldsymbol{x}, \epsilon$.

We then enlarged the seed and probe ranges to check whether the initial basis remained sufficient:

$$\begin{aligned} \text{seeds:} \quad & r \le t_{\text{top}} + 3, \qquad s \le 3, \qquad n_{\text{dot}} \le 3, \\ \text{probes:} \quad & r \le t_{\text{top}} + 2, \qquad s \le 2, \qquad n_{\text{dot}} \le 2. \end{aligned} \tag{31}$$

We repeated the calculation first at the original sample and then at a second sample with prime $P_2 = 1000000007$, dimension $d_2 = 41$ and

$$\boldsymbol{z}^{(2)} = (-31, -47, -23, -37, -53, -71, 11, 11, 29). \tag{32}$$

Changing the prime, the dimension and the independent kinematic parameters provides a second check of the linear dependencies, since particular substitutions can lower the rank of the symbolic coefficient matrix [10].

In both enlarged calculations, we supplied the original masters as the preferred basis and enabled Kira's check for additional or unused masters [9]. We required the complete list of master index vectors to agree in all three calculations, and also checked each vector against its family definition and sector label. All 16 families satisfied these requirements, giving 48 completed master-identification calculations. The agreement provides evidence for basis stability over the tested index ranges and sample points. Completeness for arbitrary integer indices would require further relations or an independent argument, and special kinematic limits can change the basis.

We stopped each calculation after master identification. This stage determines which symbolic integrals form the starting basis within the checked domains. We then obtained rational reduction coefficients for the constructed targets through the algebraic procedure described below. The numerical values and $\epsilon$ expansions of the master integrals remain separate from this basis determination.

### 6.4 Resulting bases and the starting coefficients

The massive tadpole gave one master. The massless, one-mass, equal-mass and unequal-mass bubbles gave one, two, two and three masters, respectively. The massive bubble bases include the necessary tadpole subsectors, with two distinct tadpoles when the internal masses differ.

The massless triangles with one, two and three nonzero external virtualities gave one, two and four masters, respectively. For the first two families, Kira chose bubble integrals in subsectors as the masters. The three-virtuality family also retained a triangle integral. The equal-mass triangles with one and three nonzero virtualities gave three and five masters. The massless boxes with zero, one and four off-shell legs gave three, four and eleven masters, respectively, including the required lower sectors in each count.

The massless two-loop propagator gave two masters, the equal-mass sunrise gave three, and the equal-mass vacuum sunset gave two. For the denominator order in Eq. (25), the sunrise basis is

$$M_1 = I_{(1,1,0,0,0)}, \qquad M_2 = I_{(1,1,1,0,0)}, \qquad M_3 = I_{(1,1,1,-2,0)}. \tag{33}$$

The first integral factorizes into a product of two massive tadpoles because removing $D_3$ separates the loop integrations. The other two integrals retain all three sunrise propagators, and $M_3$ contains the numerator factor $D_4^2$. This example shows how the chosen ordering can retain a numerator integral as a master.

The counts sum to 49 master entries across these 16 initial families. We count each basis within its own family, so shared tadpole and bubble subsectors can appear in more than one family.

The complete family definitions and master index vectors are available with the accompanying data [12]. Each entry specifies the denominators, kinematics, index vector, sector and mass dimension. With the normalization in Eq. (3), the latter is

$$[I_{\boldsymbol{a}}]_{\text{mass}} = Ld - 2\sum_{i=1}^{N} a_i, \tag{34}$$

where $[I_{\boldsymbol{a}}]_{\text{mass}}$ is the power of mass carried by the integral. Each loop measure contributes $d$ powers and each factor $D_i^{-a_i}$ contributes $-2a_i$. This dimension constrains the coefficients used to combine integrals in the target construction.

For that construction, we kept the identified masters symbolic and initialized each master as itself. If $\boldsymbol{\mu}^{(m)}$ is the index vector of master $M_m$, the starting coefficients are

$$M_m = I_{\boldsymbol{\mu}^{(m)}}, \qquad C_{\boldsymbol{\mu}^{(m)}n} = \delta_{mn}, \qquad m, n = 1, \ldots, n_M. \tag{35}$$

Here $n_M$ is the number of masters in the selected family, and $\delta_{mn}$ is one when $m = n$ and zero otherwise. Thus each master has coefficient one for itself and zero for the others. We held the family definition and basis fixed while constructing its targets, starting with these masters and extending the collection through the identities below. Each accepted target therefore has a specified reference basis and an exact master expansion.

### 6.5 Source identities and the reference integral order

To extend the starting masters into targets, we first generated relations among the integrals of each family. We used the IBP identity in Eq. (7) with seeds in every nonempty physical sector within the bounds of Eq. (27). The auxiliary sunrise indices remained nonpositive. We fixed this candidate seed set before constructing targets, and retained each equation with its seed $\boldsymbol{\sigma}$, differentiated loop momentum $\ell_k$ and vector $v$ [4, 2]. The resulting equations can contain integrals beyond the seed bounds because differentiation shifts the indices.

The vectors $v$ included loop momenta, external momenta and propagator momenta $q_i$. Each propagator momentum is a fixed linear combination of loop and external momenta, so its IBP equation is the corresponding linear combination of the usual IBP equations at the same seed. We retained this decomposition to specify how each equation follows from its source operators. We also obtained symmetry relations by explicit changes of loop variables whose loop transformation matrix has determinant $+1$ or $-1$. We accepted a transformation when it permuted the inverse propagators with their masses unchanged and preserved the physical and auxiliary index positions.

To give the constructions a reduction order, we protected the chosen masters from elimination and ordered every other integral by

$$(S(\boldsymbol{a}),\, n_{\text{dot}}(\boldsymbol{a}),\, s(\boldsymbol{a}),\, r(\boldsymbol{a}),\, \boldsymbol{a}). \tag{36}$$

Here $S$ is the sector number in Eq. (5), and the remaining quantities are the index sums in Eq. (14). We compare the entries in the displayed order, using the first unequal entry to decide which integral is larger. We compare the components of $\boldsymbol{a}$ in the same way when all earlier entries agree. The leading integral of an equation is its largest integral outside the master basis. This reference order specifies the construction and recorded reductions; adapting the elimination model to Kira will require its actual ordering and available pivot choices.

### 6.6 Reverse construction from combinations of masters

To obtain target expressions with known answers, we selected subsets of masters from the same family and replaced their terms through the source identities. Let $\mathcal{J} = \{m_1, \ldots, m_h\}$ be the selected master labels, where $h$ is the number of chosen masters. The initial expression is

$$\mathcal{T}_0 = \sum_{m \in \mathcal{J}} c_m(\boldsymbol{x}, d) M_m, \qquad 1 \le h \le \min(3, n_M). \tag{37}$$

The coefficients are rational functions of the independent kinematic parameters and $d = 4 - 2\epsilon$, with no dependence on the loop momenta. Writing them in terms of $d$ is equivalent to writing them in terms of $\epsilon$. Their mass dimensions must compensate for differences between the chosen masters so that every term in $\mathcal{T}_0$ has the same dimension.

For this purpose, let $B_m = \sum_i \mu_i^{(m)}$ be the sum of the indices of master $M_m$, and set $A = B_{m_1}$. For a fixed nonzero squared mass or momentum invariant $Q^2$ in the family, we used

$$c_{m_j} = u_j(d)(Q^2)^{B_{m_j} - A}, \qquad [\mathcal{T}_0]_{\text{mass}} = Ld - 2A. \tag{38}$$

Here $j = 1, \ldots, h$ identifies a selected master and $u_j$ is dimensionless. We varied the weights through the three forms

$$\begin{gathered} u_j(d) \in \left\{ \eta_j,\ \eta_j \frac{d-2}{d-4},\ \eta_j \frac{j+1}{d-2} \right\}, \\ \eta_j \in \{-3, -2, -1, 1, 2, 3\}. \end{gathered} \tag{39}$$

Here $\eta_j$ is a nonzero integer weight. For each sequence we chose one of these three forms and varied the integer weights and selected masters. The power of $Q^2$ in Eq. (38) gives $c_m$ mass dimension $2(B_m - A)$, so Eq. (34) gives the stated common dimension.

To replace a term, write the current expression and a source IBP identity as

$$\mathcal{T}_n = \sum_{\boldsymbol{b}} w_{\boldsymbol{b}}^{(n)} I_{\boldsymbol{b}}, \qquad R = \sum_{\boldsymbol{b}} r_{\boldsymbol{b}} I_{\boldsymbol{b}} = 0. \tag{40}$$

Here $n$ is the number of completed replacements, while $w_{\boldsymbol{b}}^{(n)}$ and $r_{\boldsymbol{b}}$ are the coefficients of integral $I_{\boldsymbol{b}}$ in the expression and identity. The sums contain their finitely many nonzero terms. For a present term $I_{\boldsymbol{a}}$, we selected an identity generated at seed $\boldsymbol{a}$ with $r_{\boldsymbol{a}} \neq 0$, and formed

$$\mathcal{T}_{n+1} = \mathcal{T}_n - \frac{w_{\boldsymbol{a}}^{(n)}}{r_{\boldsymbol{a}}} R. \tag{41}$$

This operation cancels the chosen term and introduces other family integrals. Since $R = 0$, the resulting expression has the same master expansion as $\mathcal{T}_0$. We required the identity to introduce a leading integral larger than every integral already present, so the recorded steps can be reversed through the order in Eq. (36).

For the initial collection, we attempted 24 sequences per family, with at most four replacements per sequence, and retained accepted intermediate expressions. A sequence stopped when no admissible replacement was found within the construction limits. We removed identical target expressions and retained only nonzero targets with complete reference reductions. When the final expression contained one integral with coefficient $q \neq 0$, we divided the expression and its master coefficients by $q$ and recorded this normalization. Expressions containing several index vectors remain targets for reduction of the complete sum. To obtain further individual integrals, we also applied the source identities in the following way.

### 6.7 Constructing individual integrals from known reductions

Starting from Eq. (35), we searched for a source identity whose leading integral was the only term without a known master expansion. Such an identity has the form

$$h_0 I_{\boldsymbol{b}} + \sum_{j=1}^{n_R} h_j I_{\boldsymbol{a}^{(j)}} = 0, \qquad h_0 \neq 0, \tag{42}$$

where $I_{\boldsymbol{b}}$ is the new integral, $n_R$ is the number of other terms and $h_0, h_j$ are the identity coefficients. Substituting the known reductions of the other terms gives

$$I_{\boldsymbol{b}} = \sum_{m=1}^{n_M} C_{\boldsymbol{b}m} M_m, \qquad C_{\boldsymbol{b}m} = -\sum_{j=1}^{n_R} \frac{h_j}{h_0} C_{\boldsymbol{a}^{(j)}m}. \tag{43}$$

We added each resulting expansion to the known set so later identities could use it. If all its master coefficients vanished, we retained the zero-integral proof as a dependency for subsequent constructions. The target collection contains the accepted nonzero examples.

For the initial collection, we made at most five passes through the source equations and allowed at most 64 new nonzero constructions of this type per family. An equation with several still-unknown integrals became usable only after other identities supplied the missing expansions. Solving remaining coupled systems would extend this construction beyond the individual integrals obtained here. Both this procedure and the reverse replacements above preserve the source equations and their dependencies, which determine the seed and elimination records.

### 6.8 Seed records and elimination sequences

For each accepted target, we retained its exact master coefficients together with the source identities used to obtain them. An IBP identity contributes its seed and derivative operator, while a symmetry identity contributes its loop-variable transformation and integral mapping. Dependencies carry the earlier identities needed to reduce intermediate integrals or establish that they vanish. For an IBP step of the form in Eq. (42), the sufficient seed set therefore obeys

$$\mathcal{S}_{\boldsymbol{b}} = \{\boldsymbol{\sigma}\} \cup \bigcup_{j=1}^{n_R} \mathcal{S}_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{S}_{\boldsymbol{\mu}^{(m)}} = \varnothing. \tag{44}$$

Here $\mathcal{S}_{\boldsymbol{a}}$ is the seed set recorded for $I_{\boldsymbol{a}}$, $\boldsymbol{\sigma}$ is the seed of the current IBP identity and $\varnothing$ is the empty set. Symmetry steps retain the dependency sets and their symmetry record. For reverse constructions, the seed set is the union of the seeds of the identities used along the recorded path.

We retained the full candidate seed set alongside each family so the seed model can compare possible choices. The recorded subset supplies enough equations for the known reduction, and seeds outside it can still contribute to other valid reductions. For the model inputs, we stored the family, basis, target coefficients and index information, including the sector and $t, r, s, n_{\mathrm{dot}}$. We kept the exact answer, construction history and reference seed choices separately.

To obtain elimination examples, we represented the target as a row of coefficients $T_{\boldsymbol{b}}$ multiplying $I_{\boldsymbol{b}}$. At each step, we selected its largest remaining integral outside the master basis, denoted by $I_{\boldsymbol{a}}$. The available pivot rows were the retained source identities with leading integral $I_{\boldsymbol{a}}$ and a nonzero coefficient for that integral. For the chosen row $R$, the update was

$$T_{\boldsymbol{b}} \longleftarrow T_{\boldsymbol{b}} - \frac{T_{\boldsymbol{a}}}{r_{\boldsymbol{a}}} r_{\boldsymbol{b}}. \tag{45}$$

This cancels $I_{\boldsymbol{a}}$ from the target and can introduce only smaller integrals or masters. Repeating the recorded choices gives the target's known master expansion. The source equations remain unchanged during this target-row reduction, so the current row, source equations and reference order specify the state seen by the elimination model.

For every step, we retained the row before and after the update, all available pivot rows, the selected equation and integral, the multiplier, and the numbers of new and cancelled nonzero entries. We kept the coefficients as exact rational functions and also stored numerical row states over a finite field. The first sample in Eq. (30), with $P_1 = 2147483647$ and $d_1 = 37$, supplied these numerical states. Substituting parameters into the coefficient functions leaves the integrals symbolic and uses exact modular arithmetic for the row operations [11].

For a selected source row with $n_{\mathrm{row}}$ nonzero terms, we counted one finite-field division and $n_{\mathrm{row}}$ multiplications and subtractions. Summing these counts gives the cost of the recorded target-row sequence. Equation generation, forming combined IBP operators, symbolic simplification, model inference and reconstruction fall outside this count. The construction gives a valid reference path and an upper bound on the minimum cost under these rules. Establishing a minimum would require a separate argument covering alternative seed sets and elimination paths.

### 6.9 Initial target collection and consistency checks

The initial collection contains 1,662 accepted targets across its 16 families, with 309 individual integrals, 1,353 integral expressions and 5,138 recorded elimination steps [13]. We divided the examples by complete families into 1,290 training targets, 185 validation targets and 187 test targets. Validation targets belong to the unequal-mass bubble and massless triangle with three off-shell legs. Test targets belong to the massless box with four off-shell legs and the equal-mass sunrise. These held-out targets contain terms in the full physical sector, while their basis definitions still include the required lower sectors.

In this initial collection, the validation set consists entirely of integral expressions, and the test set contains one individual integral and 186 expressions. These initial counts motivate the additional held-out individual constructions described below. This initial collection covers the specified one-loop and two-loop families and the stated construction bounds. The following sections describe seed-model training and its held-out comparison on the expanded collection. Section 9 describes the reduction model. Section 10 measures complete target-row sequences and their selection costs. Section 11 compares complete symbolic reductions, including native Kira seeding.

Before accepting the collection, we rederived all 55,654 stored source equations from their family definitions and checked the recorded symmetry transformations. We also checked 2,699 used IBP equations by independently differentiating the original quadratic expressions for the propagators in terms of loop scalar products. We then replayed every target construction and elimination sequence with exact rational functions, including intermediate zero-integral dependencies and final normalizations. Each final result agreed with its stored master expansion, and the checks also confirmed mass dimensions, seed records and allowed pivots.

To check the sampled coefficients, we repeated every elimination sequence at three finite-field points. The first two were the samples in Eqs. (30) and (32), with the primes and dimension values specified there. The third used $P_3 = 1000000009$, $d_3 = 43$ and

$$\boldsymbol{z}^{(3)} = (-59, -73, -41, -61, -83, -101, 17, 17, 47), \tag{46}$$

in the parameter order of Eq. (29). Each family used its own parameters and retained its prescribed zeros and equal masses. All required divisions were defined at the three points, and every sampled reduction reproduced the reference coefficients.

The exact symbolic checks establish the recorded reductions from their source identities at generic parameters, while the three-point checks verify the numerical data supplied with them. This construction already provides the symbolic coefficient functions, so obtaining the reference answers requires no rational-function reconstruction. The accompanying data contain the targets, equations, construction histories, sufficient seeds and elimination records needed to train and test the two proposed models [13].

### 6.10 Extension to different mass and kinematic conditions

To supply examples across a wider range of parameter conditions, we added 984 one-loop families with four physical propagators. For these families, we write

$$D_i = (\ell_1 - Q_i)^2 - \mu_{\alpha_i} + \mathrm{i}0, \qquad i = 1, \ldots, 4, \qquad Q_1 = 0. \tag{47}$$

Here $Q_2, Q_3, Q_4$ are three independent external momentum vectors, $\mu_0 = 0$ and $\mu_1, \mu_2, \mu_3$ are independent nonzero squared masses. Each label $\alpha_i \in \{0, 1, 2, 3\}$ assigns the squared mass of the corresponding propagator. Repeated positive labels impose equal masses.

We specify the external kinematics through the six squared differences between the propagator shifts:

$$\Delta_{ij} = (Q_i - Q_j)^2 = y_{\beta_{ij}}, \qquad 1 \le i < j \le 4. \tag{48}$$

Here $y_0 = 0$, $y_1, y_2, y_3$ are independent nonzero momentum invariants, and $\beta_{ij} \in \{0, 1, 2, 3\}$. Repeated labels impose equality between the corresponding invariants. The mass parameters and momentum invariants remain independent of one another, and a family uses only the parameters that occur in its assignments.

These distances determine the external scalar products:

$$G^{\mathrm{ext}}_{ij} = Q_i \cdot Q_j = \frac{\Delta_{1i} + \Delta_{1j} - \Delta_{ij}}{2}, \qquad i, j = 2, 3, 4, \qquad \Delta_{ii} = 0. \tag{49}$$

The matrix $G^{\mathrm{ext}}$ is the external Gram matrix. We required it to be nonsingular at all three recorded sample points, which also excludes a determinant that vanishes identically under the stated parameter conditions. Each family then defines a symbolic integral problem through its particular zero and equality relations.

To remove equivalent definitions, we compared the assignments under all 24 permutations of the four propagators, together with independent relabelling of the nonzero mass and momentum-invariant parameters. The enumeration found 1,673 inequivalent profiles satisfying the parameter and sample requirements after excluding the existing massless box definitions. A fixed ordering selected 984 of these profiles [14]. Together with the original families, this earlier collection contained 997 one-loop and three two-loop families. The added profiles share the four-propagator box topology, so the family count describes different mass and kinematic conditions within this topology.

We identified and checked a master basis separately for every added family using the three calculations in Subsection 6.3. The four-propagator families have $t_{\mathrm{top}} = 4$, so the first seed range has $r \le 6$, $s \le 2$ and $n_{\mathrm{dot}} \le 2$. We then enlarged the seed and probe ranges, supplied the first masters as the preferred basis and repeated the basis check at a second point and prime. All 1,000 families passed their three basis calculations, giving 3,000 completed checks within the recorded domains.

For the added families, define

$$\boldsymbol{w} = (y_1, y_2, y_3, \mu_1, \mu_2, \mu_3). \tag{50}$$

We used the same three primes and dimension values specified above, with the parameter assignments

$$\begin{aligned} \boldsymbol{w}^{(1)} &= (-11, -19, -29, 7, 13, 23), \\ \boldsymbol{w}^{(2)} &= (-31, -47, -61, 11, 17, 29), \\ \boldsymbol{w}^{(3)} &= (-41, -67, -83, 17, 23, 37). \end{aligned} \tag{51}$$

Each family uses only its present parameters and retains all specified zeros and equalities. The first two points supply the basis checks, while the target validation uses all three.

### 6.11 Expanded targets and their verified reductions

Using the added bases, we applied the reverse construction in Eq. (41) with 48 attempted sequences per family and at most four replacements per sequence. We also allowed up to 96 nonzero individual-integral constructions per added family, using at most five passes through the source identities. The initial master weights retain the rational functions and mass-dimension conditions in Eqs. (38) and (39).

When the first construction produced no accepted targets, we extended the candidate equations to every retained IBP identity containing the selected current integral. Its seed $\boldsymbol{\sigma}$ can then differ from that integral's index vector $\boldsymbol{a}$. The substitution still uses the coefficient of $I_{\boldsymbol{a}}$ in the selected identity and preserves the known master expansion. This extension allows a reverse path to enter a higher sector through an identity generated there, including cases whose masters all lie in lower sectors. We retain the actual seed, require the new leading integral to increase under the reference order, and check every step in the resulting reduction.

To add individual integrals to the held-out sets, we also used the checked construction proofs to select integrals whose four indices are all nonzero and include at least one negative index. Each selected integral already has a known master expansion. We retained it as a separate target and rebuilt its elimination trace from the recorded identities. This step added 384 individual targets. Each has one denominator factor and three numerator factors, with their powers specified by the index vector. The complete four-expression definition therefore enters the target, while its positive indices place it in a one-propagator sector.

Before the higher-loop extension, the collection contained 9,678 master entries and 240,472 targets: 55,768 individual integrals and 184,704 integral expressions [14]. We count master entries within each family, so related lower-sector integrals can occur in several bases. Every target has its complete master expansion, source identities, sufficient seeds and recorded elimination path.

For the added families, we kept each complete kinematic zero and equality pattern in one data split across all mass assignments. This grouping also respects the original training box profiles. We assigned the original validation bubble and triangle examples to the combined training set because those integral families also occur as lower sectors of the added training families. The new grouped families supply the expanded validation set. The original data-generation split contained 771 training families with 198,527 targets, 106 validation families with 19,600 targets, and 123 test families with 22,345 targets. The reverse sequences require validation and test expressions to contain terms in their full physical sectors. The additional individual targets instead require all four family expressions to occur, including those in the numerator. Shared lower-sector masters remain part of the family definitions. The complete family profile is held out, while some lower-sector reduction structure can also occur in training.

The original data-generation validation set contained 183 individual integrals and 19,417 integral expressions, while the corresponding test set contained 202 individual integrals and 22,143 expressions. These counts describe the original construction samples. The grouped model division described below replaces this three-way data-generation division. The added individual

targets supply tests of numerator integrals in one-propagator sectors. The higher-loop construction below supplies additional individual targets and expressions in coupled vacuum families. Sections 8 and 10 measure the two models on the current grouped split.

We rederived 3,299,902 source equations and checked 227,330 used IBP equations by independent differentiation across this earlier collection. We replayed all 240,472 target constructions and 832,924 recorded elimination steps with exact rational functions. Every reduction reproduced its reference master coefficients, and the three finite-field replays checked the corresponding numerical rows. We also checked family equivalence, the grouping of data splits and the correspondence between every saved target and its records. The accompanying collection provides the family definitions, bases, targets and complete reduction data [14].

### 6.12 Coupled vacuum families from one through twelve loops

To supply examples at every loop order considered in the model study, we first added 100 mass profiles at each order from two through twelve. We then added 1,250 further profiles at each order from one through twelve, in groups of 250 and 1,000 profiles. These extensions provide 16,100 family instances beyond the original collection. They use coupled vacuum integrals whose master expansions follow from mass partial fractions and IBP identities, which allows us to retain exact reference answers as the number of loops increases. The profiles vary the masses within a restricted set of graphs, so the family count measures mass assignments as well as graph structure.

For this extension we use Euclidean loop momenta and define

$$V_{\boldsymbol{b}}^{(L)} = \int \prod_{i=1}^{L} \frac{d^d \ell_i}{\pi^{d/2}} \, \frac{1}{\ell_1^2 \cdots \ell_L^2} \prod_{j=1}^{R} \frac{1}{(Q^2+\mu_j)^{b_j}}, \qquad Q = \sum_{i=1}^{L} \ell_i, \qquad \mu_j = \rho_j \mu. \tag{52}$$

Here $L$ is the number of loops, $R$ is the number of distinct massive denominators, $\mu = m^2 > 0$ is a common squared-mass scale and $\rho_j$ is a positive mass ratio. The nonnegative integer $b_j$ is the power of the $j$th massive denominator, and $B = \sum_{j=1}^{R} b_j \geq 1$ is their total power. Every massless denominator has power one in the generated domain. The full integral index vector is $(1, \ldots, 1, b_1, \ldots, b_R)$, with $L+R$ entries. Equation (52) specifies the normalization and signs used for this Euclidean subcollection; the earlier families retain their stated Minkowski convention.

For $L > 1$, the common momentum $Q$ couples the loop integrations. A graph representation has $L$ parallel massless edges between two vertices and a chain of $R$ massive edges joining the same vertices. It has $L + R$ edges and $R + 1$ vertices, giving $L$ independent loops. All massive denominators depend on $Q^2$, which gives algebraic relations between them. This definition is a parent family for partial fractions with fixed massless powers; it does not specify a complete independent denominator and numerator basis for a generic $L$-loop scattering integral.

For profile number $p$, we choose

$$R = 2 + ((p-1) \bmod 3), \qquad \rho_1 = 1, \qquad \rho_{j+2} = 2 + 3p + j(p+2), \quad j = 0, \ldots, R-2. \tag{53}$$

The ratios are distinct within each profile, and $\rho_1 = 1$ fixes a common rescaling. The initial profiles use $p = 1, \ldots, 100$ at $L = 2, \ldots, 12$. The later groups use $p = 101, \ldots, 350$ and $p = 351, \ldots, 1350$ at $L = 1, \ldots, 12$. The latest group has 333 profiles with two massive denominators, 333 with three and 334 with four at each loop order. A twelve-loop family with four massive denominators has sixteen indices, which fits the existing model representation.

### 6.13 Exact masters and source equations for the added families

The dependence on the common momentum makes the master basis explicit. Write $\boldsymbol{e}_j$ for the unit vector in massive index position $j$ and choose

$$M_j^{(L)} = V_{\boldsymbol{e}_j}^{(L)}, \qquad j = 1, \ldots, R. \tag{54}$$

To see why these integrals span the generated domain, first use $D_i - D_j = \mu_i - \mu_j$, where $D_j = Q^2 + \mu_j$. If $b_i, b_j > 0$, this identity gives

$$(\mu_i - \mu_j) V_{\boldsymbol{b}}^{(L)} - V_{\boldsymbol{b}-\boldsymbol{e}_i}^{(L)} + V_{\boldsymbol{b}-\boldsymbol{e}_j}^{(L)} = 0. \tag{55}$$

Repeated use removes products of distinct massive denominators. This equation is an algebraic mass relation and carries no IBP seed.

The remaining powers follow from the sum of the $L$ IBP operators $\partial_{\ell_i^\nu} \ell_i^\nu$. Applying this sum to the integrand of Eq. (52) gives

$$0 = [Ld - 2(L + B)] V_{\boldsymbol{b}}^{(L)} + 2 \sum_{j=1}^{R} b_j \mu_j V_{\boldsymbol{b}+\boldsymbol{e}_j}^{(L)}. \tag{56}$$

Each such equation retains its actual seed $(1, \ldots, 1, \boldsymbol{b})$ and the contributing derivative operators. For one massive denominator this becomes

$$V_{k\boldsymbol{e}_j}^{(L)} = \left[ \prod_{h=1}^{k-1} \frac{L + h - Ld/2}{h\mu_j} \right] M_j^{(L)}, \qquad k \geq 1. \tag{57}$$

The empty product at $k = 1$ equals one. Equations (55) and (57) therefore reduce every generated integral to the stated masters with coefficients rational in $d$ and $\mu$.

An analytic expression also checks the basis. Set $D = d/2$, $\alpha = L(D-1)$ and $c = L - (L-1)D$. Repeated integration of the massless subgraph at fixed $Q$ gives

$$C_L(D)(Q^2)^{-c}, \qquad C_L(D) = \frac{\Gamma(c)\Gamma(D-1)^L}{\Gamma(\alpha)}. \tag{58}$$

For $L = 1$, $Q = \ell_1$, $c = 1$ and $C_1(D) = 1$, so this expression reduces directly to the single massless propagator. The remaining radial integration uses $\int d^dQ/\pi^D = \Gamma(D)^{-1} \int_0^\infty dx\, x^{D-1}$ for a rotationally invariant integrand, where $x = Q^2$. It gives

$$M_j^{(L)} = \mu_j^{\alpha-1} \frac{\Gamma(c)\Gamma(D-1)^L \Gamma(1-\alpha)}{\Gamma(D)}. \tag{59}$$

Here $\Gamma$ is the Euler gamma function. These manipulations use the standard analytic continuation of dimensionally regularized integrals [3]. At generic $d$, distinct $\rho_j$ give distinct exponential functions of $d$ in $\rho_j^{\alpha-1}$. Their linear independence over rational functions of $d$ establishes independence of the $R$ masters in this restricted domain. After the common factors cancel, a proposed relation would combine exponentials with distinct growth rates. At fixed generic $\mu$, rational coefficients grow or decay only as powers of $d$, so they cannot cancel the largest exponential unless its coefficient vanishes. Repeating this argument removes each coefficient. The latest addition supplies 3,001 master entries per loop order and 36,012 entries in total. The preceding extensions supplied 3,289 and 9,000 entries. We count the basis within each family, so the master with $\rho_1 = 1$ recurs across profiles at the same loop order.

We also obtain the master coefficients without following the selected row operations. With $z = Q^2/\mu$, the rational identity

$$\prod_{i=1}^{R}(z+\rho_i)^{-b_i} = \sum_{j:b_j>0}\sum_{k=1}^{b_j} A_{jk}(z+\rho_j)^{-k} \tag{60}$$

defines rational numbers $A_{jk}$. Expanding the other factors around $z = -\rho_j$ gives them directly:

$$A_{jk} = [w^{b_j-k}]\prod_{i\neq j}(\rho_i - \rho_j + w)^{-b_i}. \tag{61}$$

Here $w = z + \rho_j$ is the expansion variable, and $[w^n]$ means the coefficient of $w^n$ in this Taylor series. Combining these coefficients with Eq. (57) yields the independent reference

$$V_{\boldsymbol{b}}^{(L)} = \mu^{1-B}\sum_{j:b_j>0}\sum_{k=1}^{b_j} A_{jk}\left[\prod_{h=1}^{k-1}\frac{L+h-Ld/2}{h\rho_j}\right] M_j^{(L)}. \tag{62}$$

The factor $\mu^{1-B}$ ensures that both sides have mass dimension $Ld - 2L - 2B$. This reference allows us to check the reduction path through a separate calculation.

### 6.14 Reverse target construction and the expanded data division

We supply every Euler seed with $1 \le B \le 5$ and retain the summed Euler identity in Eq. (56) as its source equation together with all mass partial-fraction identities through $B = 6$. Scalar target roots have $2 \le B \le 5$. The source pools contain 35, 160 or 545 equations for $R = 2, 3, 4$, respectively. These counts stay fixed as $L$ changes because the massless powers and massive-index bounds stay fixed. The additional loop orders therefore test this controlled family structure, while large generic IBP systems require wider denominator and numerator domains. After obtaining a checked scalar relation, we reverse its steps to construct the corresponding integral from its master combination. For expressions containing several roots, we first combine their known master expansions and then apply the recorded inverse steps. Each inverse step adds a stated multiple of an exact zero identity, so the master coefficients remain known throughout the construction.

For roots with total powers $B_\nu$, we use target weights of the form

$$\mathcal{T} = \sum_\nu \kappa_\nu d^{\eta_\nu}\mu^{B_\nu - B_*}V_{\boldsymbol{b}_\nu}^{(L)}, \qquad B_* = \max_\nu B_\nu, \qquad \eta_\nu \in \{0,1\}. \tag{63}$$

We choose $\kappa_\nu = n_\nu/q_\nu$, where $n_\nu \in \{-3,-2,-1,1,2,3\}$ and $q_\nu \in \{1,2,3\}$. The powers of $\mu$ give every term the same mass dimension. Each family contains eighteen individual target integrals for $R = 2$ and thirty-two for $R = 3$ or $R = 4$, with full massive sectors included before lower sectors. The remaining targets are weighted expressions with one, two or three distinct roots. Each such expression includes a root with every massive denominator present and at least one additional denominator power. We retain 128 distinct targets per family. The latest addition supplies 1,536,000 targets, following the earlier additions of 140,800 and 384,000 targets.

For each target we retain its master coefficients, exact reverse construction, source equations, sufficient IBP seed set and a forward elimination trace. The trace records all allowed source rows, the chosen pivot, the coefficients before and after the operation, and the arithmetic operation counts. Mass partial-fraction steps remain separately identified, so a seed label always refers to an actual IBP seed. The reference path minimizes the next symbolic row's number of nonmaster terms, then its total support and immediate operation count, with a fixed final tie rule. This supplies a reproducible valid path without establishing a globally minimal sequence.

We verify every source equation against Eq. (62) and separately check the partial-fraction formula and the Euler derivative before integration. We replay the symbolic constructions and forward reductions, then independently replay all saved records at three finite-field samples. A candidate target is rejected if a reference pivot vanishes at one of those samples. The checks also verify connected graphs and their loop counts. At $d = 2 + 1/L$ the radial master integral converges; numerical quadrature then checks Eq. (59) at both ends of the loop-order range.

The complete collection now contains 17,100 family instances, 57,979 family-local master entries and 2,301,272 targets. It contains 2,247 one-loop families, 1,353 two-loop families and 1,350 families at each order from three through twelve. We preserve every earlier grouped model assignment and assign 700 of the latest profiles to training and 300 to testing. A profile has the same assignment at every loop order, giving 1,610,890 training targets in 11,963 families and 690,382 test targets in 5,137 families. Shared lower-sector integrals can occur across different profiles. These tests measure transfer between the specified mass profiles with every loop order represented during training; they do not establish transfer to unseen loop orders or general high-loop topologies. The accompanying records contain the complete definitions and checks [15].

# 7 Machine Learning Model for Seed Selection

To train and test the seed-selection model, we use the target expressions and sufficient seed sets constructed in Section 6. We have implemented a small neural network that assigns a binary decision and its estimated confidence to each supplied candidate seed. We trained the model on the fixed training division and then evaluated it on the held-out targets. This section defines the inputs, probabilities and training procedure, and Section 8 gives the resulting seed predictions.

## 7.1 Model inputs and seed probabilities

A seed specifies an integral index vector at which the solver generates IBP identities. The resulting equations can contain other integrals whose elimination uses identities from additional seed vectors [2]. A target can therefore use several seeds in a sufficient reduction set. Some targets admit a reduction from one seed, while a target already expressed in the chosen masters needs no IBP seed. We formulate the prediction as a separate binary decision for every candidate, so the model can select several seeds for the same target.

For a fixed family and chosen master basis, write a target as

$$\mathcal{T} = \sum_{j=1}^{n_T} c_j(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{C} = \{\boldsymbol{\sigma}_1, \ldots, \boldsymbol{\sigma}_{n_C}\}. \tag{64}$$

Here $n_T$ is the number of terms in the target, $c_j$ is the rational coefficient of its $j$th term, $\boldsymbol{x}$ is the set of kinematic parameters and $d$ is the space-time dimension. The vector $\boldsymbol{a}^{(j)}$ gives the integral indices of that term. The set $\mathcal{C}$ contains $n_C$ candidate seed vectors within the stored family bounds. It supplies all candidates that this implementation scores for the target. Extending the seed bounds would require supplying an enlarged candidate set.

The model receives every term in the stored order, including its indices and symbolic coefficient. Each integral contributes its index values, the locations of positive indices and the quantities $r, s, t$ and $n_{\mathrm{dot}}$ defined in Eq. (14). We transform an index value $a$ to $\mathrm{sgn}(a)\log(1 + |a|)$ and transform the nonnegative counts with $\log(1 + v)$, where $v$ denotes the count. The input also records the number of family indices and which positions are present. The current representation has capacity for 16 indices; the expanded collection uses up to sixteen.

To include the coefficient functions, we separate their written expressions into variable names, integers and mathematical symbols. We encode counts of individual symbols and consecutive groups of two and three symbols in a fixed vector with 64 components. A fixed signed assignment maps each group to a component, and division by the square root of the total number of groups sets the scale. Every coefficient symbol enters this representation, although different groups can share a component. The resulting vector is a compressed description of the coefficient expression. It does not perform symbolic simplification or establish equality between different expressions. In particular, modular residues do not enter as ordinary real coefficient values.

For each term we append its relative position in the stored list and the inverse term count. Denote the resulting term vector by $\boldsymbol{f}_j$. A shared learned map $\phi_\theta$ forms

$$\boldsymbol{h}_j = \phi_\theta(\boldsymbol{f}_j), \qquad \boldsymbol{h}_j \in \mathbb{R}^{32}. \tag{65}$$

Here $\theta$ denotes the adjustable parameters of the model and $\boldsymbol{h}_j$ is a vector describing the term. The same map acts on each term, so its number of parameters does not depend on $n_T$. Combining shared term maps through averages follows the set-based construction of Ref. [16], with additional position information to retain the stored order.

We also include every ordered pair of distinct terms. If $n_T > 1$, their combined description is

$$\boldsymbol{h}_{\text{pair}} = \frac{1}{n_T(n_T-1)} \sum_{\substack{j,k=1 \\ j\neq k}}^{n_T} \psi_\theta([\boldsymbol{h}_j, \boldsymbol{h}_k, \boldsymbol{h}_j - \boldsymbol{h}_k, \boldsymbol{h}_j \odot \boldsymbol{h}_k])\,. \tag{66}$$

Square brackets join vectors, $\odot$ multiplies corresponding components, and $\psi_\theta$ maps each joined vector to 32 components. For an individual-integral target, we set $\boldsymbol{h}_{\text{pair}} = \boldsymbol{0}$. This representation includes both orders of each pair and the positions of the associated terms. The present model uses pairs as its specified form of term combinations; it does not enumerate longer subsets or all permutations of the full expression. The fixed stored term order must also be used when preparing later inputs.

The family information includes the propagator expressions, masses, external scalar products, invariant names, and the numbers of loops, propagators, external momenta and invariant parameters. We encode its symbolic structure with the same fixed 64-component rule and map it to a 32-component family vector $\boldsymbol{h}_F$. A further shared map acts on the index vectors of the chosen masters, and their mean gives the 32-component basis vector $\boldsymbol{h}_M$. This input identifies the basis available for reduction. The target's master coefficients, construction history and recorded seed choices remain outside the input.

Each candidate seed contributes its index and complexity information through a shared map that gives a vector $\boldsymbol{u}_b \in \mathbb{R}^{32}$ for candidate $b$. To associate a seed with particular target terms, we form

$$\begin{aligned} \omega_{bj} &= \frac{\exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_j/\sqrt{32})}{\sum_{k=1}^{n_T} \exp(\boldsymbol{u}_b \cdot \boldsymbol{h}_k/\sqrt{32})}, \\ \boldsymbol{v}_b &= \sum_{j=1}^{n_T} \omega_{bj} \boldsymbol{h}_j. \end{aligned} \tag{67}$$

The weight $\omega_{bj}$ determines how much term $j$ contributes to candidate $b$'s description, and $\boldsymbol{v}_b$ is their weighted mean. This uses the scaled dot-product weighting of Ref. [17]. The weights sum to one over target terms for each seed.

The model then joins the mean and componentwise maximum of the term vectors with the pair, family, basis, seed and weighted-term vectors:

$$\begin{aligned} \boldsymbol{z}_b &= [\text{mean}_j\, \boldsymbol{h}_j,\ \max_j \boldsymbol{h}_j,\ \boldsymbol{h}_{\text{pair}}, \boldsymbol{h}_F, \boldsymbol{h}_M,\ \boldsymbol{u}_b, \boldsymbol{v}_b], \\ g_b &= F_\theta(\boldsymbol{z}_b), \qquad p_b = \frac{1}{1+\exp(-g_b)}. \end{aligned} \tag{68}$$

Here $F_\theta$ has a hidden layer with 64 components and one scalar output $g_b$. The other learned maps each use one layer with 32 components. All hidden layers use the hyperbolic tangent function. The implementation has 28,097 adjustable parameters and shares them across candidates, so reordering the candidate list only reorders the returned scores.

The quantity $p_b$ represents the model's probability that $\boldsymbol{\sigma}_b$ belongs to the recorded sufficient seed set. Several seeds can contribute to the same reduction, so these probabilities are separate and need not sum to one. A high score ranks a seed for selection. The current data establish a valid reference set for each target; they do not establish the probability that a seed belongs to a globally minimal reduction. Exact reduction remains the criterion for deciding whether the selected seeds supply enough equations.

For each candidate we report probabilities for both possible labels:

$$\begin{aligned} P_\theta(y_b = 1 \mid \mathcal{T}, F, \mathcal{M}, \boldsymbol{\sigma}_b) &= p_b, \\ P_\theta(y_b = 0 \mid \mathcal{T}, F, \mathcal{M}, \boldsymbol{\sigma}_b) &= 1 - p_b. \end{aligned} \tag{69}$$

Here $y_b$ is the reference label, $F$ is the family definition and $\mathcal{M}$ is the chosen master basis. Label 1 means membership in the recorded sufficient seed set. Label 0 means absence from that set. The two class probabilities for one seed sum to one, while probabilities for different seeds remain separate.

The binary prediction uses a fixed threshold of one half:

$$\widehat{y}_b = \begin{cases} 1, & p_b \geq \frac{1}{2}, \\ 0, & p_b < \frac{1}{2}. \end{cases} \qquad q_b = \widehat{y}_b p_b + (1 - \widehat{y}_b)(1 - p_b). \tag{70}$$

The value $\widehat{y}_b$ is the returned label, and $q_b$ is the model's estimated probability that this label agrees with the reference label. For example, $p_b = 0.90$ gives label 1 with confidence $q_b = 0.90$, while $p_b = 0.08$ gives label 0 with confidence $q_b = 0.92$. These values illustrate the output rule; they are not trained results. Candidate selection uses $p_b$, while $q_b$ describes confidence in the returned binary decision.

The predicted seed set is

$$\widehat{\mathcal{S}}(\mathcal{T}) = \{\boldsymbol{\sigma}_b \in \mathcal{C} : \widehat{y}_b = 1\}. \tag{71}$$

Its size can vary between targets. Exact reduction must establish whether the identities from this set suffice to reduce the target. The probability $q_b$ concerns one reference-membership decision; it is not a probability that the complete selected set gives a valid or minimum-cost reduction. Section 8 compares these confidence values with their observed agreement with the reference labels.

### 7.2 Training labels and the grouped data division

For each target, let $\mathcal{S}_\mathcal{T} \subseteq \mathcal{C}$ be the sufficient seed set used by its recorded reduction. The reference label for candidate $b$ is

$$y_b(\mathcal{T}) = \begin{cases} 1, & \boldsymbol{\sigma}_b \in \mathcal{S}_\mathcal{T}, \\ 0, & \boldsymbol{\sigma}_b \notin \mathcal{S}_\mathcal{T}. \end{cases} \tag{72}$$

A zero label records absence from this particular set; another valid reduction can use that seed. Algebraic mass partial fractions carry no seed label. For example, a target whose reference uses the first and third of five candidates has

$$\boldsymbol{y}(\mathcal{T}) = (1, 0, 1, 0, 0). \tag{73}$$

The target coefficients and public family information enter the prediction, while this label vector enters the training loss.

We keep each family together and preserve the grouping of related mass and kinematic profiles described in Section 6.12. The expanded division contains 1,610,890 training targets in 11,963 families and 690,382 test targets in 5,137 families. Every loop order from one through twelve occurs in both sets. Each added mass profile stays in one set across all loop orders. The ratio applies to complete target examples, each with its full candidate list. Shared lower-sector integrals can occur across different profiles, and previously reserved families were also evaluated in earlier project stages. The latest mass profiles were assigned before the new parameter updates.

The training objective gives each target equal weight:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}_{\mathrm{train}}|} \sum_{\mathcal{T} \in \mathcal{D}_{\mathrm{train}}} \frac{1}{n_C(\mathcal{T})} \sum_{b=1}^{n_C(\mathcal{T})} \left[ y_b \log p_b + (1 - y_b) \log(1 - p_b) \right]. \tag{74}$$

Here $\mathcal{D}_{\mathrm{train}}$ is the training set and $n_C(\mathcal{T})$ is the candidate count for target $\mathcal{T}$. All positive and zero labels enter this binary cross-entropy without additional class weighting. The threshold of one half determines reported decisions after the model supplies its probabilities.

### 7.3 Completed training and implementation checks

We continued the saved model and its AdamW optimizer on the expanded training set for twenty complete passes. The starting checkpoint had completed 20 passes over the preceding expanded collection. It also retains the earlier training lineage. The latest phase uses the enlarged collection and its own twenty-pass counter. It contains 32,217,800 target presentations and 1,012,000 parameter updates.

The architecture still has 28,097 parameters. AdamW uses a learning rate of $10^{-3}$, weight decay of $10^{-4}$ and a maximum gradient norm of one [18]. Each update uses up to 32 targets from a training family. We change family and target order between passes and use the actual batch size for the final group. Masked batch calculations retain every target term, every ordered pair of distinct terms and every supplied seed. Comparisons with the individual-target calculation verified matching scores, losses and parameter derivatives across individual-integral and multi-term examples. Training uses four CPU threads, while inference uses one. A comparison on training examples verified matching scores and parameter derivatives across the selected thread counts.

The mean training loss changed from 0.0516102 in the first expanded-data pass to 0.008548661 in the twentieth. Figure 1 shows the complete history. These means include parameter changes within each pass and describe fitting on the training examples.

The checks verify target-label correspondence, membership of every recorded seed in the candidate pool, complete coverage during each pass, and equality of the published weights with the resumable checkpoint. The model retains the separate class probabilities and confidence rule in Eq. (70). Held-out examples supplied no gradient updates or threshold fitting.

The representation accepts up to sixteen indices, 512 target terms and 10,000 supplied candidates. The added twelve-loop examples reach the index limit. This capacity describes the input representation; the training evidence concerns the actual family domains defined above. Section 8 evaluates the frozen parameters on their reserved mass profiles.

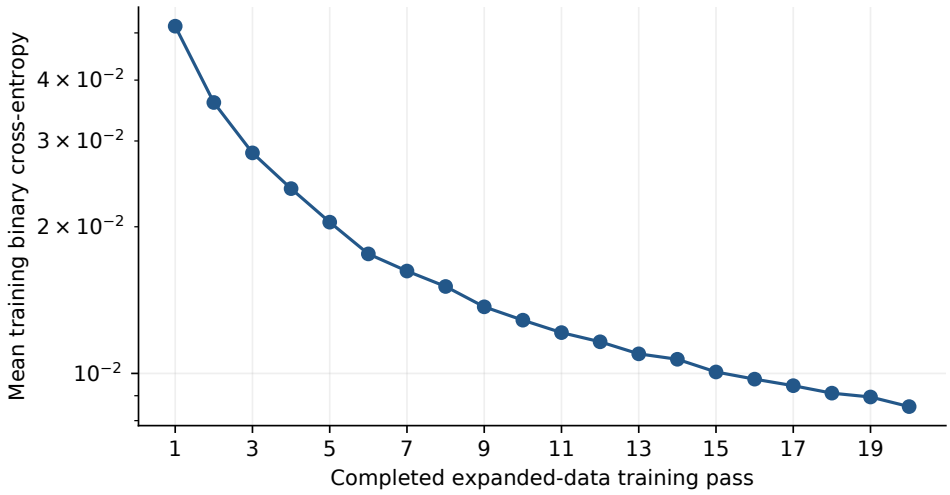



Figure 1: Mean training binary cross-entropy during the twenty expanded-data passes. Each target has equal weight within the reported mean. The logarithmic vertical axis shows the training history; held-out examples do not enter this curve.

# 8 Testing the Seed-Selection Model

To measure seed selection after the expanded training, we froze the weights at the end of the twenty new passes and evaluated all 690,382 targets from 5,137 reserved families. The test contains 148,494 individual integrals and 541,888 integral expressions, with loop orders from one through twelve. It evaluates mass profiles excluded from parameter updates within the family domains of Section 6. The fixed threshold remains $p_b = 1/2$, and no test result changes the weights or the confidence map [19].

## 8.1 Reference agreement and seed ranking

The test supplies 76,536,115 candidate labels, of which 2,346,128 belong to the recorded sufficient sets. A true positive is a selected candidate with reference label one, while a false positive is a selected candidate absent from that set. A false negative is a recorded seed that the model omits. With these counts,

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{precision}\,\text{recall}}{\text{precision} + \text{recall}}. \tag{75}$$

The pooled precision is 95.750%, recall is 96.892% and $F_1$ is 96.318%. The fraction of targets whose complete predicted set equals its recorded set is 84.332%. Candidate-label accuracy is 99.773%; the all-zero rule gives 96.935% because zero labels are frequent. Precision, recall and complete-set agreement therefore describe seed recovery more directly than accuracy alone [20].

To examine the scores without changing the threshold, we rank candidates by $p_b$ and group candidates with equal scores. For a target $T$, let $P_g$ and $R_g$ be the precision and recall after including score group $g$ in descending order. With $R_0 = 0$, the target average precision is [21]

$$\text{AP}_T = \sum_g (R_g - R_{g-1}) P_g. \tag{76}$$

Each increase in recovered reference seeds therefore weights the precision at that point. Averaging over targets with at least one recorded seed gives mean target average precision 99.090%. The first ten candidates recover, on average, 99.787% of recorded seeds and contain every recorded

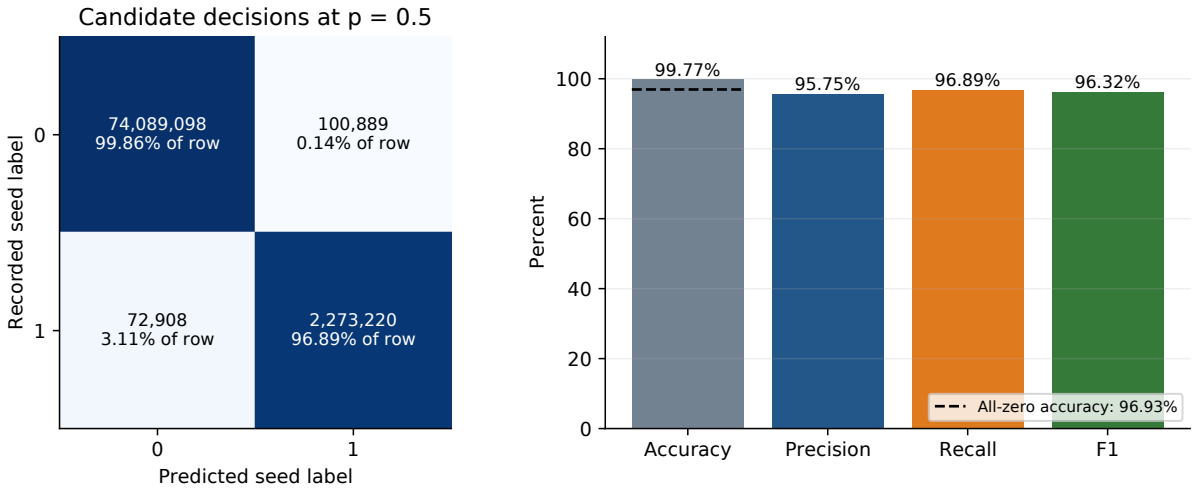



Figure 2: Candidate decisions at the fixed threshold, with pooled precision, recall and $F_1$. Each confusion-matrix percentage is normalized within its reference-label row.

seed for 98.947% of targets in the corresponding nonempty-reference comparison. The score-curve calculation uses 4,096 fixed probability intervals; it supplies a descriptive curve and does not select a new operating threshold.

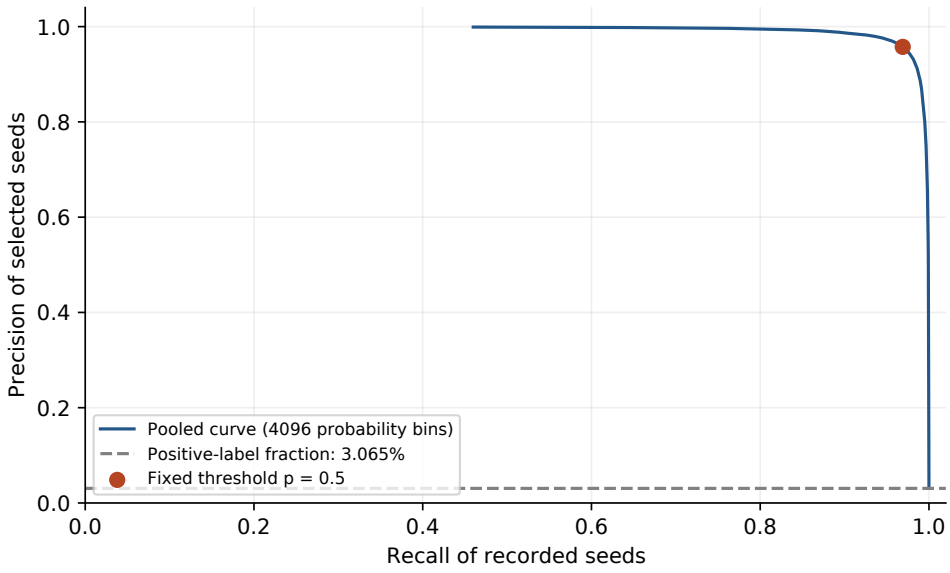



Figure 3: Pooled precision and recall as the score threshold varies. The marked point retains the fixed threshold of one half. The horizontal reference gives the positive-label fraction.

## 8.2 Confidence and dependence on the family

For a returned binary label, the model reports $q_b$ from Eq. (70). We group decisions by this value and compare their mean confidence with the fraction matching the reference. Equality would place a group on the diagonal of Figure 5. These empirical comparisons assess the saved scores without fitting a calibration correction.

Figure 6 separates performance by family and loop order. The one-loop group has 681 families, the two-loop group has 406, and each remaining order has 405 held-out mass profiles. This

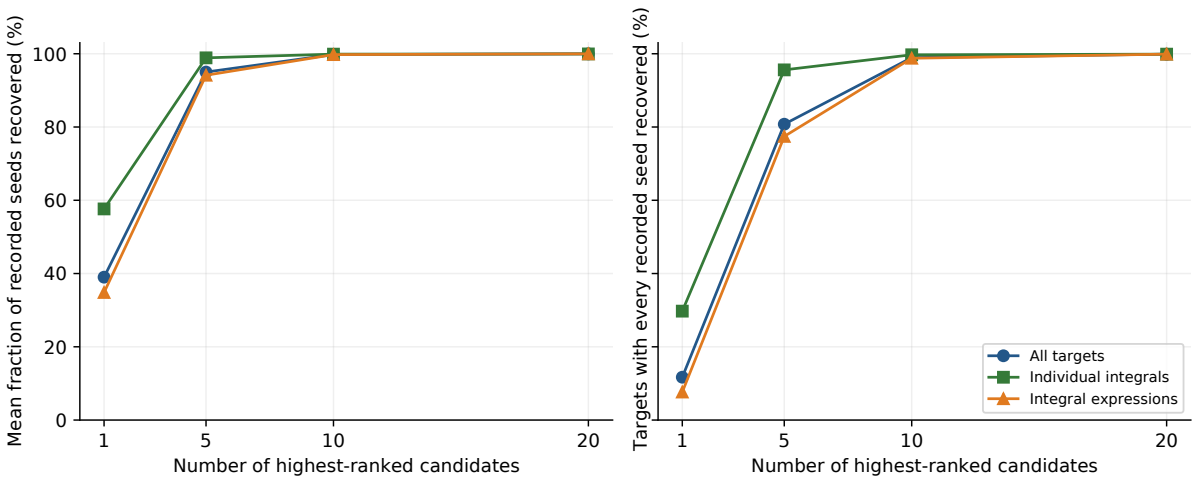



Figure 4: Recorded-seed recovery as the number of supplied highest-ranked candidates increases. The right panel requires every recorded seed to appear in the selected prefix. Individual integrals and integral expressions are also shown separately.

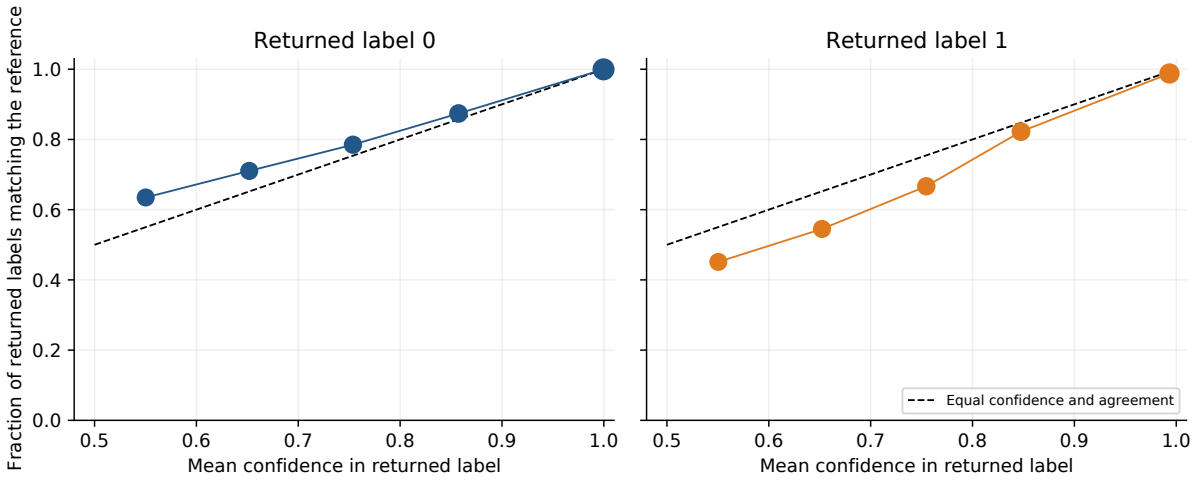



Figure 5: Reported confidence and observed label agreement, shown separately for returned labels zero and one. Marker area increases with the number of decisions in a group.

separation matters because the added vacuum families use different identities and candidate bounds from the earlier one-loop collection. Loop-order comparisons also reflect these differences in the mathematical domain. At one loop the pooled recall is 93.790% and $F_1$ is 92.181%; at twelve loops they are 97.334% and 97.300%, respectively. Across the groups from two through twelve loops, the lowest recall is 96.184% at $L = 6$. These missed reference seeds show why a proposed set still needs an exact sufficiency check before use in a complete reduction.

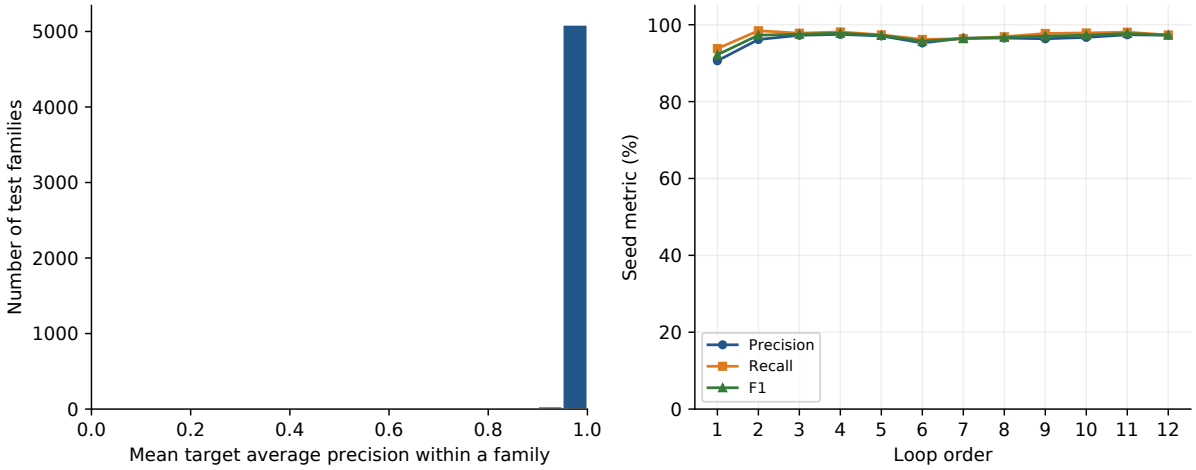



Figure 6: Distribution of mean target average precision across held-out families, and pooled seed precision, recall and $F_1$ at each loop order. The higher-loop groups cover the restricted vacuum mass profiles defined in the text.

### 8.3 Checks and interpretation

Independent checks reproduce the saved confusion counts and target counts, verify the prediction hashes and confirm that the weights remain fixed. All stored probabilities are finite and lie between zero and one. The mean time for input preparation and model inference is 1.6881 ms per target on the test allocation; this excludes a subsequent exact reduction.

The labels identify one recorded sufficient set. Other seed sets can also reduce the target, so disagreement with these labels does not by itself establish failure of the proposed reduction. The classification measurements above describe reference-set recovery and confidence. To determine whether the selected equations complete the target, we next use the saved predictions in Kira.

### 8.4 Sufficiency of the predicted seeds inside Kira

To determine whether the predicted seeds supply enough equations, we use Kira 3.1 to test whether each target belongs to the span of its specified masters modulo the available identities. This means that the target can be written as a linear combination of those masters after using the selected equations. We retain the frozen model, reserved families and threshold of one half from Eq. (71).

Let $\mathcal{E}_{\boldsymbol{\sigma}}$ contain the stored IBP identities from seed $\boldsymbol{\sigma}$, and let $\mathcal{B}$ contain the background identities, including partial fractions and symmetries. A supplied seed set $\mathcal{S}$ provides the equations

$$\mathcal{A}(\mathcal{S}) = \mathcal{B} \cup \bigcup_{\boldsymbol{\sigma} \in \mathcal{S}} \mathcal{E}_{\boldsymbol{\sigma}}. \tag{77}$$

Every selected seed contributes all its stored IBP identities. The reference seed labels, construction paths and master coefficients supply no equations to this test.

The sufficiency condition follows from linear algebra over the field $\mathbb{K} = \mathbb{Q}(\boldsymbol{x}, d)$ of rational functions in the kinematic parameters $\boldsymbol{x}$ and dimension $d$. Let $V$ be the vector space with one formal basis vector for each integral in the supplied system, let $v_{\mathcal{T}}$ be the coefficient vector of the target, and let $\mathcal{R}(\mathcal{S})$ be the row space of the equations in $\mathcal{A}(\mathcal{S})$. Also let $V_M$ be the span of the basis vectors for the specified masters. Then

$$\begin{aligned} \mathcal{T} = \sum_m r_{\mathcal{T}m} M_m \quad \text{using } \mathcal{A}(\mathcal{S}) \quad &\Longleftrightarrow \quad v_{\mathcal{T}} \in \mathcal{R}(\mathcal{S}) + V_M, \\ &\Longleftrightarrow \quad [v_{\mathcal{T}}] = 0 \quad \text{in } V/(\mathcal{R}(\mathcal{S}) + V_M). \end{aligned} \tag{78}$$

Here $M_m$ is a specified master, $r_{\mathcal{T}m} \in \mathbb{K}$ is its reduction coefficient, and square brackets denote the equivalence class after removing combinations of available equations and master vectors. This condition tests whether such coefficients exist without requiring their full symbolic reconstruction. Sufficiency applies to the supplied target expression and permits exact cancellation among its terms.

We implement the condition through Kira's user-defined-equation interface, which accepts homogeneous linear systems with rational coefficients and uses Fermat for symbolic operations [8, 9]. Each target has an auxiliary variable $Z_{\mathcal{T}}$ and the input equation

$$Z_{\mathcal{T}} - \sum_{j=1}^{n_T} c_j(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(j)}} = 0. \tag{79}$$

All terms in this equation come from the public target expression in Eq. (64). We add one unit row for each specified master vector. These rows impose $M_m = 0$ in the formal quotient used for the test, where every possible master combination represents zero. They do not change the physical values of the master integrals.

Kira reduces $Z_{\mathcal{T}}$ exactly in this system. A zero remainder establishes the condition in Eq. (78), and a nonzero remainder shows that the supplied equations leave an independent contribution outside the specified master span. The conclusion applies to rational functions at generic parameter values where their denominators are defined. A successful program exit alone does not establish sufficiency.

Different seed sets use separate copies of every integral variable so that they cannot share equations. Targets with the same selected set can share a system within one family. We retain every equation in a component connected to the target terms and remove only disconnected components. For each target we test its predicted set, the full candidate set $\mathcal{C}$, and the empty seed set. The full set checks the available fallback, while the empty set measures reduction through the target definition, master basis and background identities alone.

We independently reconstruct the selected row spaces from the original identities at each of the three stored finite-field samples. At each sample, sparse elimination checks membership after removing master columns and must agree with Kira's symbolic decision. Each nonzero symbolic remainder also has a regular sample with a nonzero value, which provides an exact witness that its coefficient does not vanish identically. The sample coordinates enter these checks after the symbolic calculation; reference master coefficients remain outside the full sufficiency test. For two pilot families, 765 target-and-seed-set comparisons also agree with reductions that retain the masters and compare their explicit rational coefficients.

### 8.5 Sufficiency results and the role of additional seeds

The test includes all 690,382 reserved targets from 5,137 families. The predicted sets supply enough equations for 652,517 targets (94.515%), while 37,865 (5.485%) have a nonzero remainder

outside the master span. The full candidate pool is sufficient for 690,382 targets (100.000%), and background identities alone suffice for 26,915 (3.899%). Within the coupled-vacuum subset, the predicted sets suffice for 595,919 of 618,240 targets (96.390%). Within the one-loop box subset, they suffice for 56,294 of 71,184 targets (79.082%). For the other one-loop samples, the predicted sets suffice for 89 of 380 bubble targets (23.421%), 152 of 431 triangle targets (35.267%), 60 of 144 propagator targets (41.667%), and 3 of 3 tadpole targets (100.000%). The overall fraction therefore depends on the mixture of tested family domains. All symbolic membership decisions agree with the independent row-space checks at the three stored samples. All requested targets have a recorded outcome, and the final evaluation has no unresolved solver or export errors [22].

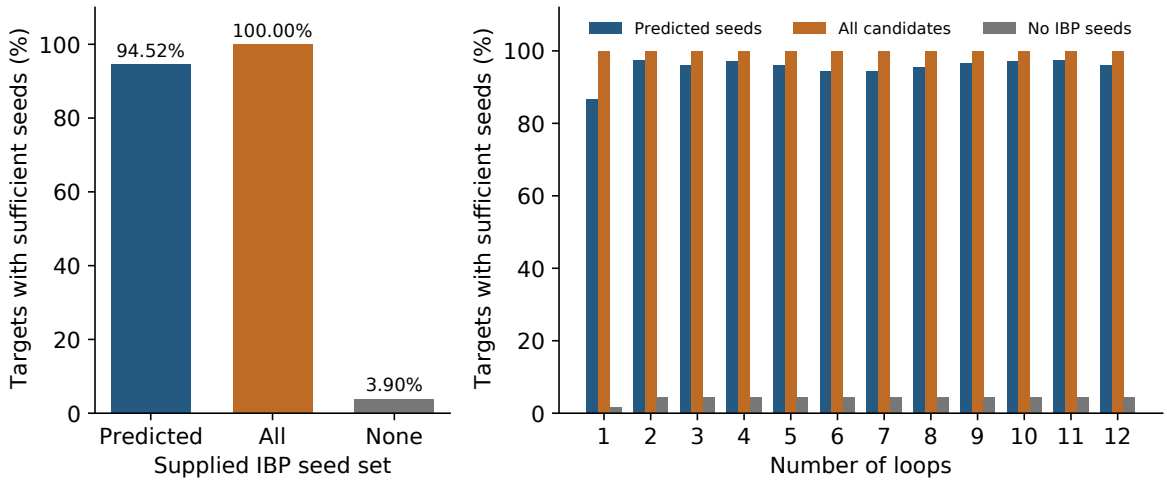



Figure 7: Targets that belong to the specified master span after using the supplied identities, shown for predicted seeds, all candidate seeds and no IBP seeds. Each loop-order fraction uses every reserved target at that order. All three policies retain the same background identities.

Of the targets with sufficient predicted sets, 70,302 use a seed set different from the recorded reference set. This comparison shows that a model can supply enough equations through an alternative selection, so reference-set agreement and algebraic sufficiency measure different properties. The model selects 3.439 seeds per target on average. Across all target–candidate pairs, it selects 3.102% of the supplied candidates. This fraction describes selection within the bounded pools; it does not measure a runtime improvement over Kira's usual strategies.

The all-candidate control supplies a direct check of the fallback within the stored equation domain. A predicted set that leaves extra integrals requires further equations before we accept its reduction. We solve the full-pool control independently, which checks whether adding all candidates would supply a sufficient fallback. This test does not identify the smallest necessary addition or measure reuse of a preceding elimination. Its conclusions concern rational functions at generic parameter values, where the denominators in the reduction are defined.

These results establish algebraic sufficiency rates inside Kira for the stored identities. The user-defined interface is needed for the present family representation, including the dependent massive denominators of the higher-loop vacuum families. An insufficient outcome refers to the stored identities for the selected seeds. Additional valid IBP operators at the same seeds can enlarge that pool. Section 11 separately compares native Kira seeding through learned sector bounds with the stated native baselines. The grouped execution also shares work among targets with identical selected sets, so its elapsed times do not provide an end-to-end solver speedup measurement. The next section addresses the separate learned choice of an allowed elimination step.

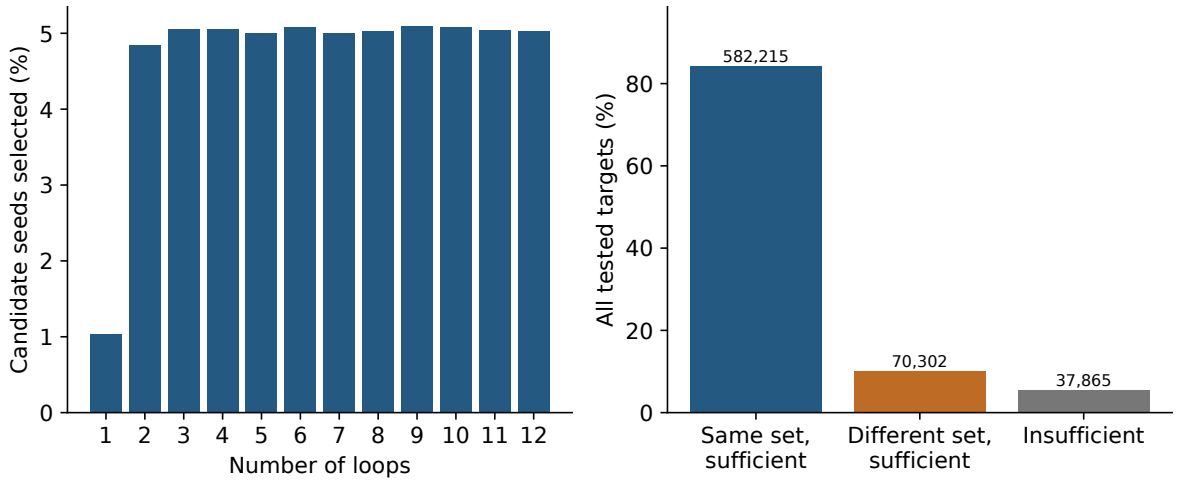



Figure 8: The selected fraction of candidate seeds at each loop order, and the relation between reference-set agreement and seed sufficiency in Kira. Numbers above the bars are target counts. Different sufficient sets account for the difference between the two tests.

# 9 Machine Learning Model for Reduction Propagation

The seed model selects identities from which a reduction can proceed. Once these equations are available, the reduction model selects the next row operation from a supplied list of allowed steps. We use the current coefficient matrix and its integral labels to describe each choice, and the model returns a probability for every listed step. The step with the largest probability gives the proposed next operation. Exact arithmetic then updates the matrix, which supplies the input for the next choice. This section describes the model and its training on the reductions constructed in Section 6.

## 9.1 Current equations and allowed reduction steps

Let $A^{(n)}$ be the coefficient matrix after $n$ reduction steps. A column $j$ corresponds to an integral $I_{\boldsymbol{a}^{(j)}}$, and a row contains its coefficients in an identity or in the target expression currently being reduced. The chosen master basis specifies the columns protected from elimination. An action $a = (q, r, k)$ uses source row $q$ to eliminate column $k$ from destination row $r$:

$$A_{rj}^{(n+1)} = A_{rj}^{(n)} - \frac{A_{rk}^{(n)}}{A_{qk}^{(n)}} A_{qj}^{(n)}, \qquad q \neq r, \quad A_{qk}^{(n)} \neq 0, \quad A_{rk}^{(n)} \neq 0. \tag{80}$$

All other rows remain unchanged during this elementary step. If both rows are identities, this is an elementary row operation. If row $r$ represents a target, row $q$ must be a zero identity so that the target retains its value. The solver supplies the allowed action set $\mathcal{A}_n$, enforcing its integral ordering, master basis and row roles. The model ranks this finite list; completeness of the list remains the responsibility of the solver.

The stored examples describe a particular instance of this matrix. They contain one changing target row and the source identities available for eliminating its largest remaining nonmaster integral under Eq. (36). Each candidate specifies one of these identities, the target row and that integral. After a choice, the source identities remain fixed and the target row changes. The prediction interface also accepts several changing rows, while the present training data cover the stored target-row procedure. Section 12 tests the frozen policy on Kira's evolving selected

IBP matrix using admissible actions that retain its integral ordering [9]. This test changes the inference states while retaining the target-row training described here.

### 9.2 Information supplied to the model

For each allowed action, we construct a vector $\boldsymbol{f}_a$ from the current matrix, the pivot integral and the family definition. The pivot description includes its indices, sector membership, and the values of $r, s, t, n_{\mathrm{dot}}$ in Eq. (14). Descriptions of the source and destination rows give their nonzero counts, master content and the mean and largest integral complexities. A matrix summary gives its number of rows, columns and nonzero entries. The public family definition supplies the numbers of loops and kinematic parameters together with the propagators and scalar-product relations. We encode their written form through the same fixed signed token counts described in Section 7.1, using 32 components here. This encoding admits collisions between different groups of symbols. We transform nonnegative counts with $\log(1+v)$, where $v$ is the count, and each signed index $a_i$ with $\operatorname{sgn}(a_i)\log(1+|a_i|)$. The representation allows up to 16 indices and records which positions belong to the family. These definitions give 132 features for each candidate.

The current records supply coefficients in $\mathbb{F}_P$, the field of integers modulo a prime $P$. We use these coefficients to calculate the overlap of each source row with its destination, the number of terms that the proposed operation would cancel or introduce, and its immediate arithmetic count. For a source with $n_q$ nonzero entries, this count is $1 + 2n_q$: one division and $n_q$ multiplications and subtractions, using the convention in Section 6. These quantities require one sparse trial row update per action. They use only the current matrix and do not inspect later reduction steps or reference master coefficients. A symbolic coefficient can vanish at the chosen sample, so the model uses counts recomputed from the sampled rows. The data audit retains the distinction from the original symbolic support counts. Structural counts and integral descriptions carry this information into the model. The numerical size of a modular residue does not represent the magnitude of a real coefficient.

The representation compresses the supplied matrix into these features, so different states can have the same description. It includes local cancellation information and a summary of the candidate set. These quantities describe the current choices without enumerating complete reduction paths. Forming these features and evaluating the model contribute to the eventual runtime and must be included when measuring a reduction speedup.

### 9.3 Action probabilities and confidence

A shared map $\phi_\theta$ converts each candidate vector into a 32-component vector,

$$\boldsymbol{h}_a = \tanh(W_1 \boldsymbol{f}_a + \boldsymbol{b}_1), \qquad \boldsymbol{u} = \frac{1}{|\mathcal{A}_n|} \sum_{a \in \mathcal{A}_n} \boldsymbol{h}_a, \qquad \boldsymbol{v} = \max_{a \in \mathcal{A}_n} \boldsymbol{h}_a. \tag{81}$$

Here $\theta$ denotes the adjustable parameters, $W_1$ is a learned matrix, $\boldsymbol{b}_1$ is a learned offset, and the maximum acts on each component separately. Sharing the map and combining candidates through averages and maxima makes the scores independent of their position in the supplied list [16]. The model joins $\boldsymbol{h}_a, \boldsymbol{u}, \boldsymbol{v}$ and applies a second map with 64 hidden components to obtain a scalar score $g_\theta(a)$. For a fixed family, master basis and finite-field sample, it returns

$$\begin{aligned} p_\theta(a \mid A^{(n)}, \mathcal{A}_n) &= \frac{\exp g_\theta(a)}{\sum_{b \in \mathcal{A}_n} \exp g_\theta(b)}, \\ \widehat{a}_n = \arg\max_{a \in \mathcal{A}_n} p_\theta(a), &\qquad q_n = p_\theta(\widehat{a}_n). \end{aligned} \tag{82}$$

Thus the probabilities sum to one over the supplied alternatives, and $q_n$ is the reported confidence for the selected step. With one allowed step the probability is one by construction, which provides no evidence of learned discrimination. An empty candidate list returns no proposed operation; the solver must determine whether the reduction is complete or further identities are required.

Training gives these probabilities an interpretation as agreement with the recorded next choice, conditional on the supplied state and actions. Several choices can give valid reductions, and an alternative choice may even lead to a shorter sequence. The reference paths establish valid reductions and their recorded costs; they do not establish global minima. Section 10 compares this confidence with agreement on reserved choices. Confidence does not certify exactness, minimal cost or the success of a complete reduction. Equation (80) and the solver's checks retain responsibility for the algebra.

### 9.4 Training examples and separation from testing

We retain the same grouped family division used for the seed model: 11,963 families with 1,610,890 targets supply training records, while 5,137 families with 690,382 targets remain reserved for testing. Every state from one target stays with that target's family. We read the row before each recorded step and its complete stored list of admissible source equations. The chosen equation supplies the reference action $a_n^\star$. The row after the step and the stored operation counts check the data preparation but remain outside the predictive inputs. The target's construction history, final master coefficients and future choices also remain outside those inputs.

For the set $\mathcal{D}_{\text{choice}}$ of training states with at least two allowed actions, the objective is

$$\mathcal{L}_{\text{red}}(\theta) = -\frac{1}{|\mathcal{D}_{\text{choice}}|} \sum_{(A,\mathcal{A},a^\star)\in\mathcal{D}_{\text{choice}}} \log p_\theta(a^\star \mid A, \mathcal{A}). \tag{83}$$

Each such state has equal weight. States with one admissible action have zero loss and remain in the data audit. The recorded reference choice is the training label; absence of a label on another action does not establish that its reduction is invalid.

We use AdamW [18] with a learning rate of $10^{-3}$, weight decay $10^{-4}$, batches of 256 states and a gradient norm limit of one. We continued the saved expanded-data model and optimizer for twenty complete passes over the enlarged training states. The new phase retains the architecture and optimizer settings while adding 1,250 mass profiles at every represented loop order. Preparation replays every selected row operation with exact modular arithmetic and verifies its final recorded master row. Training shuffles states within bounded groups of families and retains complete resume states. The reserved families remain outside parameter updates and confidence fitting. Earlier project stages evaluated the preceding held-out families; the current experiment also reserves the newly assigned mass profiles at every added loop order.

### 9.5 Completed training and present scope

The expanded training set contains 14,684,059 recorded states from 1,610,890 targets in 11,963 families. Of these states, 5,840,122 supply at least two allowed actions and 8,843,937 supply one action. The records contain 26,775,999 candidate appearances. The unchanged model uses 132 features per candidate and has 10,529 adjustable parameters.

Preparation replays every reference row operation, checks continuity between steps and verifies the final master support. Symbolic coefficients can vanish at the sampled kinematic point, so the model uses the support of the actual sampled matrix. The audit records these differences without changing the exact symbolic references.

We continued the preceding expanded-data model and its AdamW optimizer for twenty complete passes over the enlarged data. Each new pass uses every training state with an action choice; a forced step has probability one and supplies no ranking decision. The new phase contains 116,802,440 choice-state presentations and 463,763 parameter updates. The preserved lineage contains thirty passes over the original smaller collection and twenty passes over the preceding 2,100-family collection. The latest twenty passes use the enlarged collection across all represented loop orders.

Mean training cross-entropy changed from 0.3823373 to 0.3804633 during this phase, while agreement with the recorded action changed from 99.841% to 99.895%. These values describe losses and choices encountered during parameter updates. Section 10 measures the frozen parameters on reserved families.

The implementation retains the original architecture, learning rate, gradient bound and state-batch limits. Training uses four CPU threads, while inference uses one. Parallel preparation reproduces the saved training features and labels exactly. Checks verify equal weights in the inference and resumable checkpoints, complete state coverage and the preserved optimizer lineage. The published implementation retains the inputs, learned parameters and exact reference records [23].

The training domain still consists of one changing target row and a fixed source-equation pool. The new records extend this domain through twelve loops for the coupled vacuum families specified above. They provide evidence about these allowed reduction steps; general sparse elimination inside Kira requires a separate integration and benchmark.

# 10 Testing the Reduction-Propagation Model

To test the model after twenty expanded-data passes, we froze its parameters and evaluated 690,382 targets in 5,137 held-out families. Every loop order from one through twelve occurs in this set. We first compared the predicted next step with the recorded choice, then allowed the model to choose a complete path from the initial target row. Both tests retain exact finite-field row operations [24].

## 10.1 Allowed steps and the comparison rule

At a given state, the candidates are the source identities whose leading nonmaster integral matches the target row's leading nonmaster integral. Both the learned policy and the comparison rule receive this same list and respect the recorded integral order. The source rows remain fixed while elimination changes the target row. This restriction makes the available actions and complete reference answers explicit.

For each allowed source row $E$, let $u_E$ be the target row after the corresponding exact elimination. The local comparison rule selects the smallest tuple

$$(n_{\mathrm{nm}}(u_E),\ \mathrm{nnz}(u_E),\ 1 + 2\,\mathrm{nnz}(E),\ \mathrm{id}(E))\,. \tag{84}$$

Here $n_{\mathrm{nm}}$ is the number of nonmaster columns with nonzero coefficients, nnz is the total number of nonzero coefficients and $\mathrm{id}(E)$ is a fixed equation identifier used only to resolve ties. The third entry counts one division and one multiplication and subtraction for each source entry. The rule compares the immediate next rows; it does not search all complete reduction paths.

### 10.2 Next-step agreement and confidence

The reserved traces contain 6,364,695 states, including 2,558,472 with an action choice and 3,806,223 with one allowed action. On states with a choice, the learned model selects the recorded action in 95.845% of cases and places it among its first three candidates in 99.980%. Its selected action gives the same next sampled row as the reference in 95.846% of cases. The local rule selects the recorded action in 99.473% of these states.

When $n_{\text{tie}}$ actions share the largest saved probability, the program selects one through its fixed identifier order. Such ties occur in 37.747% of choice states. We also assign credit $1/n_{\text{tie}}$ when the recorded action belongs to the highest-scoring set, and zero otherwise. Averaging this credit gives 74.506% reference agreement. This diagnostic describes the scores while retaining the stated deterministic choice rule for the actual reductions.

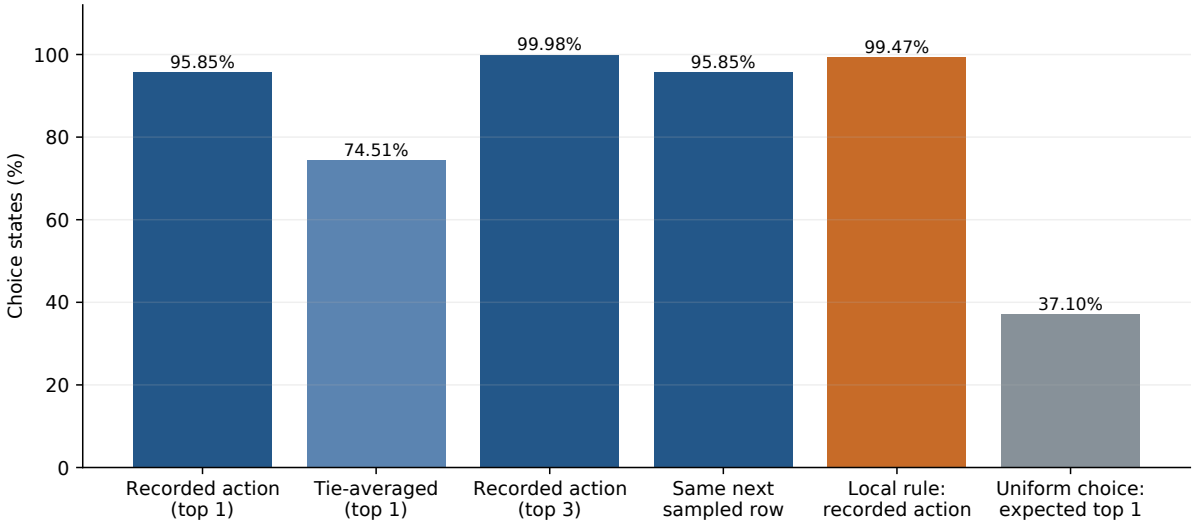



Figure 9: Next-step comparisons on states with at least two allowed actions. The uniform reference assigns equal probability to every available action. The tie-averaged result shares credit equally among highest-scoring tied actions. The same-row comparison permits different identities that produce the same sampled next row.

The model assigns a probability to every supplied action, with probabilities summing to one. We compare the selected action's reported probability with observed agreement after grouping predictions into ten fixed confidence intervals. Let $\mathcal{D}^{\text{test}}_{\text{choice}}$ be the set of held-out states with at least two allowed actions, and let $a^{\star}_{n}$ be the recorded action at state $n$. Applying the negative logarithmic loss in Eq. (83) to this set gives 0.400146. The calculation replaces probabilities below $10^{-38}$ by $10^{-38}$ when taking their logarithms.

To compare the full probability distribution with the recorded choice, we also calculate the multiclass Brier score [25],

$$B_{\text{test}} = \frac{1}{|\mathcal{D}^{\text{test}}_{\text{choice}}|} \sum_{n \in \mathcal{D}^{\text{test}}_{\text{choice}}} \sum_{a \in \mathcal{A}_n} \left[ p_\theta(a \mid A^{(n)}, \mathcal{A}_n) - \mathbf{1}_{a=a^{\star}_{n}} \right]^2 . \tag{85}$$

Here $\mathcal{A}_n$ is the supplied action set and $\mathbf{1}_{a=a^{\star}_{n}}$ equals one for the recorded action and zero for every other action. We sum over all actions without dividing by their number, then give each state equal weight. The resulting score is 0.250632. The confidence comparison in Figure 10 uses the frozen scores without fitting a correction. A forced action has probability one, so these ranking and confidence comparisons exclude forced states.

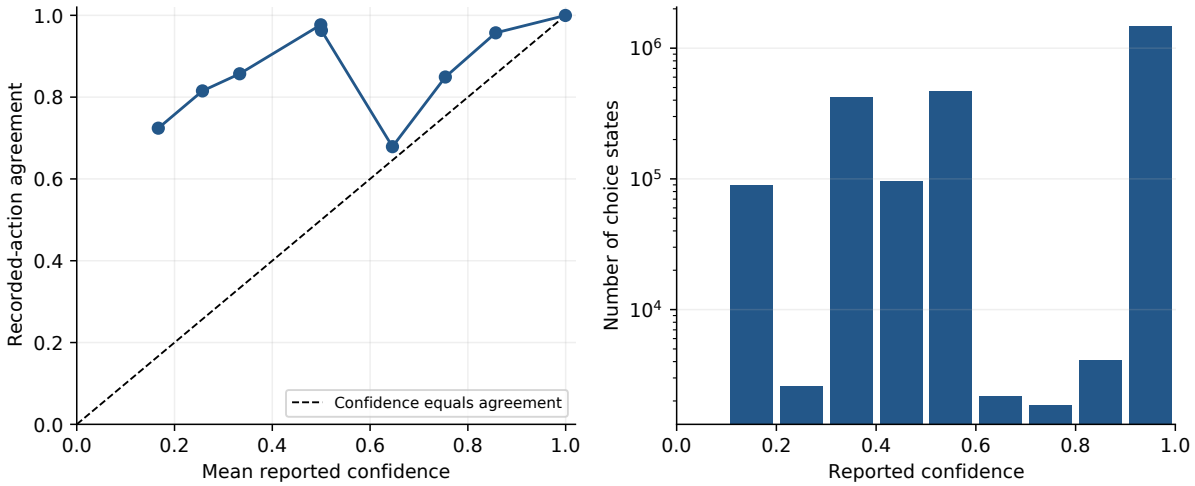



Figure 10: Reported confidence and recorded-action agreement, with the number of choice states in each confidence interval. The diagonal represents equality between confidence and agreement.

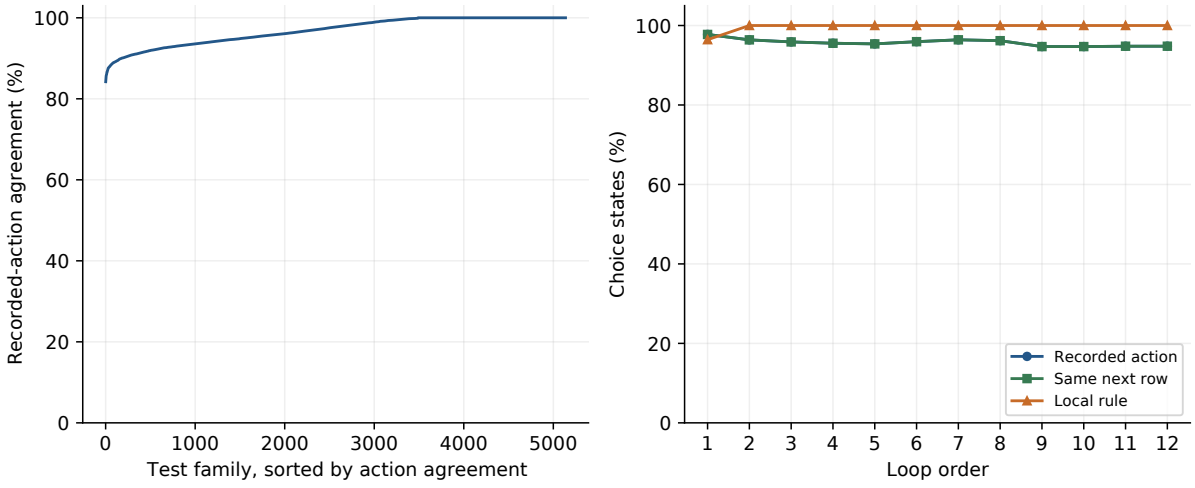



Figure 11: Agreement across individual families and loop orders. Every higher-loop group uses held-out mass profiles of the restricted vacuum families. These comparisons include changes in the mathematical family domain as well as loop order.

## 10.3 Complete reductions and arithmetic work

For complete-path testing, the policy receives the initial target row and repeatedly selects an allowed identity until the row contains only protected masters or a stated failure condition occurs. Decisions use the first finite-field sample. We then replay the selected sequence at both additional samples and compare the resulting rows with the exact reference master coefficients evaluated at those points.

The learned policy matches all three sampled master expansions for 690,302 of 690,382 targets (99.988%). The local rule matches all three for 689,700 targets (99.901%). The respective numbers without this full three-sample agreement are 80 and 682. For the learned policy, 77 targets stop when the supplied pool has no allowed identity for the remaining leading integral, and 3 complete at the decision sample but fail an additional-sample check. The local rule gives 664 and 18 in these two categories. Agreement at the samples is the stated test criterion; the test does not reconstruct a new symbolic rational result from the learned path.

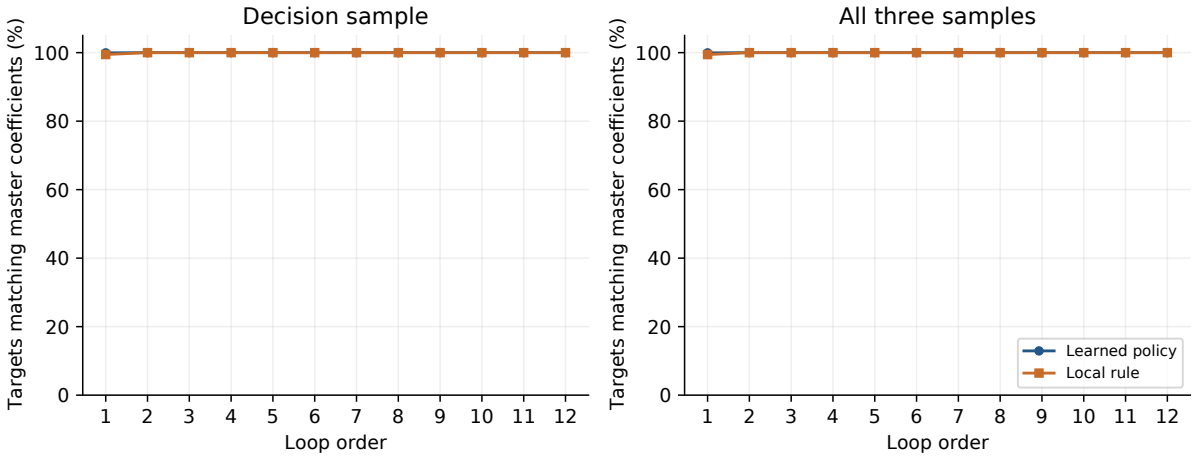



Figure 12: Complete target reductions at each loop order. The left panel requires agreement at the decision sample; the right panel requires agreement at all three samples.

We compare arithmetic counts on the 689,684 targets for which both policies pass all three samples and the reference count is nonzero. The learned policy uses fewer committed row operations than the local rule for 6,846 targets, the same number for 610,196 and more for 72,642. A committed operation changes the selected target row. Candidate trials used to construct model features or evaluate the local rule also perform arithmetic, so Figure 13 shows both accounting choices. The recorded path supplies a valid reference count without a proof that this count is minimal.

## 10.4 Timing, checks and scope

The mean attempted reduction time is 12.3438 ms per target for the learned policy and 0.6991 ms for the local rule. The learned policy therefore takes 17.657 times as long on this allocation. These times include action selection and committed elimination within the target-row experiment. We alternate policy execution order between targets and keep verification and disk output outside the timed interval. Four CPU workers on the same compute node process disjoint families, with one computation thread per worker. Both policies run in the same worker for each target. Figure 14 shows the timing distributions on the test allocation.

Independent checks reproduce probability summaries and replay the committed finite-field operations from the saved paths at the decision sample. They also verify the source coefficients,

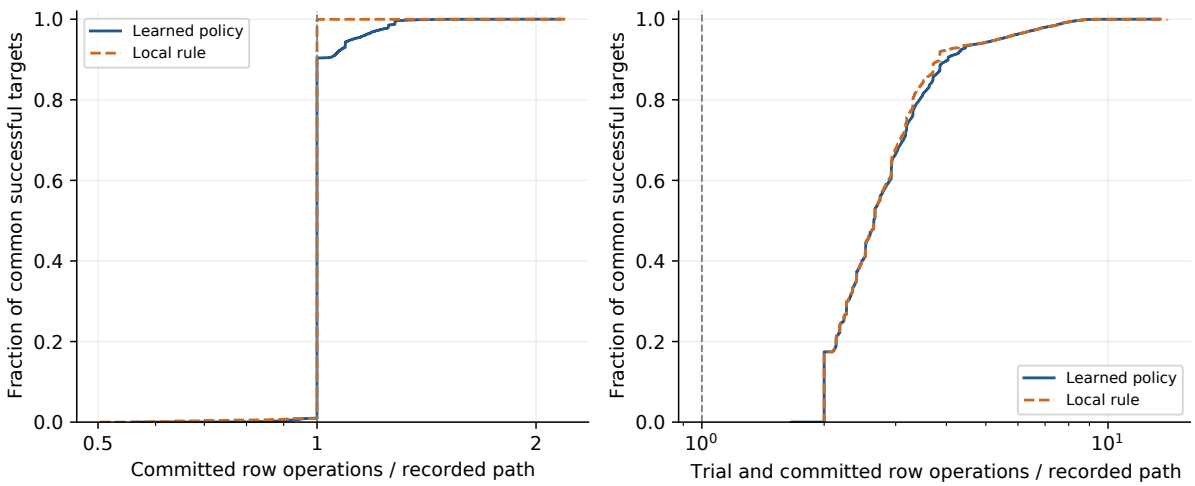



Figure 13: Cumulative distributions of arithmetic counts relative to the recorded path, restricted to targets successful for both policies. The right panel also includes trial row operations used when choosing an action. Both horizontal axes use logarithmic scales.

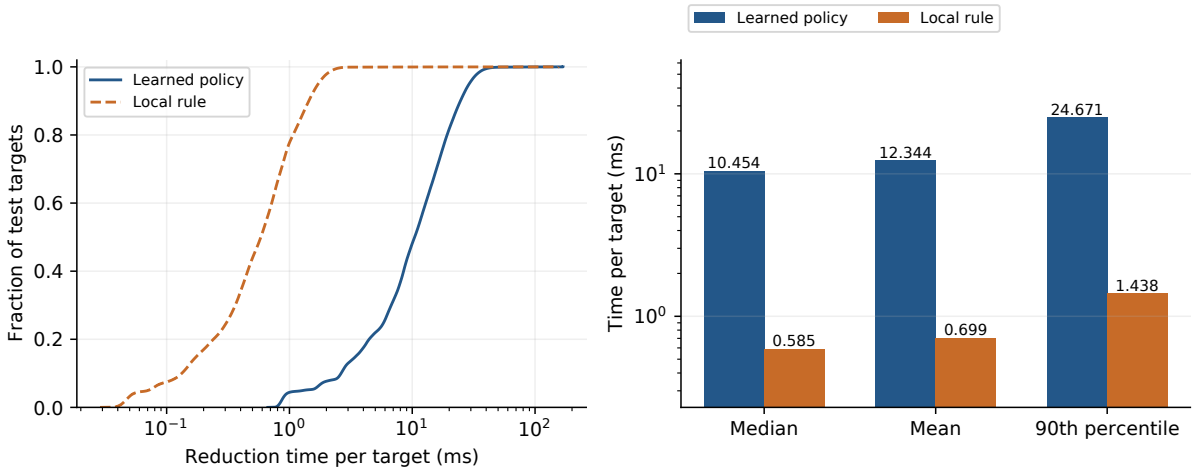



Figure 14: Attempted target-row reduction times for the learned policy and local rule. Both receive identical initial rows, source pools and integral ordering.

data split and unchanged model hash. The tests distinguish agreement with a recorded next action from completion of an allowed alternative path.

The results concern the supplied equation pools and family domains. Every tested loop order also occurs in training, and related lower-sector integrals can occur in different mass profiles. Consequently, the experiment measures held-out profile performance within these domains. It does not establish globally optimal seed sets, globally minimal elimination paths or an end-to-end Kira speedup. Section 11 then measures complete symbolic reductions with Kira, including model and fallback costs.

# 11 Comparison with Kira

A useful learned choice must reduce the cost of obtaining the final master coefficients. The sufficiency test in Section 8.4 establishes which stored seed sets permit a reduction, while the preceding target-row test measures individual reduction paths. We now compare complete symbolic reductions with Kira's existing equation selection and seeding. The comparisons retain both trained models, the held-out families and the seed threshold of one half. The candidate seed lists and prescribed master bases are inputs; their earlier construction is outside the measured reduction costs.

## 11.1 Targets, baselines and exact checks

We use 19 held-out family instances: one restricted vacuum profile at each loop order from one through twelve, three box profiles, and one bubble, triangle, tadpole and two-loop propagator family. The three box profiles represent the smallest, middle and largest candidate-pool sizes in that domain. A fixed hash order selects the other profiles and the target expressions without consulting previous model outcomes. Nested batches contain 1, 4 and 16 targets, with batches of 1 and 3 for the three-target tadpole family. This gives 56 batch cases and 291 distinct targets. The restricted vacuum profiles retain the dependent denominators described in Section 6.12; their loop orders do not represent the complexity of general multiloop scattering.

For the supplied-equation comparison, Kira receives the connected part of the complete stored source pool and applies its own equation selection, forward elimination and back substitution. The learned seed method takes the union of the predicted seed sets for the requested batch and supplies the corresponding stored IBP identities together with the same background identities. If any target retains an integral outside the prescribed master basis, the method reruns the batch with the complete pool. Both attempts contribute to its cost. This comparison isolates selection from a common source library; generating that library is outside these timings.

For the row comparison, the learned model chooses source identities using a fixed finite-field sample, while Eq. (80) updates the target coefficients symbolically in $\mathbb{K} = \mathbb{Q}(\boldsymbol{x}, d)$. Kira then reduces any remaining terms. A second version applies the local rule from Section 10 through the same symbolic procedure. This control identifies the cost of the learned choice relative to the local choice. Both methods retain fixed source identities and a changing target row, which is the procedure covered by the training data.

We compare each final coefficient with its constructed rational reference and independently evaluate the coefficients and saved row operations at three finite-field points. The physical master coefficients remain present throughout this experiment. A reported completion therefore includes the full symbolic target expansion and its checks. Each batch-method pair runs three times on the same compute node, with the execution order rotated. Kira and the model each use one computation thread. At most two trials run concurrently on the cluster.

### 11.2 Learned seed bounds in native Kira

The seven families with independent propagator bases also permit native Kira equation generation. Kira reduces the component integrals, and its exact algebra combines their results into the requested target expressions. We record generation and reduction separately from this final combination. Native equation counts refer to the component reduction; the later combination has separate counts and contributes to every reported total time. For the default baseline, the seeding bounds are one above the largest values of $r$ and $s$ among the target components and prescribed masters. The restricted baseline also bounds $n_{\mathrm{dot}}$ one above its largest required value and uses Kira's subsector truncation with $l = N - s_{\max}$, where $N$ is the number of propagators. A third control sets $r$, $s$ and $n_{\mathrm{dot}}$ directly to the largest target/master values, uses the same subsector truncation, and falls back to the default bounds if needed. Both this control and the learned method reserve 60 seconds for the first native attempt and 100 seconds for its fallback, within the complete 180-second limit. These fixed settings retain Kira's own symmetry handling and equation selection [9]; they do not represent an exhaustive search over its settings.

To apply the seed model before native equation generation, let $\widehat{\mathcal{S}}_S$ be the predicted seeds in sector $S$. For every nonempty set, we pass the bounds

$$\begin{aligned} r_S &= \max_{\boldsymbol{\sigma}\in\widehat{\mathcal{S}}_S} r(\boldsymbol{\sigma}), \qquad s_S = \max_{\boldsymbol{\sigma}\in\widehat{\mathcal{S}}_S} s(\boldsymbol{\sigma}), \\ n_{\mathrm{dot},S} &= \max_{\boldsymbol{\sigma}\in\widehat{\mathcal{S}}_S} n_{\mathrm{dot}}(\boldsymbol{\sigma}). \end{aligned} \tag{86}$$

to Kira. These bounds enclose the selected seeds, and Kira can generate additional seeds within the bounds and inherited subsectors. Its own generated and selected equation counts therefore measure the resulting work. An incomplete first attempt triggers the default native bounds. This direct comparison repeats all four native methods on the largest fixed target batch in each of the seven families, giving 99 distinct targets and 84 trials. The learned method uses the same model and threshold as the supplied-pool test.

### 11.3 Measured cost and results

We record the complete elapsed time from process start to a checked result. It includes input loading, model loading, feature preparation, inference, exact algebra, any fallback, coefficient checks and output. We also record the time with the family already loaded and the model ready for use. This second quantity represents repeated use of prepared inputs; the complete process time retains the cost of starting the model. For a matched case, we compare the median of the three repeats and define

$$\rho_T = \frac{\operatorname{median} T_{\mathrm{learned}}}{\operatorname{median} T_{\mathrm{baseline}}}. \tag{87}$$

Thus $\rho_T < 1$ indicates a lower measured time for that case. Aggregate ratios use the geometric mean of the paired case ratios. Trials that reach their time or memory limit remain in the completion counts and are excluded from ratios that require two completed results.

We distinguish supplied equations, equations selected by Kira, logged forward insertions and coefficient-field operations. For a committed target-row step with $n_q$ nonzero source entries, the last count is $1 + 2n_q$, as in Section 9. Feature construction also performs trial updates of candidate rows. These counts do not measure the internal polynomial work needed to simplify a rational coefficient. Runtime includes that work. Memory is the largest sampled sum of resident process memory for the driver, Kira and its algebra process; the sampling interval is 50 ms, and shared pages can contribute to more than one process's count. Every trial has a 180-second limit, a 1.5 GiB address-space limit per process and a 32 MiB scratch limit. The

comparisons use the existing symbolic Kira 3.1 build with Fermat. They do not benchmark finite-field reconstruction.

The main comparison contains 792 trials. The full stored-pool baseline completed 168/168, learned seeds completed 168/168, learned rows completed 168/168, and local-rule rows completed 168/168. On the 56 completed matched batch cases, learned seed selection gave complete-process and prepared-input time ratios of 1.720 and 1.116, respectively. It used the full-pool fallback in 24 trials. Across those batches, its supplied source-equation count, including fallback, was 0.251 times the baseline count, while the number of equations selected by Kira was 1.027 times the baseline. The distinction shows how Kira's existing selection changes the work left after an external seed choice. The learned method had lower complete-process median time in 0 of 56 batches and lower prepared-input median time in 15 of 56 batches.

The learned row procedure gave time ratios of 1.719 and 1.205 against the full stored-pool baseline over 56 matched cases. Its ratios against the local row rule were 1.520 and 1.043. Before the final Kira call, the learned row procedure completed every target in 54 of the 56 batches, while the local rule did so in 55. Kira completed the remaining terms in the other batches. For the 53 batch cases in which both row procedures themselves finished all targets before the final Kira call, their summed committed coefficient-operation counts were 19,401 and 18,881, giving a ratio of 1.028. This comparison excludes paths that left part of the reduction to Kira. The elapsed-time measurements retain all of that later work, as well as candidate feature preparation and exact coefficient simplification [26].

Kira's own equation selection removes much of the redundant input in the full-pool baseline. The learned seed filter reduces the supplied pool, and its measured complete cost includes the added model work and every fallback. Both learned methods have aggregate complete-process and prepared-input ratios above one in this comparison. The row-operation ratio also exceeds one relative to the local rule on the matched complete preprocessing paths. These results identify added cost in the current implementations and give no aggregate reduction in the stated time or committed-operation measures.

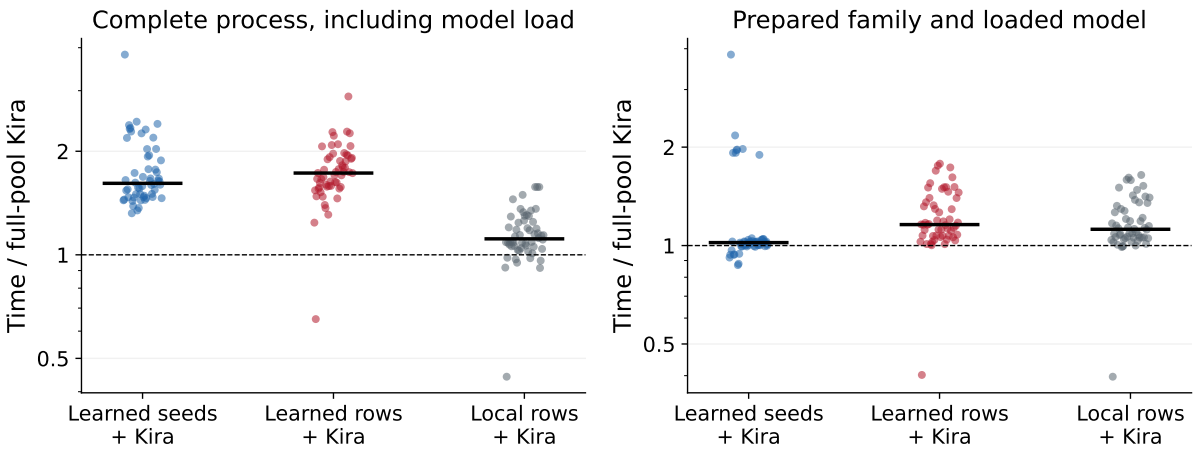



Figure 15: Matched time ratios relative to Kira with the full stored source pool. Each point uses a batch's three-repeat median. The left panel includes process and model loading; the right panel uses prepared family inputs and a loaded model. Short solid lines show the median case ratio; the dashed line denotes equal time.

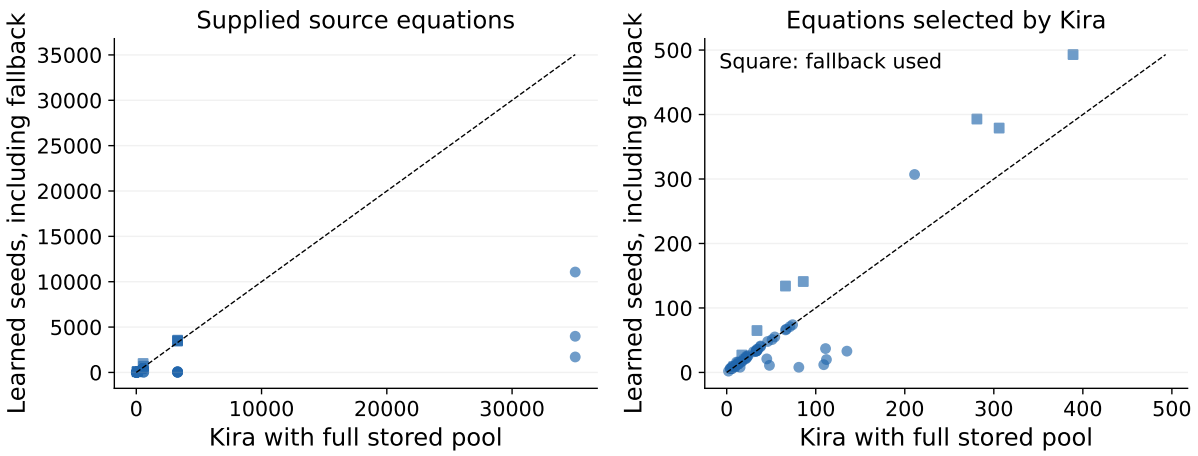



Figure 16: Supplied source equations and equations selected by Kira for identical batches. Learned-seed counts include any full-pool fallback. Squares identify batches with fallback, and the diagonal denotes equal counts.

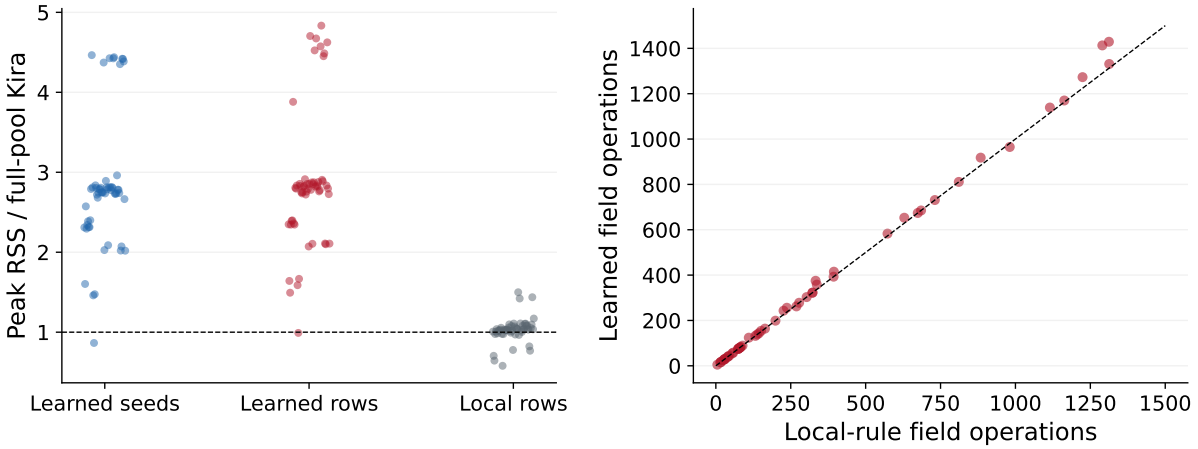



Figure 17: Sampled summed resident-memory ratios and committed coefficient-operation counts. Memory ratios use the largest recorded peak within each three-repeat group. The operation panel compares only batches that both row procedures completely reduced before the final Kira call. These counts exclude polynomial simplification and candidate-scoring work.

### 11.4 Native seeding results

The direct native-seeding comparison completed 21/21 learned trials, 21/21 default-baseline trials, 21/21 restricted-baseline trials and 21/21 target-index-bound control trials. Learned bounds required a default-bound fallback in 9 trials. For the 7 paired completed family batches, its complete-process and prepared-input time ratios against default native Kira were 1.596 and 1.344. Against restricted native Kira, the corresponding ratios were 1.671 and 1.373 over 7 paired batches. Against the control that sets bounds directly from target indices, the ratios were 1.425 and 1.141 over 7 paired batches. These measurements include native equation generation, exact reduction, target combination, coefficient checks and any repeated attempt.

The learned native method had lower complete-process median time in 0 of 7 matched batches against the default baseline, 0 of 7 against the restricted baseline and 1 of 7 against the target-index control. Across the completed paired batches, the ratios of summed initial equation counts were 0.624, 1.144 and 0.881, respectively. These counts include repeated attempts. The learned bounds generate fewer equations than the stated default bounds, and the restricted deterministic baseline generates still fewer equations on these batches. The learned method has aggregate complete-process and prepared-input time ratios above one against all three native controls. Thus the current native integration adds total cost under the tested settings.

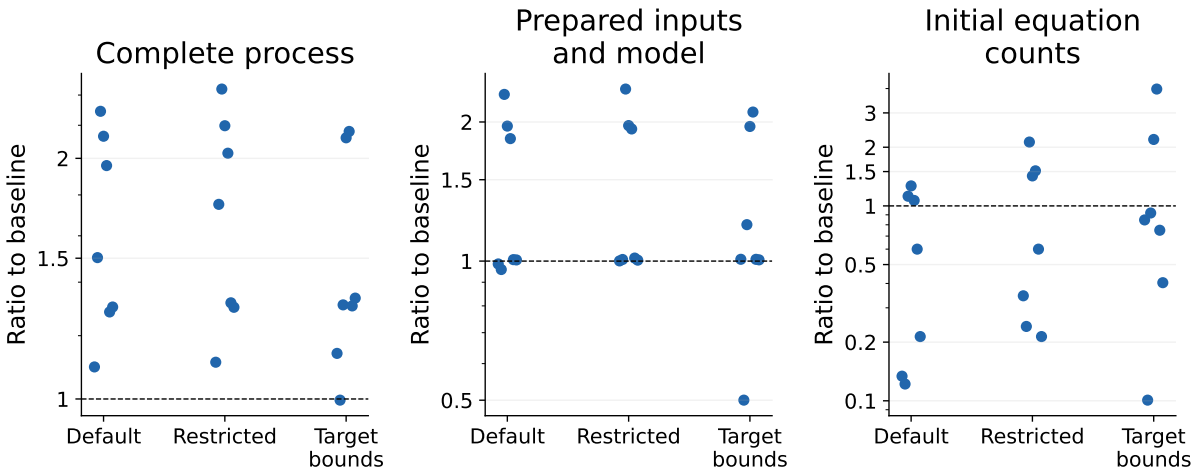



Figure 18: Direct native-seeding comparison on the largest frozen batches in seven independent-basis families. Ratios compare learned sector bounds with the three stated native baselines. Equation counts sum the initial system sizes that Kira reports for each attempt; internal regeneration can perform additional work.

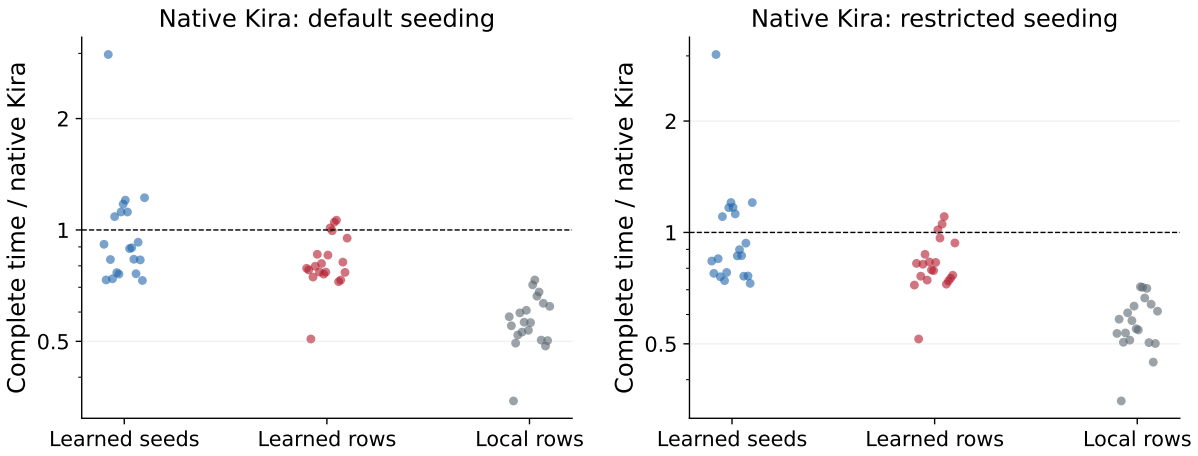



Figure 19: The methods using stored identities compared with native component reduction at the same targets and master basis. Native timings include equation generation and exact combination of the component results. This comparison changes the available source representation as well as the external method, so it does not isolate the benefit of learning.

Across both comparisons, all 876 trials produced a checked complete result within the stated bounds. The paired timing ratios above use three completed repeats for both methods. The recorded limits and three-repeat design bound the interpretation of small timing differences on the shared cluster.

These results describe the implemented policies and the stated symbolic Kira settings. The stored-pool tests reuse previously generated identities, and the native tests measure learned sector bounds that can include seeds beyond the model's selected set. The row comparison in this section acts on fixed source identities and target rows. Section 12 extends the test to changing identity rows inside Kira. General high-loop scattering families and finite-field reconstruction require further tests. Reduced input counts alone do not establish a speed improvement because equation selection, model execution, exact simplification and fallback all contribute to the total cost.

# 12 Learned Elimination Inside Kira's Evolving Matrix

The row comparison in Section 11 used fixed source identities to simplify target expressions before the final Kira call. To test the intended use inside the solver, we now apply the frozen row model during Kira's symbolic forward elimination. The input is the current selected IBP matrix, and every exact row update changes the state used for the next prediction. Kira continues to generate and select equations, preserve its integral ordering, normalize the triangular system and perform back substitution [9].

## 12.1 Choices in the current IBP matrix

We retain the matrix notation of Eq. (80). At step $n$, $A^{(n)}_{rj}$ is the coefficient of integral $j$ in identity row $r$. Let $w(j)$ be Kira's ordering weight for integral column $j$, with larger weights denoting integrals to eliminate earlier. The leading column of a nonzero row $r$ and the rows with a common leading column are

$$h_n(r) = \arg\max_{j:A^{(n)}_{rj}\neq 0} w(j), \qquad R^{(n)}_k = \{r : h_n(r) = k\}. \tag{88}$$

At each step we take the largest-weight column that leads at least two rows. If this column is $k_n$, the allowed choices are

$$\mathcal{A}_n = \{(q, r, k_n) : q, r \in R_{k_n}^{(n)},\ q \neq r\}. \tag{89}$$

Here $q$ is the source row and $r$ is the destination row. The prescribed master columns remain protected. A proposed elimination of a protected master, or a zero pivot at the public sampling point, causes the learned trial to fail its interface check.

The model assigns probabilities to every pair in Eq. (89) and selects the largest score. Kira then uses its existing symbolic update to calculate

$$A_{rj}^{(n+1)} = A_{rj}^{(n)} - \frac{A_{rk_n}^{(n)}}{A_{qk_n}^{(n)}} A_{qj}^{(n)}. \tag{90}$$

All these rows represent zero identities, so this operation preserves their solution space. The new destination row has a lower leading integral, or becomes zero and leaves the matrix. It can supply a source row in a later step. The next prediction therefore uses the updated identities throughout the matrix.

We use the saved 132-feature representation and model weights without retraining. The family definition, master labels and current rows supply the features; construction records and reference master coefficients remain outside the predictions. Rational coefficients at the fixed public finite-field point supply cancellation features. Kira and Fermat perform the actual row updates with symbolic rational coefficients. The process holding the model remains available during one forward solve, and it receives the initial matrix followed by each changed row. A system with no allowed row updates does not load the model.

### 12.2 Matched controls and measurements

We use the same seven families with independent master bases and 99 targets as the native comparison in Section 11. They include three one-loop box profiles, a one-loop bubble, triangle and tadpole, and a two-loop propagator. All methods use the same restricted native seed bounds, target components, preferred masters and symbolic output checks.

Four methods distinguish the learned choice from changes needed to measure it. Stock Kira uses the installed solver. Instrumented Kira retains that solver's forward algorithm and records its exact updates. The deterministic control uses the largest repeated leading integral defined above and selects a source by Kira's row-length and integral-order rule. A stable row identifier selects its destination. The learned method uses the same serial schedule and selects both rows from the allowed pairs. This control separates the model's decisions from the effect of the new schedule. Both methods return the triangular system to Kira for normalization and back substitution.

Each method runs three times per family on the same node, with rotated execution order and one computation thread. Complete elapsed time includes preparation, inference, exact algebra, target combination and coefficient verification. We also time the forward-elimination phase directly inside Kira and record model loading separately. The diagnostic quantity

$$T_{\text{fwd,load excluded}} = T_{\text{fwd}} - T_{\text{model load}} \tag{91}$$

subtracts that recorded loading interval; it retains feature construction, scoring, communication and exact row updates. It is a timing decomposition of these runs; a separate run with a preloaded model has not been measured. Ratios use three-repeat medians and the paired geometric mean of Eq. (87). Forward-stage ratios include only cases with actual row updates

in both methods. The tadpole batch needs no forward row updates and does not contribute to those phase ratios.

For each committed update, we retain the convention $1 + 2n_q$, where $n_q$ is the source-row nonzero count. This counts coefficient-field operations and excludes the internal polynomial work in rational simplification. Elapsed time includes that work and the candidate trial updates used to prepare features. The tests keep the action capacity at 10,000, the matrix capacity at 10,000 rows and 250,000 nonzeros, and use bounded time and memory. These limits bound the systems and action sets tested here.

### 12.3 Exact checks and measured results

All 84 final trials produced the prescribed symbolic master coefficients. Independent evaluation of the archived native exports passed 3,564 checks of target coefficients at independent sample points, and the saved row updates passed 12,708 independent finite-field checks. The model made 1,413 choices across its repeated runs. In 465 choices, its source row had already changed in an earlier elimination. These records establish that the model acted on an evolving identity matrix [27].

The complete elapsed-time ratios for the learned method were 1.305 against stock Kira, 1.286 against instrumented Kira, and 1.274 against the deterministic control. Within forward elimination itself, the ratio against instrumented Kira was 276.149 over 6 active family batches. After subtracting the recorded model-loading interval, the ratio was 62.270. For those active batches, the native forward-stage medians ranged from 0.003267 to 0.076143 seconds, while the learned medians ranged from 3.061 to 6.909 seconds.

Summing one median operation count per family gives 5,771 coefficient operations for the learned method, 5,070 for instrumented Kira and 5,097 for the deterministic control. The corresponding learned and native row-update totals were 471 and 470. These sums count each family once, while the independent checks above cover all repeated runs. Figures 20 and 21 show the family-level measurements.

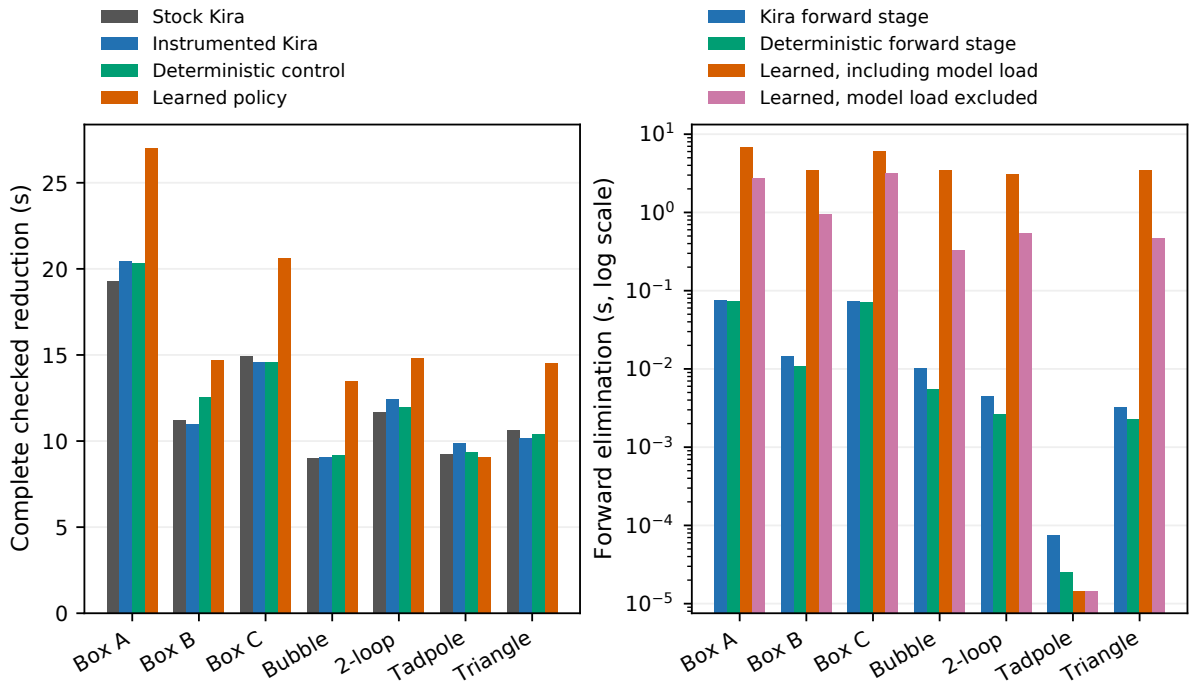



Figure 20: Complete checked reduction time and the forward-elimination time measured inside Kira. Bars show three-repeat medians. The forward panel uses a logarithmic time axis and separately shows the learned time after subtracting the recorded model-loading interval. The tadpole case has no row updates, so its forward time measures phase overhead. Box A, B and C denote profiles 0173, 0633 and 0156, respectively.

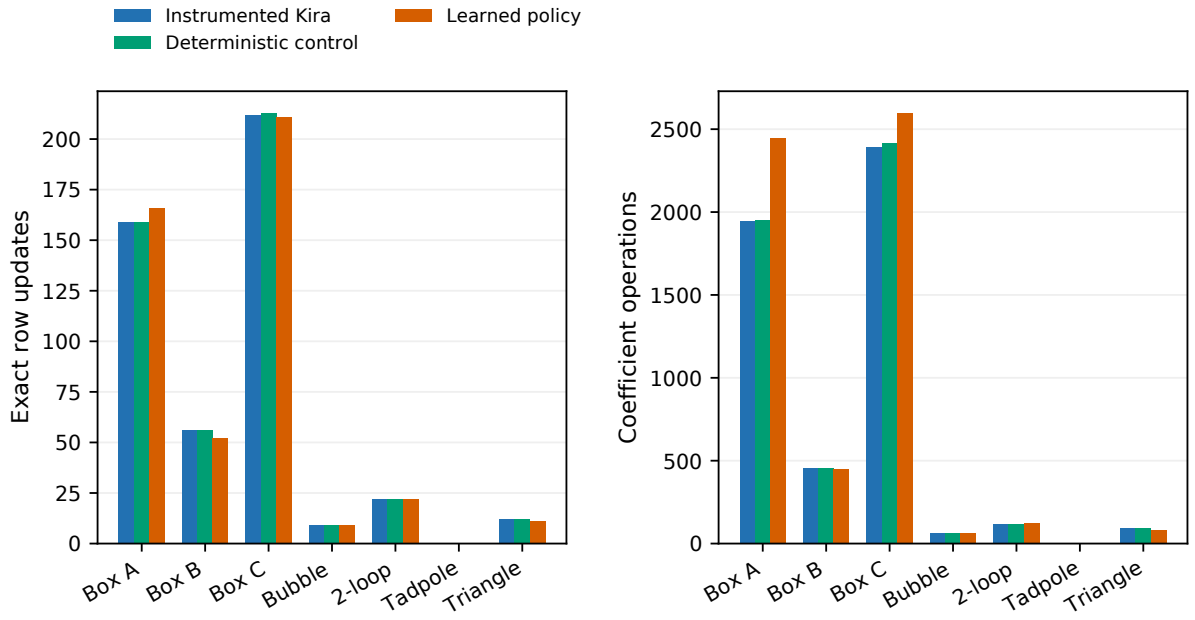



Figure 21: Committed row updates and coefficient-field operations for the same evolving matrices. Each update contributes $1 + 2n_q$ operations. The count excludes candidate scoring and polynomial simplification, whose costs remain in the time measurements.

### 12.4 Implications for the proposed reduction policy

This experiment tests learned choices inside Kira's changing selected IBP matrix. The measured cost includes the consequences of those choices for later rows and for the final exact reduction. The current model increases aggregate runtime, and its summed coefficient-operation count also exceeds the native count. Excluding model loading does not produce a forward-elimination speedup in this comparison.

Kira's native forward phase is already short for these selected systems. The added feature construction and scoring therefore have a small available time budget. The model was trained on target-row reductions with fixed source identities, so its application to changing identity rows also tests transfer beyond those training states. The results apply to the stated one- and two-loop families and a forward policy that preserves Kira's integral ordering. Larger independent multiloop systems, other admissible pivot-column choices and learned back-substitution order remain outside the present experiment.

## 13 Direct Seed Filtering and Lower-Cost Model Inference

The preceding comparisons show that a learned choice must save more algebraic work than its prediction costs. We therefore changed the interfaces between the frozen models and Kira, and measured the complete reductions again. The seed interface can now select individual seeds before Kira constructs their equations. We also tested a smaller set of row choices that follows the form of the reduction model's training examples. The original checkpoints remain unchanged.

### 13.1 Relating the predicted seeds to Kira's targets

A stored target can contain several integrals,

$$\mathcal{T}_\nu = \sum_{\boldsymbol{a}\in\mathcal{J}_\nu} c_{\nu\boldsymbol{a}}(\boldsymbol{x}, d) I_{\boldsymbol{a}}. \tag{92}$$

Here $\nu$ identifies a target, $\mathcal{J}_\nu$ is its set of component index vectors, and $c_{\nu\boldsymbol{a}}$ is the known coefficient of each component. The native interface asks Kira to reduce these components and then combines their reductions with exact algebra. This procedure provides two possible inputs for the seed model: the complete expression $\mathcal{T}_\nu$, or each component $I_{\boldsymbol{a}}$ with coefficient one.

Let $\mathcal{C}$ be the stored candidate seed pool and let $p_\theta(\boldsymbol{b} \mid \mathcal{T}_\nu)$ denote the frozen model's score for seed $\boldsymbol{b}$, with the family and chosen masters also supplied. For a threshold $\tau$, the expression-based seed set is

$$\widehat{\mathcal{S}}_{\mathrm{expr}}(\tau) = \bigcup_{\nu} \{\boldsymbol{b} \in \mathcal{C} : p_\theta(\boldsymbol{b} \mid \mathcal{T}_\nu) \geq \tau\}. \tag{93}$$

The component-based set instead uses

$$\widehat{\mathcal{S}}_{\mathrm{comp}}(\tau) = \bigcup_{\boldsymbol{a}\in\bigcup_\nu \mathcal{J}_\nu} \{\boldsymbol{b} \in \mathcal{C} : p_\theta(\boldsymbol{b} \mid I_{\boldsymbol{a}}) \geq \tau\}. \tag{94}$$

Components already in the prescribed master basis need no prediction. A seed set sufficient for the combined expression can differ from a set sufficient for all components separately, since terms can cancel in Eq. (92). Both interfaces therefore require a check of the final expression in the prescribed master basis.

We also test a local enlargement of a predicted set,

$$\mathcal{N}_1(\mathcal{S}) = \left\{ \boldsymbol{b} \in \mathcal{C} : \min_{\boldsymbol{a} \in \mathcal{S}} \sum_{i=1}^{N} |b_i - a_i| \leq 1 \right\}. \tag{95}$$

Here $N$ is the number of family indices. This rule adds candidate seeds that differ by one unit in one index. It supplies nearby IBP relations without using the reference reduction. It gives no general proof that the set suffices.

### 13.2 Filtering before equation construction

The earlier native adapter represented predictions by sector bounds. Those bounds include intermediate seeds even when the model omitted them. The new interface still supplies bounds that contain the chosen seeds, but it also checks each seed vector before Kira constructs its IBP equations. A vector outside the selected set contributes no IBP equations through this call. Kira retains its symmetry handling, equation selection, exact elimination and back substitution [9]. The audit records both the allowed seed vectors and the vectors that the generator actually used.

The model scores and the reference master coefficients serve separate purposes. Predictions use only public family, target, master-label and candidate information. After reduction, we check whether the target contains any integral outside the prescribed master basis. An incomplete learned attempt triggers a standard restricted native calculation within the remaining time allowance. The final symbolic coefficients are then compared with the recorded exact answer. Prediction, preparation, unsuccessful attempts, fallback, target combination and exact checks all contribute to the measured complete time.

### 13.3 Reducing prediction costs and choosing a source row

We exported the trained weights to a NumPy implementation of the same layers. This removes the need to import the training framework for each reduction, and it reuses the family, master and candidate embeddings within a target batch. On 64 development targets, the seed probabilities differed from the original implementation by at most $4.18 \times 10^{-6}$, and every decision at threshold 0.5 agreed. Separate bounded feature checks verified the exported row network. These checks concern numerical inference; exact reduction still comes from Kira and the final coefficient checks.

For the row model, we first retained the full changing-matrix interface of Section 12 with the exported network. We then restricted each decision to a source row for one current destination. Using the leading-column group $R^{(n)}_{k_n}$ defined there, the largest row identifier selects the destination $r_n$, and the model scores

$$\mathcal{A}^{\mathrm{source}}_n = \{(q, r_n, k_n) : q \in R^{(n)}_{k_n},\ q \neq r_n\}. \tag{96}$$

The feature input contains the current rows in this group, with the destination assigned the target role. Thus the model sees one changing destination and its allowed sources, as in its training examples. Kira applies the selected exact update from Eq. (90). The source can itself have changed in an earlier step. The candidate count falls from $g(g-1)$ to $g-1$ for a group of $g$ rows, while Kira's ordering still fixes the pivot column. If the group contains only two rows, the supplied action set has one element, so we apply that forced action without evaluating the network.

We also implemented the source-row network inside Kira's compiled reduction code. Each choice reads the current rows directly, samples their rational coefficients at the fixed finite-field

point and forms the same input features. The trained weights and features retain their single-precision values, while the compiled layer arithmetic uses double precision. An independent implementation with the same arithmetic precision checks the probabilities, and development tests also compare selected sources with the original single-precision network. This numerical prediction only selects a valid source; Kira continues to perform the rational row update exactly.

The compiled interface removes per-choice communication with a Python process. It retains an audit of chosen rows and their probabilities, and the measured time includes this audit. A combined method applies both the learned seed filter and the compiled row policy during the same reduction. An incomplete seed attempt uses the same exact fallback rule as the seed-only method.

This restriction changes the model's supplied state and action set. Its purpose is to reduce prediction work and test transfer to a closer training configuration. It does not establish an optimal elimination order, and its benefit must be measured against Kira's native choices.

### 13.4 Checking and expanding an incomplete seed set

In separate development trials, we tested whether Kira's initial finite-field master check could reject an incomplete seed set before symbolic elimination [9]. This check compares the integrals left by the generated system with the prescribed master basis. A rejected set can then be enlarged before Kira starts the more expensive symbolic calculation. The final coefficients still require exact checks, since a numerical rank check alone does not prove the full symbolic reduction. We retained the frozen policies in the reserved comparison while testing these extensions.

For the first expansion rule, we rank each candidate seed by its largest model score across the supplied expressions,

$$Q_\theta(\boldsymbol{b}) = \max_\nu p_\theta(\boldsymbol{b} \mid \mathcal{T}_\nu). \tag{97}$$

We begin with the first $k_0 = 32$ seeds and, after a rejected master check, increase the budget according to

$$k_{j+1} = \min(2k_j, |\mathcal{C}|). \tag{98}$$

Here $j$ is the number of completed expansion steps and $\mathcal{C}$ is the candidate pool defined above. A control ranks seeds by their smallest index distance to a component of a target expression, and uses the same budget rule. The full candidate pool and restricted native seeding provide two further controls.

A second rule begins with 64 seeds and uses the additional integrals returned by the master check as new model inputs. Each such input contains one integral with coefficient one. We add its four highest-ranked unused candidates and repeat the check, using the full pool after five rejected checks. A distance-based control applies the same expansion to the same additional integrals. These checks require every component to reduce in the prescribed basis, which can be stricter than reducing the complete expression when terms cancel.

The 26 development trials gave 23 exact completions and three bounded incomplete calculations. The latter used 64-target batches and affected the native control and both learned variants in the initial check experiment. All completed outputs passed 1104 independent finite-field target checks. Across four complete 16-target comparisons, the doubling rule with learned ranking took 1.022–1.422 times the native control's complete time. In the subsequent residual-driven trial, learned ranking reached a complete basis with 71 seeds, while the distance control used 121. Its complete-time ratio against native seeding was 0.996. These are single development measurements; they do not establish a repeatable speedup for either expansion strategy [28].

## 13.5 Development, validation and reserved comparisons

We selected family instances before the new timing comparisons. Development and adapter-validation instances use separate kinematic groups from the original model-training partition. The reserved cost-test instances come from the original test partition, and their kinematic groups exclude every box group in the earlier cost comparison. The network weights remain fixed throughout this selection. The adapter-validation families belong to the original training partition, so this stage selects an interface configuration and does not measure generalization of the network itself. We performed development comparisons on three of the reserved development instances, selected the seed configuration on six validation instances and then froze the policy before testing twelve instances from six reserved kinematic groups.

All new native cases use symbolic reduction of one-loop box families with independent propagator bases. Each case uses the same target expressions and master basis for every method. Controls include restricted native seed bounds, bounds taken directly from target indices, and the full stored candidate pool with the individual-seed filter. These controls separate the learned selection from changes to seed bounds or equation generation.

Every trial uses one computation thread with explicit time, memory and scratch limits. We increased the array allowance from two simultaneous jobs to four and then eight after checking the available node memory and project storage. This changed the scheduling of independent trials while preserving their individual resource limits and frozen policies. Methods run in rotated order on the same node for each family and target-batch size. The retained-evidence limit interrupted some batches before a new trial could start; after lossless archive recompression, we resumed the missing trials on their original nodes and retained every completed observation. Every reserved method receives the same initial 160-second native calculation allowance within a 180-second complete-process limit. The memory and scratch limits are also the same. We retain failed trials and report their completion status. The validation rule assigns the 180-second total cap to an incomplete trial when choosing the learned seed configuration. This capped score is a selection rule; it is not a measured time for a completed reduction. The reserved comparison uses the frozen configuration, five repetitions and batches of 16 and 64 stored target expressions. After an unsuccessful attempt, the remaining identical repetitions for that method and batch are marked as skipped. Skipped repetitions receive no measured time. Timing ratios require five successful repeats for both methods, while completion and failure counts retain every attempted case.

## 13.6 Reserved results and exact checks

The frozen comparison contains 720 planned records across twelve family instances, two target-batch sizes, six methods and five repetitions. We executed 616 trials and obtained 590 exact completions; the remaining 26 executed trials exceeded a resource limit or failed to complete. The early-stop rule left 104 repetitions unexecuted. These skipped records contain no timing measurement. The retained inputs, policy choices, successful reductions and failure records accompany the data release [28].

Validation selected the complete target expression with threshold $\tau = 0.01$. This configuration and both trained checkpoints remained fixed during the reserved tests. Each method used the same master basis and targets in a batch.

For the three controls, restricted native Kira produced 105 exact completions, 3 failed trials and 12 skipped repetitions, the target-bound control produced 100 exact completions, 4 failed trials and 16 skipped repetitions, and the full candidate-pool control produced 105 exact completions, 3 failed trials and 12 skipped repetitions. These counts describe the bounded experiment and

do not measure whether an unlimited calculation would complete.

Successful restricted-native trials took 10.54–112.46 seconds for the complete checked calculation. This range includes different family instances, target-batch sizes and CPU models. The paired ratios compare methods on the same node within each batch.

For the learned methods, the learned seed filter produced 100 exact completions, 4 failed trials and 16 skipped repetitions, the compiled row policy produced 105 exact completions, 3 failed trials and 12 skipped repetitions, and the combined method produced 75 exact completions, 9 failed trials and 36 skipped repetitions. Completed seed and combined trials used the standard fallback 30 and 50 times, respectively.

Every completed final expression matched its recorded rational master coefficients. The independent audit performed 67,920 additional master-coefficient sample checks and 65,355 exact-row sample checks. It checked 11,405 compiled model choices and 10,380 forced source actions. Trace checks cover completed native attempts; 55 failed attempts had incomplete traces. The final coefficient checks still apply to any successful fallback. Input and checkpoint hashes agree with the values frozen before testing.

## 13.7 Complete reduction time and algebraic work

For each family and batch size, we take the median of five successful repetitions. We then form the ratio of learned time to control time and report the geometric mean across complete paired batches. A ratio below one means a lower measured time. We resample complete kinematic groups 10,000 times to obtain descriptive 95% intervals, keeping related profiles and batch sizes together. These intervals describe the completed subset of this small collection; failed and skipped trials remain in the completion counts above.

Restricted native Kira completed all five repetitions in 1 batches where the learned seed filter did not, while the learned seed filter completed all five repetitions in 0 batches where that control did not.

The learned seed filter has a complete-time ratio of 1.295 (descriptive 95% interval 1.156–1.542; 20 batches) against restricted native Kira. Its native-calculation subtotal, including seed prediction when used, has ratio 1.400 (descriptive 95% interval 1.201–1.816; 20 batches). The subtotal excludes input preparation, final target combination and verification, which remain included in the complete time.

Restricted native Kira completed all five repetitions in 0 batches where the compiled row policy did not, while the compiled row policy completed all five repetitions in 0 batches where that control did not.

The compiled row policy has a complete-time ratio of 1.024 (descriptive 95% interval 0.998–1.053; 21 batches) against restricted native Kira. Its native-calculation subtotal, including seed prediction when used, has ratio 1.025 (descriptive 95% interval 1.001–1.078; 17 batches). The subtotal excludes input preparation, final target combination and verification, which remain included in the complete time.

Restricted native Kira completed all five repetitions in 6 batches where the combined method did not, while the combined method completed all five repetitions in 0 batches where that control did not.

The combined method has a complete-time ratio of 1.983 (descriptive 95% interval 1.575–2.476; 15 batches) against restricted native Kira. Its native-calculation subtotal, including seed prediction when used, has ratio 2.289 (descriptive 95% interval 1.247–3.673; 8 batches). The subtotal excludes input preparation, final target combination and verification, which remain included in

the complete time.

Against the target-bound control, complete-time ratios are 0.902 (descriptive 95% interval 0.711–1.163; 20 batches), 0.712 (descriptive 95% interval 0.597–0.864; 20 batches), 1.415 (descriptive 95% interval 0.902–2.160; 14 batches), in seed, row and combined order. Against the full candidate-pool control, the corresponding ratios are 1.317 (descriptive 95% interval 1.167–1.569; 20 batches), 1.040 (descriptive 95% interval 1.016–1.061; 21 batches), 2.028 (descriptive 95% interval 1.565–2.616; 15 batches). Each ratio uses its own complete paired cohort, so comparisons with different failure patterns can contain different batches.

The learned seed filter has a generated-equation ratio of 0.168 (descriptive 95% interval 0.097–0.294; 20 batches), a selected-equation ratio of 0.982 (descriptive 95% interval 0.797–1.205; 20 batches) and a forward-update ratio of 3.575 (descriptive 95% interval 2.647–5.211; 20 batches) against restricted native Kira. These are counters from the native reduction passes, including completed fallback passes.

The compiled row policy has a generated-equation ratio of 1.141 (descriptive 95% interval 1.032–1.275; 21 batches), a selected-equation ratio of 1.141 (descriptive 95% interval 1.032–1.275; 21 batches) and a forward-update ratio of 0.905 (descriptive 95% interval 0.840–0.972; 17 batches) against restricted native Kira. These are counters from the native reduction passes, including completed fallback passes.

The combined method has a generated-equation ratio of 0.462 (descriptive 95% interval 0.254–0.745; 15 batches), a selected-equation ratio of 1.341 (descriptive 95% interval 1.080–1.582; 15 batches) and a forward-update ratio of 2.363 (descriptive 95% interval 1.041–4.534; 8 batches) against restricted native Kira. These are counters from the native reduction passes, including completed fallback passes.

Generation counters count equations reported in successive generation passes and can include repeated equations. A forward update is one committed elimination step; this count does not measure every polynomial or rational-function operation. Failed attempts can have truncated work counters, so the complete time and completion status remain the primary measures.

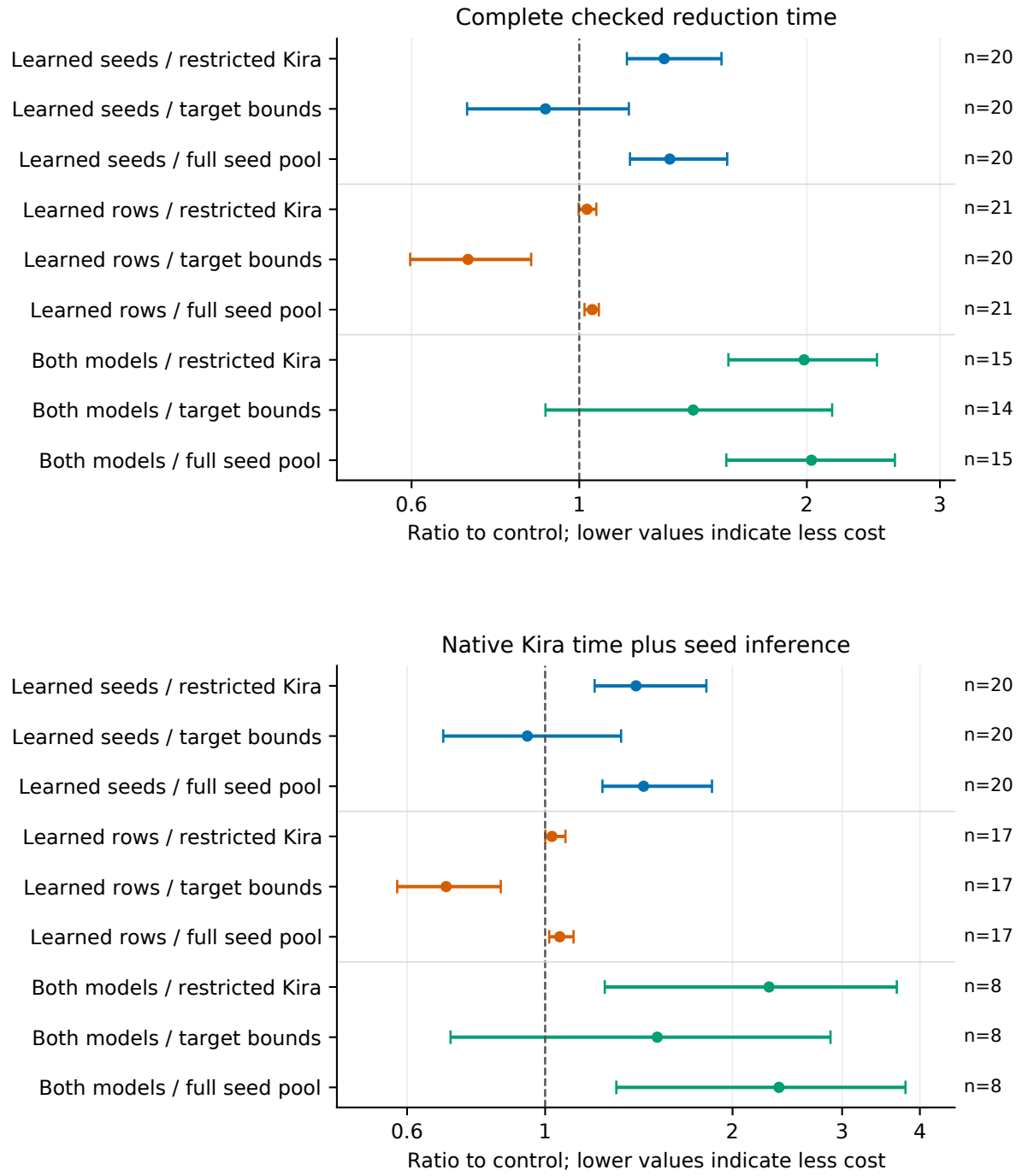
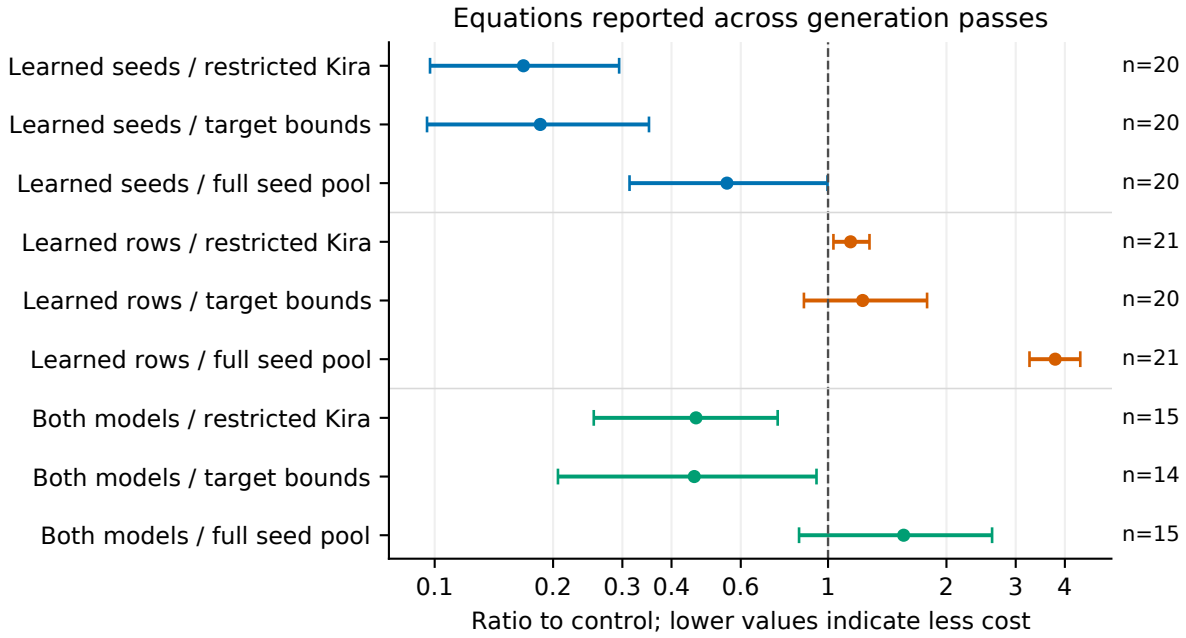



Figure 22: Learned-to-control time ratios in the reserved native comparison. The upper panel includes the complete checked reduction, and the lower panel shows the native subprocess subtotal plus seed prediction. Points show geometric mean ratios and lines show descriptive intervals obtained by resampling kinematic groups. The label $n$ gives the number of paired batches with five successful repeats for both methods.



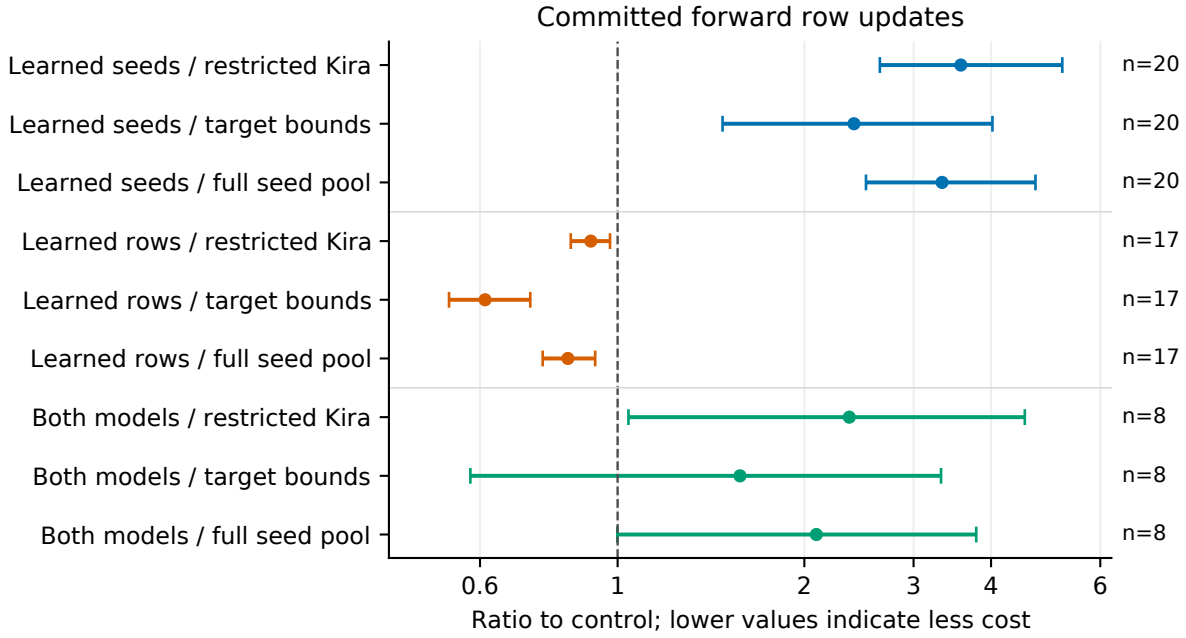




Figure 23: Generated-equation and forward-update ratios on complete paired batches. The controls and cohort labels follow Fig. 22. These counters measure reported generation work and committed updates, while symbolic coefficient complexity can change the cost of each update.

## 13.8 Numerical inference and scope of the result

The compiled-model development audit checked 256 action feature vectors and 2,247 sampled coefficients. The maximum feature difference was 0, and the largest probability difference from the independent double-precision calculation was $1.41 \times 10^{-13}$. All selected sources agreed with the original single-precision model in these development trials. The separate maximum probability difference from that original arithmetic was $1.6 \times 10^{-05}$.

The largest sampled process-tree memory was 0.506 GB, and the largest trial scratch directory was 49.29 MB. The bounded runs kept their scientific records and model files on Hoffman; the report contains the derived plots and results.

The complete paired comparisons do not show a lower aggregate runtime than restricted native Kira for these learned methods. Lower inference costs alone do not ensure that the chosen seeds and rows save enough algebraic work. The new native tests cover one-loop boxes and do not establish performance on independent high-loop scattering families. The existing weights were trained to recover recorded seed sets and row choices, so further cost-oriented training on native Kira trajectories remains a separate experiment.

## 13.9 Transfer to a two-loop propagator family

We also applied the unchanged interfaces to the massless two-loop propagator family used in the earlier cost comparisons. Its five independent family indices fit the compiled row interface. We fixed two batches of 16 and 64 stored target expressions and all policies before this transfer calculation. Each method uses five rotated repetitions, the same master basis and the same time and memory limits. Storage-interrupted trials resume on their original node, with earlier observations preserved. Because this family appeared in earlier comparisons, this experiment measures transfer of the revised implementation and does not establish generalization to a newly unseen family.

Alongside native seeding and the full candidate pool, we include a deterministic source-row control using the same compiled reduction structure. This control selects a source by row length and integral ordering within the same changing leading-column group. Comparing the learned row policy with this control separates the source choice from changes to the compiled reduction structure. Every completed reduction receives the same exact coefficient check and an independent finite-field check.

The transfer experiment contains 60 measured trials among 60 planned records, with 60 exact completions. The independent audit performed 7,200 target-coefficient checks and 2,355 row-update checks.

For 16 targets, the native control's median complete time was 12.003 seconds. The learned seed filter gave a time ratio of 0.992 with descriptive interval $[0.962, 1.030]$; the learned source-row policy gave a time ratio of 0.991 with descriptive interval $[0.984, 1.022]$; the combined policy gave a time ratio of 0.976 with descriptive interval $[0.961, 0.996]$. In the matched controls, the learned seed filter relative to the full pool had ratio 1.051 with interval $[1.024, 1.072]$ and the learned source choice relative to the deterministic compiled control had ratio 0.987 with interval $[0.979, 1.034]$.

For 64 targets, the native control's median complete time was 12.770 seconds. The learned seed filter gave a time ratio of 1.758 with descriptive interval $[1.744, 1.777]$; the learned source-row policy gave a time ratio of 0.976 with descriptive interval $[0.964, 0.992]$; the combined policy gave a time ratio of 1.755 with descriptive interval $[1.740, 1.777]$. In the matched controls, the learned seed filter relative to the full pool had ratio 1.907 with interval $[1.880, 1.920]$ and the

learned source choice relative to the deterministic compiled control had ratio 0.987 with interval $[0.952, 1.012]$.

The compiled learned-row pipeline reduced the median complete time for 64 targets from 12.770 to 12.460 seconds, a measured reduction of 2.4%. Its ratio against the deterministic compiled control has an interval that includes one, so this experiment does not resolve a benefit caused by the learned source choice itself. The full-pool control was faster, at 11.773 seconds. For 16 targets, the combined pipeline reduced the native median by 2.4%, but the full-pool control again completed sooner. For 64 targets, every learned-seed and combined trial required fallback, whose cost is included in the reported time. These observations establish limited gains relative to the stated native baseline and also identify stronger deterministic controls.

Figure 24 shows all complete paired comparisons. A ratio below one denotes a lower median complete time. The intervals come from 10,000 paired resamples of the five repetitions and describe repeat-to-repeat variation within this one family. They do not measure uncertainty across different integral families. Incomplete pairs remain in the completion counts and receive no timing ratio [28].

### 13.10 Applying the seed model to complete target expressions

The native interface reduces each component of Eq. (92) before combining the result. We also applied the fast seed model directly to that complete expression, using Kira's supplied-equation mode. This tests the form of the target used to train the seed model and allows cancellations between its components during the reduction.

Let $\mathcal{E}_{\boldsymbol{b}}$ be the stored IBP identities generated from candidate seed $\boldsymbol{b}$, and let $\mathcal{E}_{\mathrm{bg}}$ contain the stored symmetry and other background identities. The selected equation set is

$$\mathcal{E}(\mathcal{S}) = \mathcal{E}_{\mathrm{bg}} \cup \bigcup_{\boldsymbol{b}\in\mathcal{S}} \mathcal{E}_{\boldsymbol{b}}. \tag{99}$$

For each requested expression, we add its defining relation

$$\mathcal{T}_\nu - \sum_{\boldsymbol{a}\in\mathcal{J}_\nu} c_{\nu\boldsymbol{a}} I_{\boldsymbol{a}} = 0. \tag{100}$$

Kira eliminates the supplied system with the target expressions ordered above their component integrals and the prescribed masters preferred at the end of the ordering. It therefore obtains the master expansion of $\mathcal{T}_\nu$ in one exact solve. The same public identity pool, target definitions and master basis enter every compared method.

The model takes the union of candidates whose score reaches $\tau = 0.01$, using the unchanged weights and the fast inference implementation. We tested this set both with and without an early finite-field master check. An incomplete selected system triggers a full-pool calculation within the remaining time allowance. The early check compares the remaining integrals with the prescribed master basis; exact final coefficient checks still determine whether the requested reductions agree with their recorded answers. A global master check can require more relations than a particular target combination needs.

These calculations begin with stored IBP identities, whose prior generation cost is excluded for every method. The measured complete time includes loading those inputs, prediction, equation selection and export, Kira's exact calculation, admission checks when used, failed attempts, fallback and final verification. Thus the result concerns repeated reduction from a common identity pool. It does not measure native generation of a new family's equations.

The separate development and adapter-validation collection contains 80 trials, with 74 exact completions and 6 incomplete calculations. Independent checks evaluated 2,013 target coeffi-

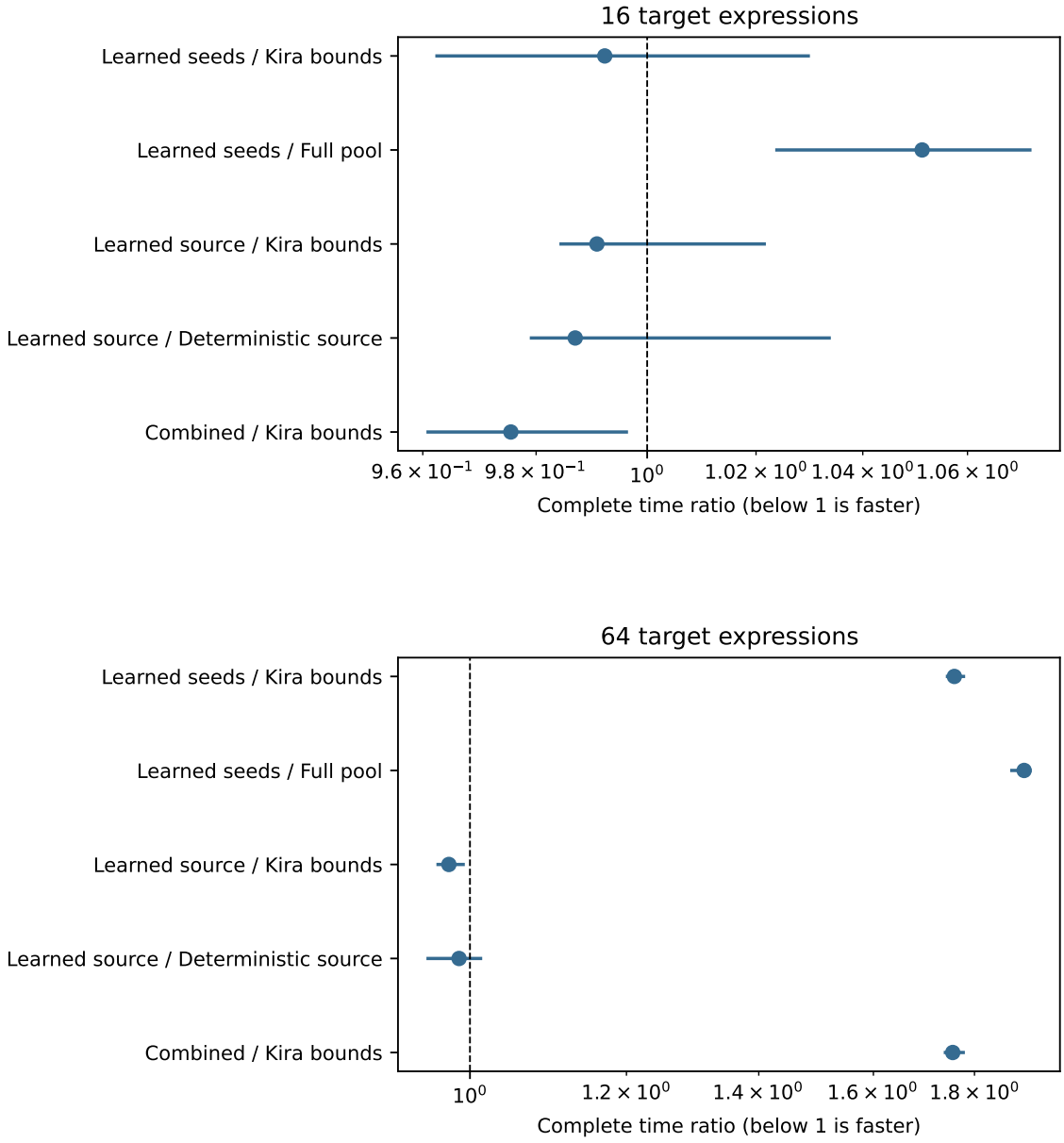



Figure 24: Complete-time ratios in the two-loop transfer comparison for 16 targets (upper panel) and 64 targets (lower panel). Points show ratios of five-repeat medians, and bars show descriptive paired-repeat intervals. All times include prediction, target preparation, exact reduction, combination, checks and any required fallback.

cients at stored finite-field points. The six adapter-validation family instances belong to the original training partition and use two batch sizes, one and sixteen targets. We retained every measured cost and selected a learned interface only when its geometric mean complete-time ratio to the full-pool control was below one and it preserved every control completion.

The validation ratio without the early master check was 1.113 across 11 complete paired batches. The full-pool control completed 0 additional batches that this learned interface did not complete. These single validation measurements select the interface and do not establish a repeatable timing gain.

The validation ratio with the early master check was 0.930 across 11 complete paired batches. The full-pool control completed 0 additional batches that this learned interface did not complete. These single validation measurements select the interface and do not establish a repeatable timing gain.

## 13.11 Reserved tests of complete-expression reduction

Validation selected the early master check. We froze this choice before applying it to the twelve original-held-out box instances used in the native comparison. The network weights and threshold remain fixed. Each instance supplies batches of one and sixteen target expressions, with five rotated repetitions per method on the same node. These are reused physical families and constructed targets; the new comparison tests the revised interface on original-held-out model data.

The controls use the full stored identity pool and a fixed set of 64 seeds ranked by their smallest index distance to any target component. The distance control uses the same exact reduction and full-pool fallback as the learned method. This first distance control proceeds directly to symbolic elimination, while the selected learned method uses the early master check. Their time ratio therefore includes both seed selection and the difference in admission checks.

The frozen plan contains 360 records. We executed 348 trials and obtained 345 exact completions; 3 measured trials failed to complete within the bounds, and 12 remaining repeats were skipped after an earlier failure. Every completed master expansion matched its exact reference, and the independent audit checked 8,460 target coefficients directly against the exported Kira reductions. Skipped repetitions have no assigned runtime.

The learned interface produced 115 exact completions, 1 failed trial and 4 skipped repeats. Completed trials used fallback 25 times.

The full-pool control produced 115 exact completions, 1 failed trial and 4 skipped repeats. Completed trials used fallback 0 times.

The distance control produced 115 exact completions, 1 failed trial and 4 skipped repeats. Completed trials used fallback 55 times.

We form a median from five successful repetitions for each method and batch, and take geometric mean ratios over complete paired batches. Descriptive 95% intervals resample the six kinematic groups 10,000 times, preserving the relationship between profiles and batch sizes. Completion losses remain separate from these ratios.

The learned interface has a complete-time ratio of 0.944 against the full-pool control, with descriptive interval $[0.863, 1.042]$ across 23 paired batches. The control completed all repeats in 0 additional batches, and the learned method completed all repeats in 0 additional batches.

The learned interface has a complete-time ratio of 0.604 against the distance control, with descriptive interval $[0.534, 0.678]$ across 23 paired batches. The control completed all repeats in

0 additional batches, and the learned method completed all repeats in 0 additional batches.

The observed geometric mean complete time is 5.6% lower than the full-pool Kira control on batches where both methods completed all five repetitions. Because the descriptive interval includes one, the aggregate timing gain across both batch sizes remains uncertain. The result applies to these constructed target expressions and their existing identity pools; it does not establish a speedup for general multiloop amplitude reductions [28].

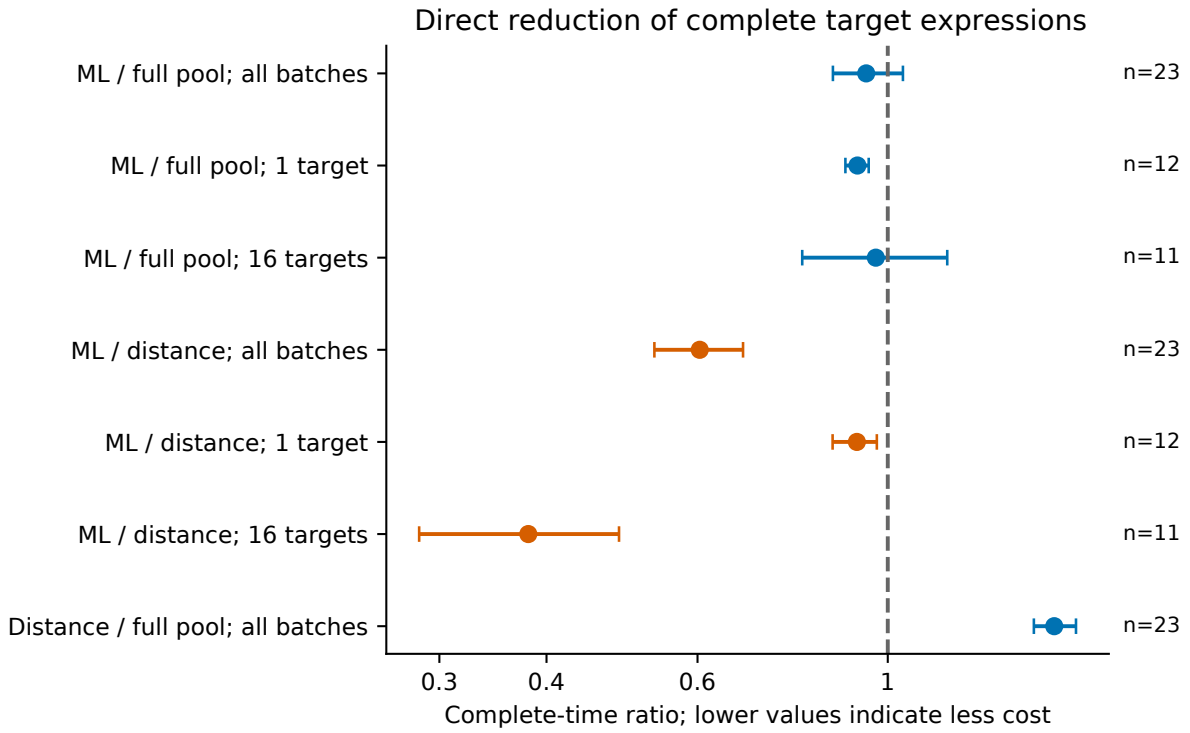



Figure 25: Complete-time ratios for direct reduction of complete target expressions. Points show geometric means of five-repeat median ratios, and intervals resample kinematic groups. The label $n$ gives the number of complete paired batches. Failed and skipped trials remain in the completion counts.

### 13.12 Using the same master check for both seed selectors

The preceding distance control enters symbolic elimination directly, while the learned method first checks the selected master basis. To compare seed choices under the same admission procedure, we fixed a second confirmation in which both selected sets receive the same early check and full-pool fallback. The learned scores, threshold, distance rule, twelve family instances and target batches stay unchanged. The full-pool Kira control again uses its standard supplied-equation reduction. We retained every measurement in the first comparison.

Each method again uses five rotated repetitions on the same node within a family and batch. This new comparison contains independent timing repetitions of the same physical cases, so it checks the effect of matching the admission procedure and does not introduce new integral families. Complete times and descriptive intervals follow the definitions in the preceding subsection.

The comparison with the same early check contains 360 records, of which 348 are measured trials. It produced 345 exact completions, 3 incomplete measured trials and 12 skipped repetitions. Independent checks evaluated 8,460 target coefficients against the raw Kira exports. All completed reductions matched their exact reference coefficients.

The learned method completed 115 trials exactly, with 1 failed trial and 4 skipped repetitions. Its completed trials required fallback 25 times.

The full-pool control completed 115 trials exactly, with 1 failed trial and 4 skipped repetitions. Its completed trials required fallback 0 times.

The distance control with the early check completed 115 trials exactly, with 1 failed trial and 4 skipped repetitions. Its completed trials required fallback 55 times.

Across both batch sizes, the learned complete-time ratio to the full-pool control is 0.932, with descriptive interval $[0.845, 1.031]$ across 23 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 1 target, the learned complete-time ratio to the full-pool control is 0.914, with descriptive interval $[0.871, 0.958]$ across 12 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 16 targets, the learned complete-time ratio to the full-pool control is 0.952, with descriptive interval $[0.772, 1.148]$ across 11 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

Across both batch sizes, the learned complete-time ratio to the distance control with the same early check is 0.925, with descriptive interval $[0.848, 1.024]$ across 23 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 1 target, the learned complete-time ratio to the distance control with the same early check is 0.913, with descriptive interval $[0.855, 0.972]$ across 12 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For batches of 16 targets, the learned complete-time ratio to the distance control with the same early check is 0.939, with descriptive interval $[0.764, 1.136]$ across 11 paired batches. The control completed 0 additional batches, while the learned method completed 0 additional batches.

For the predefined single-target comparison, the learned method reduced geometric mean complete time by 8.6% relative to the full pool and by 8.7% relative to the distance rule with the same early check. Both descriptive intervals lie below one, and every method completed all five repeats for all twelve instances. The first confirmation gave a 7.8% reduction relative to the full pool for this batch size. These results support a repeated gain for one constructed target expression at a time in this stored-identity setting. The sixteen-target intervals include one, so that batch size does not establish a consistent gain. The geometric means of the twelve per-family median complete times were 5.478 seconds for the full pool and 5.005 seconds for the learned method.

The learned workflow has an observed 6.8% lower geometric mean complete time than the full-pool Kira control on batches where both methods completed all five repetitions. Because the descriptive interval includes one, the aggregate timing gain across both batch sizes remains uncertain. These conclusions concern reduction from the supplied identity pools and the tested constructed target expressions. Native equation generation, arbitrary scattering amplitudes and larger independent multiloop families remain outside this result [28].

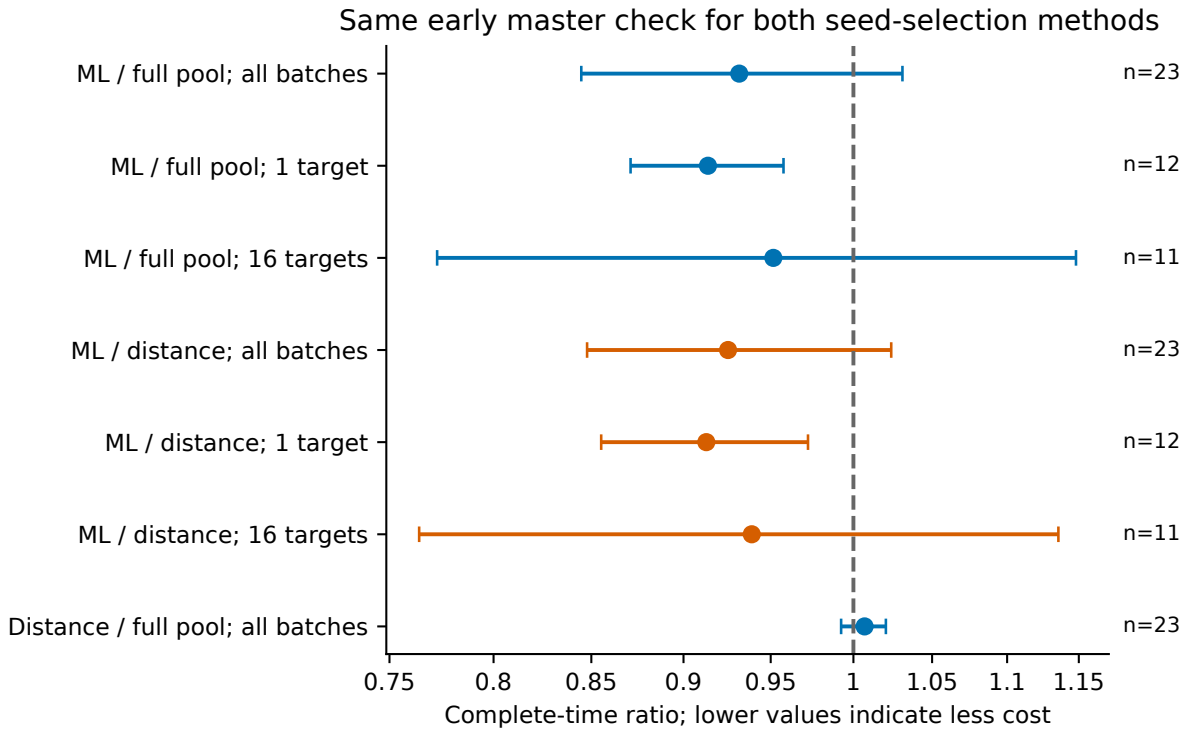



Figure 26: Complete-time ratios with the same early master check for learned and distance-selected seeds. Points show geometric means of five-repeat median ratios, intervals resample kinematic groups, and $n$ gives the number of complete paired batches. The full-pool control uses standard supplied-equation Kira reduction.

## 14 Conclusion

We constructed exact master-based integral data and tested learned policies for seed selection and row reduction. The collection contains 17,100 family instances, 57,979 family-local master entries and 2,301,272 targets with recorded identities, seed sets and reduction paths. These paths establish valid constructions and reference costs; they do not establish globally minimal reductions.

On the 690,382 held-out targets, the seed model reached 96.318% $F_1$, and its predicted stored equations supplied complete master-span reductions for 94.515% of targets. The full candidate pool supplied a sufficient fallback for all targets. The earlier sampled row test reproduced reference coefficients for 99.988% of targets and required 17.657 times the local rule's mean reduction time [19, 22, 24].

The 876 earlier cost-comparison trials all matched their exact master coefficients. Learned seed filtering reduced supplied equations to 0.251 times the full-pool count, while Kira's selected-equation count was 1.027 times the baseline. Complete and prepared-input time ratios were 1.720 and 1.116. Target-row preprocessing gave corresponding ratios of 1.719 and 1.205. Direct learned seed bounds gave ratios of 1.671 and 1.373 against restricted native Kira. These results include prediction, exact algebra, checks and required fallback [26].

The additional 84 trials place the frozen row policy inside Kira's evolving selected IBP matrix. Every final reduction passed its exact checks. The learned method took 1.305 times the complete stock-Kira time and used 5,771 committed coefficient operations against 5,070 for Kira's native policy. The forward-stage time ratio remained 62.270 after subtracting recorded model loading. The earlier implementations increased aggregate runtime, and this evolving-matrix policy also

increased the summed coefficient-operation count [27].

The new native integration filters individual IBP seeds and evaluates the source-row network inside Kira. Its complete-time ratios against restricted native Kira are 1.295, 1.024, 1.983, in seed, row and combined order. These ratios use 20, 21, 15 complete paired batches, respectively, with five repetitions per method. The complete paired comparisons do not show a lower aggregate runtime than restricted native Kira for these learned methods. Lower inference costs alone do not ensure that the chosen seeds and rows save enough algebraic work. All 590 completed trials matched their exact master coefficients [28].

The separate two-loop transfer gave 60 exact completions. The compiled learned-row pipeline reduced the median complete time for 64 targets from 12.770 to 12.460 seconds, a 2.4% decrease. Its advantage over the deterministic compiled source control remained unresolved, and the full-pool control was faster at 11.773 seconds. This limited native-baseline gain does not establish a general benefit from learned row choices.

Direct reduction of complete target expressions provides a different use of the seed model. The first confirmation contains 360 planned records and 345 exact completions. A second comparison applies the same early master check to learned and distance-selected seeds and contains 360 planned records and 345 exact completions. In this matched comparison, complete-time ratios are 0.932 against the full supplied-equation pool and 0.925 against the fixed distance control. The learned workflow has an observed 6.8% lower geometric mean complete time than the full-pool Kira control on batches where both methods completed all five repetitions. Because the descriptive interval includes one, the aggregate timing gain across both batch sizes remains uncertain. For the predefined single-target comparison, the learned method reduced geometric mean complete time by 8.6% relative to the full pool and by 8.7% relative to the distance rule with the same early check. Both descriptive intervals lie below one, and every method completed all five repeats for all twelve instances. The first confirmation gave a 7.8% reduction relative to the full pool for this batch size. These results support a repeated gain for one constructed target expression at a time in this stored-identity setting. The sixteen-target intervals include one, so that batch size does not establish a consistent gain. The geometric means of the twelve per-family median complete times were 5.478 seconds for the full pool and 5.005 seconds for the learned method. Prior generation of the common identity pool is outside these measurements [28].

The higher-loop collection varies mass profiles within restricted coupled vacuum families, and all tested loop orders occur in training. The earlier native comparisons cover seven independent-basis families at one and two loops, while the new reserved comparison covers twelve one-loop box instances in six kinematic groups. These results support conclusions within the tested domains, resource limits and action sets. Larger independent multiloop systems and cost-oriented training on their changing IBP matrices remain necessary to assess broader performance.

## Citations


[1] Stefan Weinzierl. Introduction to Feynman integrals, 2010. arXiv:1005.1855.

[2] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[3] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers*. UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[4] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate $\beta$-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[5] Thomas Gehrmann and Ettore Remiddi. Differential equations for two-loop four-point functions. *Nuclear Physics B*, 580:485–518, 2000. arXiv:hep-ph/9912329.

[6] A. V. Smirnov and A. V. Petukhov. The number of master integrals is finite, 2010. arXiv:1004.4199.

[7] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[8] Philipp Maierhöfer and Johann Usovitsch. Kira 1.2 release notes, 2018. arXiv:1812.01491.

[9] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection, 2025. arXiv:2505.20197.

[10] Philipp Kant. Finding linear dependencies in integration-by-parts equations: A Monte Carlo approach. *Computer Physics Communications*, 185:1473–1476, 2014. arXiv:1309.7287.

[11] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.

[12] Rushil Saraswat. Master-integral bases for sixteen one-loop and two-loop families. Accompanying family definitions, master bases and validation data, 2026.

[13] Rushil Saraswat. Target integrals constructed from master bases with seed and elimination records. Accompanying target integrals, source identities and validation data, 2026.

[14] Rushil Saraswat. Master bases and constructed targets for one thousand integral families. Accompanying family definitions, master bases, target reductions and validation data, 2026.

[15] Rushil Saraswat. Coupled vacuum master bases and reverse targets through twelve loops. Family definitions, source identities, grouped splits and independent checks, 2026.

[16] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1703.06114.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1706.03762.

[18] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101.

[19] Rushil Saraswat. Seed-selection model after twenty expanded-data passes: Held-out tests through twelve loops. Frozen predictions, metrics, plots and validation records, 2026.

[20] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

[21] scikit-learn developers. Average precision score: Definition and treatment of score thresholds. scikit-learn 1.5 documentation, 2024.

[22] Rushil Saraswat. Predicted-seed sufficiency through exact master-span tests in Kira. Frozen protocol, selected identities, symbolic results and independent checks, 2026.

[23] Rushil Saraswat. Reduction-propagation model trained on integral families through twelve loops. Model, optimizer lineage, state features and training records, 2026.

[24] Rushil Saraswat. Expanded-data reduction model: Held-out next-step and complete-path tests. Frozen predictions, exact row replays, plots and validation, 2026.

[25] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[26] Rushil Saraswat. Exact reduction costs with learned seeds, learned rows and native Kira baselines. Frozen cases, exact reductions, repeated costs and independent checks, 2026.

[27] Rushil Saraswat. Frozen learned row choices inside the evolving symbolic matrix of Kira. Source changes, frozen protocol, exact row records and repeated comparisons, 2026.

[28] Rushil Saraswat. Learned seed selection and row choices in exact Kira reductions. Frozen policies, native and supplied-equation comparisons, and exact checks, 2026.